

ETAS RTA-A2L 12.7.0



RTA-A2L Reference Guide

Copyright

The data in this document may not be altered or amended without special notification from ETAS GmbH. ETAS GmbH undertakes no further obligation in relation to this document. The software described in it can only be used if the customer is in possession of a general license agreement or single license. Using and copying is only allowed in concurrence with the specifications stipulated in the contract.

Under no circumstances may any part of this document be copied, reproduced, transmitted, stored in a retrieval system or translated into another language without the express written permission of ETAS GmbH.

© Copyright 2025 ETAS GmbH, Stuttgart

The names and designations used in this document are trademarks or brands belonging to the respective owners.

ETAS RTA-A2L 12.7.0 - RTA-A2L Reference Guide R01 EN – 05.2025

Contents

1	Introduction.....	5
1.1	Intended Use	5
1.2	Target Group	5
1.3	Safety Information	5
1.4	Document Conventions	5
1.5	Definitions and Abbreviations.....	6
2	Invocation.....	7
2.1	Command Line Usage.....	7
2.2	Output Files.....	7
2.3	Error and Information Messages	8
2.4	Exit Code	8
3	Command Line Options	9
3.1	Basic Examples	9
3.2	RTA-A2L Options	9
3.3	Important Points	13
3.4	Sample Options File	14
4	Using RTA-A2L	15
4.1	Translating MCSD to A2L.....	15
4.2	Translating Zero A2L to A2L.....	21
4.3	Translating ELF/ADX to A2L.....	25
5	RTA-A2L Generator	28
5.1	Introduction	28
5.2	A2L Generation Overview.....	28
5.3	File Generations Mode.....	28
5.4	RTA-A2L Template Handling.....	29
5.5	Input File Content and Processing	30
5.6	< MC-DATA-INSTANCE > Supported Categories.....	34
5.7	A2L File Contents.....	57
5.8	Generation of a Mapping ADX File	119
5.9	Restrictions	124
6	Non-AUTOSAR A2L Generation.....	137
6.1	Introduction	137
6.2	A2L Generation Overview	137

6.3	A2L File Contents	138
6.4	Restrictions	148
7	Address Extractor (Address Dumper)	150
8	A2LHeader Module	151
8.1	Introduction	151
8.2	Input Files	151
8.3	List of APIs Supported by A2LHeader	151
9	A2L Merger Module	156
9.1	Fetching Address for an A2L Element	156
9.2	ADXProcessor	156
9.3	Merging the A2L	157
9.4	Namespace Validations	158
9.5	Merger for CPFlex Use Case	159
10	File Formats	164
10.1	A2L Output File	164
10.2	MCSD File	164
10.3	Symbols File	164
10.4	Template File	164
10.5	ADX File	164
11	ETAS Contact Addresses	167
11.1	Technical Support	167
11.2	ETAS HQ	167

1 Introduction

RTA-A2L is a command-line tool that enables the creation of A2L files i.e. structured ASCII text data files used in the measurement and calibration of ECUs. Specifically, it generates an A2L file snippet out of AUTOSAR 4.x components.

The format of A2L files (*.a2l) that describe the internal ECU variables used in measurement and calibration is defined by ASAM MCD-2 MC (aka ASAP2).

Measurement and calibration systems (MC-systems) require this descriptive format for the parameterization of scalar constants, curves and maps of the ECU software and for recording the system's response via measurement variables during real-time testing.

The description contains information about data types, dimensions, record layouts and the memory locations of ECU variables. It also describes how the variable values are converted into human-readable quantities and displayed in an MC-system.

A2L files, along with Hex files, are loaded into calibration tools (e.g. INCA) and calibration is then performed by the calibration engineers.

1.1 Intended Use

This document describes in detail the underlying principles of RTA-A2L and its features.

1.2 Target Group

This document is for ETAS RTA-CAR users who want to generate A2L files for AUTOSAR components.

1.3 Safety Information

See the RTA-CAR User Guide and the *ETAS_Safety_Advice.pdf* supplied with the RTA-CAR installation package for additional safety information.

1.4 Document Conventions

OCI_CANTx	Message msg0 Code snippets are shown in a monospaced Inconsola typeface.
Choose File > Open	Menu commands are shown in boldface.
Click OK	Buttons are shown in boldface.
< ANNOTATION >	ARXML tags are shown in angled brackets.
The "Open File" dialog box is displayed	Names of program windows, dialog boxes, fields, etc. are shown in quotation marks.
Select the file <i>setup.exe</i>	Text in drop-down lists on the screen, program code, as well as path and file names are shown in italics.
A <i>distribution</i> is always a one-dimensional table of sample points.	General emphasis and new terms are set in italics.

1.5 Definitions and Abbreviations

2 Invocation

The RTA-A2L Generator is a Win64 executable that creates the AUTOSAR A2L file. RTA-A2L takes one or more *McSupportData ARXML*-based configuration files as input.

2.1 Command Line Usage

The RTA-A2L Generator is invoked from the command line as follows:

 **NOTE**

RTA-A2L is used as the executable in Windows and Linux.

```
RTA-A2L -vm <java21.exe> --optionsfile <optionsfile>
```

or

```
RTA-A2L -vm <java21.exe> --optionsfile [options]
```

Command line options must be specified using long names (the supported options are listed below).

Any number of *ARXML* input files can be specified.

When a command line option takes an argument, the argument must be specified as a trailing word. For example, an option *opt* with argument *arg* must be specified as `--opt arg`.

The ordering of command line parameters is unimportant: options and *ARXML* files can be mixed freely. Command line options are read left-to-right and are processed before any input files are read.

(See Section 3.2 for a reference to all RTA-A2L command-line options.)

2.2 Output Files

Each execution of the RTA-A2L Generator creates a single AUTOSAR A2L file as output.

The table below describes the output files created by the RTA-A2L Generator:

File	Description
<i>RTA-A2L.a2l</i>	AUTOSAR A2L file.
<i>RTA-A2L_tool.txt</i>	Tool log file.

The following files are optional - whether or not they are generated depends on the configuration of the RTA-A2L Generator:

File	Description
<i>a2l_user_log.txt</i>	User log file.
<i>Mapping.adx</i>	Mapping file which provides mapping information of C and A2L labels.

2.2.1 Redirecting Output

Output paths must be specified for all the output options. e.g.:

- The RTA-A2L Generator can be directed to write the A2L file to folder *abc* using the option:

```
--pathToA2LFile abc\ara2l.a2l
```

- The RTA-A2L Generator can be directed to write the Mapping ADX file to folder *abc* using the option:

```
--pathToMapAdxFile abc\mapping_AR.adx
```

By default, the tool log file is generated in the current working directory - this is typically the root directory where RTA-A2L executed and where all output files are sent.

- The RTA-A2L Generator can be directed to write the tool log file to folder *abc* using the option:

```
--a2l_tool_log abc\rt-a2l_tool.txt
```

2.3 Error and Information Messages

The RTA-A2L Generator uses status messages to inform on progress. The messages have the following classification:

<i>Fatal</i>	The detected error prevents further processing and the A2L Generator terminates immediately. No A2L or associated files are generated.
<i>Error</i>	The detected error is serious but may or may not prevent further processing and generation of the A2L file.
<i>Warning</i>	The detected error does not prevent further processing. The A2L and associated files will be generated but should not be considered as correct until the warning(s) have been investigated.
<i>Information</i>	A status message that does not indicate an error.

2.4 Exit Code

RTA-A2L uses its exit status code to pass failure conditions to the calling environment:

- 0: Success i.e the Generator completed without fatal errors. There may have been errors, warnings, or other information messages.
- Non-zero: Failure i.e. at least one fatal error occurred.

3 Command Line Options

The RTA-A2L Generator is controlled by command line options. All options begin with “-” and parameters are specified either on the command line or in an options file.

If the options file name contains spaces, it must be quoted according to the rules of the Windows command line environment:

```
RTA-A2L -vm <java21.exe> --optionsfile [options] "input filename.arxml"
```

3.1 Basic Examples

- To display the RTA-A2L product versions using long-form option names:

```
RTA-A2L -vm <java21.exe> --version
```

- To display the RTA-A2L help using long-form option names:

```
RTA-A2L -vm <java21.exe> --help
```

- To display the RTA-A2L copyright using long-form option names:

```
RTA-A2L -vm <java21.exe> --copyright
```

3.2 RTA-A2L Options

AUTOSAR defines certain configuration settings within the *RteGeneration* container that can also be specified on the RTA-A2L command line:

All options to start with...	Value
--mcsupportfilelist	(filePath: .\McSupportFilelist.txt) The path of the <i>McSupportFilelist.txt</i> . (This <i>McSupport-FileList.txt</i> file is a text file containing the paths of the input <i>McSupport</i> files).
--pathToA2LFile	(filePath: ._gen\main.a2l) The path of the output A2L file.
--pathToMapAdxFile	(filePath: ._gen\address_mapping.adx) The path of the output ADX file. This argument is optional. It is used only when there is a need to generate a mapping ADX file.
--a2l_user_log	(filePath: ._log\a2l_user.txt) Path of the user log file.
--a2l_tool_log	(filePath: ._log\a2l_tool.txt) Path of the tool log file.

--enableStaticRecordLayouts	(y/n) This switch is used to enable or disable the generation of static <i>RecordLayouts</i> in the A2L file generated by RTA-A2L. Its default value is <i>FALSE</i> . The switch has a dependency on the <i>asap2Version</i> switch but, if it is used, then preference will be given to the supplied value. If <i>enableStaticRecordLayouts</i> is not used but <i>asap2Version</i> is v1.61, then all the <i>RecordLayouts</i> are generated as <i>STATIC</i> .
--asap2_version	(String) This switch is used to specify the <i>ASAP2_VERSION</i> of the A2L file to be generated by RTA-A2L. The default <i>ASAP2_VERSION</i> is v1.4 i.e. major version 1, upgrade number 4. In the options file, the key <i>asap2Version</i> should hold both the major version and upgrade number e.g. v1.61.
--language	(EN) Key to select the language for the <i>LongIdentifier</i> creation.
--enabletimestampinlogs	(y/n) This switch is used to specify if the user wants time stamps to be generated or not. If set to "y", time stamps are generated. By default, the time stamps are disabled.
--enableStaticAddressOffsets	(y/n) This switch is used to enable/disable the generation of <i>STATIC_ADDRESS_OFFSETS</i> in <i>RecordLayouts</i> in the A2L file. The default value is <i>FALSE</i> . There is a dependency with the <i>asap2Version</i> switch. If <i>enableStaticAddressOffsets</i> is used, then preference will be given to the supplied value. In it is not used, but the <i>asap2Version</i> is v1.70 is then all the <i>RecordLayouts</i> will be generated with the <i>STATIC_ADDRESS_OFFSETS</i> attribute.
--enableMatrixDim	(y/n) <i>MATRIX_DIM</i> is generated in the A2L file instead of <i>Number</i> (for <i>CHARACTERISTICS</i>) and <i>ARRAY_SIZE</i> (for <i>MEASUREMENTS</i>).
--enablePrecisionForLimits	(y/n) This switch is used to enable or disable the generation of limits in A2L file. If the switch is set to "y", then the limits are generated as it is from an input <i>ARXML</i> file to get the limits with precision.

--enableSymbolLink	(y/n) This switch is used to enable or disable the generation of <i>SYMBOL_LINK</i> inside <i>MEASUREMENTS</i> , <i>CHARACTERISTICS</i> and <i>AXIS_PTS</i> . Also, <i>SYMBOL_LINK</i> is only generated for A2L files of ASAP_VERSION 1.60 or greater.
--prj_root	(<i>Absolute path of the project</i>) Allows absolute file paths to be built in RTA-A2L i.e. a concatenation of the <i>absolute path of the project</i> and the corresponding input file's relative file path. If the --prj_root is missing /invalid, then the following error message is thrown: "ERROR - absolute path can't be constructed for specified path as Parent directory does not exist/invalid. If there are only absolute paths in the supplied options file and in corresponding input file lists, then --prj_root value is not used by RTA-A2L".
--ifData_template_fileList	(<i>filePath: IfDataTemplates</i>)
--ma2l	The main/master A2L file path.
--internalARA2L_filelist	The text file path containing list of AUTOSAR A2L file paths.
--a2lheadertemplate	The path to the FTL template containing the A2L content file used for generating the .a2l file.
--genericfiletemplate	The path to the FTL template file used for generating the .h/.c file.
--generatedoutputfile	The output-generated file for the template file configured with the --genericfiletemplate option.
--adxfile	The path to the final ADX file generated by the Dumper and will be the input to the A2L header generator.
--addressupdate	The update addresses for the A2L Elements, along with the <i>IF_DATA</i> block. Will be set for the 0A2L to A2L use case.
--cluster_mapping_file	The list file to specify the mapping information of all the cluster a2l file, ADX and binary manifest files.
--mode	Productive - tool aborts immediately after it encounters errors.
--force	If an address is not found in the ADX/ELF /SYMBOLS file for a label, the address is considered as 0x0 for merging the corresponding block.

--a2l_out	The output A2L file name. If not specified, the input file (--ma2l) shall be overwritten.
--symbols	The map/txt which consists of addresses and C-label names (e.g. 0x2003000A element1).
--encoding_UTF_8	(./customer1.a2l, ./customer2.a2l, ./customer3.a2l) Path of input A2L files with encoding considered as UTF-8.
--encoding_US_ASCII	(./customer1.a2l, ./customer2.a2l, ./customer3.a2l) Path of input A2L files with encoding considered as US-ASCII.
--encoding_ISO_8859_1	(./customer1.a2l, ./customer2.a2l, ./customer3.a2l) Path of input A2L files with encoding considered as ISO-8859-1 (ANSI/WINDOWS-1252/CP-1252).
--enableCustomerA2LAlignment	(y/n) This switch is used to enable/disable the generation of alignment sizes from the <i>MOD_COMMON</i> block of the customer A2L file, and will be updated into the customer A2L record layout after merging.
--adcskipDupLabels	The switch to skip the elements that have the same label name but different absolute addresses, while merging the project and mapping ADX files. This is an optional switch. By default, the value = FALSE.
--custadllist	The list file which contains the absolute/relative paths of Customer and Core modules' .adx files. These files, in ADX format, contain offset and/or mapping information.
--adx_parameters	(./characteristics.txt) Path to input text file containing <i>CHARACTERISTICS</i> label names, which can include wildcards or patterns (*text*, text*, or * text) to match multiple labels.
--adx_signals	(./measurements.txt) Path to input text file containing <i>MEASUREMENTS</i> label names, which can include wildcards or patterns (*text*, text*, or * text) to match multiple labels.
--outputa2lfromADX	Output A2L file is generated using the ADX file.
--recLay_indexmode	(<i>ROW_DIR/COLUMN_DIR</i>) Sets the indexing mode for record layouts to either row direction (<i>ROW_DIR</i>) or column direction (<i>COLUMN_DIR</i>). The former is the default.

--expand_arrays	(y/n) This switch is used to enable/disable the generation of all array elements in an A2L file.
--byte_order	(<i>MSB_FIRST/MSB_LAST</i>) This switch specifies the byte order. For example, INTEL address format is <i>MSB_LAST</i> and Motorola is <i>MSB_FIRST</i> .
--format	This switch is used to specify the format for limit generation.
--output_utf8_BOM	(y/n) To generate the A2L file in UTF-8 format without BOM information, set the value to "n".
--log	The path of the user log file (e.g. <i>filePath: .\output\TextLog.txt</i>).
--toollog	The path of the tool log file (e.g. <i>filePath: .\output\tool.log</i>).
--adxoutput	The output of the ADX file path.
--compiler	HighTec

3.3 Important Points

- If there are relative paths in the options file and in the corresponding input file lists, it is mandatory to supply the command line argument --prj_root.
- If the *enableStaticAddressOffsets* switch is enabled, and *asap2Version* is set to "v1.60" or "v1.61" in the options file, then RTA-A2L would give priority to the *enableStaticAddressOffsets* switch and the *STATIC_ADDRESS_OFFSETS* attribute will be generated for the *RecordLayouts*.
- If the *enableStaticRecordLayouts* switch is enabled, and *asap2Version* is set to "v1.7" in the options file, then RTA-A2L aborts as this would be an invalid scenario.
- If *enableStaticRecordLayouts* and *enableStaticAddressOffsets* are both enabled in the options file, then RTA-A2L aborts because *STATIC_ADDRESS_OFFSETS* and *STATIC_RECORD_LAYOUT* together are not allowed in the *RECORD_LAYOUT*.

3.4 Sample Options File

```
--McSupportfilelist .\tmp\McSupportFilelist.txt
--pathToA2LFile .\output\main.a2l
--pathToMapAdxFile .\output\address_mapping.adx
--a2l_user_log .\log\a2l_user.txt
--a2l_tool_log .\log\a2l_tool.txt
--enableStaticRecordLayouts y
--prj_root .
--cut_a2l_longname_255 y
--enabletimestampinlogs n
--asap2_version V1_6
```

4 Using RTA-A2L

RTA-A2L typically runs in a two-step approach to generate an A2L file.

Step 1: RTA-A2L reads *MCSD* files and generates an AUTOSAR A2L snippet with elements having the 0x0 address.

Step 2: Typically, RTA-A2L reads an A2L template, AUTOSAR A2L snippet and *ELF* /*ADX*/*Symbols* to generate the merged A2L file with actual addresses. Various use cases are provided below:

4.1 Translating MCSD to A2L

An A2L file is generated by using the MCSD file generated from RTA-RTE, along with inputs like ELF and Symbols.

Compiler-generated ELF is provided to RTA-A2L to generate actual addresses in the A2L file. MCSD to A2L is a two-step approach in RTA-CAR, where the A2L Snippet and the final A2L are generated separately.

The list of options required to run MCSD to A2L (Symbols/ELF/ADX) is provided below:

Step 1:

```
--mcsupportfilelist (input)
--prj_root (input)
--pathToA2LFile (output)
--pathToMapAdxFile (output)
--asap2Version (Optional)
--a2l_tool_log (output)
```

Step 2:

```
--elffile (input)
--symbols (input)
--adxfile (input)
--prj_root (input)
--mode (input)
--compiler (input)
--custadllist (input)
--a2lheadertemplate (input)
--internalara2l_filelist (input)
--adxoutput (output)
--a2l_out (output)
--log (output)
--toollog (output)
--force (Optional)
--adcskipDupLabels (Optional)
```

Please refer to Section 3.2 for more details.

4.1.1 Translating MCSD to A2L, Zero Address Instantiation

The A2L file is generated with an address of 0x0. The AUTOSAR A2L file and A2L file template are merged to generate the final A2L file.

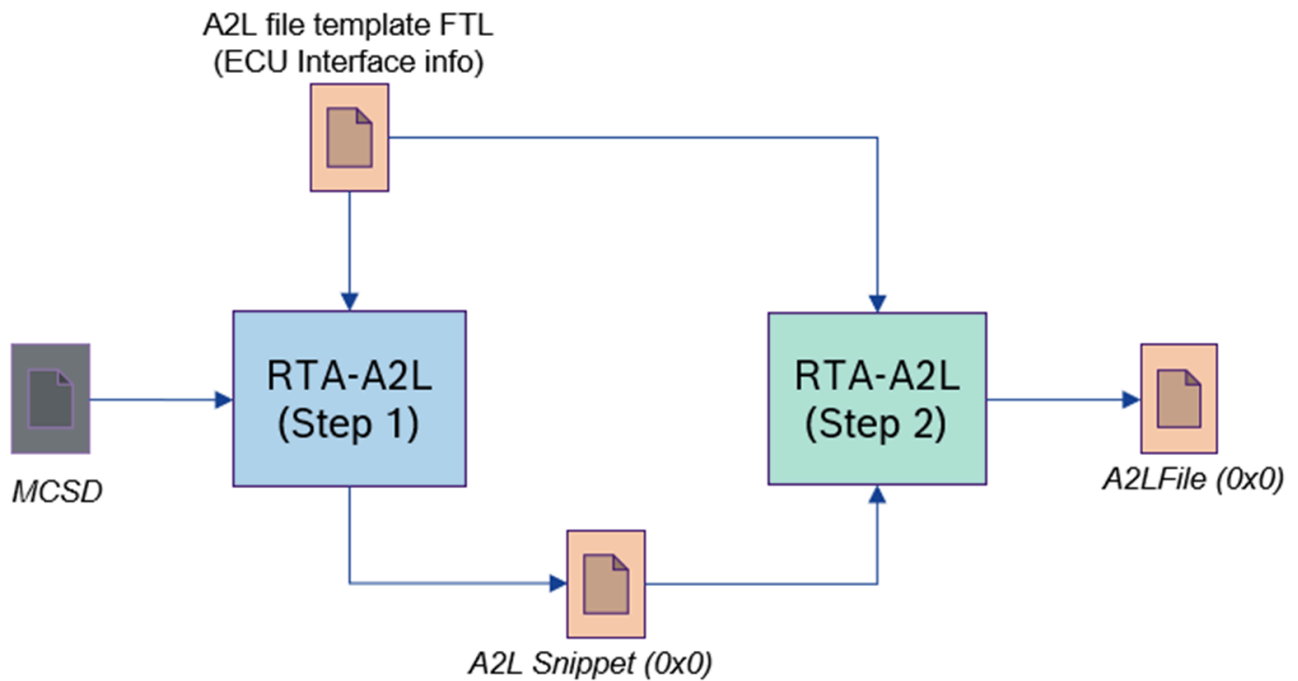


Figure 4.1. Translating MCSD to A2L, Zero Address Instantiation

4.1.2 Translating MCSD to A2L, with Address Instantiation from ELF

The MCSD file is provided to RTA-A2L, along with the ELF file, to generate an A2L file with addresses fetched from the ELF file. The ELF file is generated by the compiler, from which the addresses can be extracted through the *HighTec Dumper*.

During **Step 1**, the AUTOSAR A2L Snippet is generated with 0x0 addresses. In **Step 2**, the Dumper runs internally to generate an ADX file, where actual addresses are extracted from ELF. Later, the AUTOSAR A2L file and A2L file template are merged by replacing actual addresses to generate a final A2L file.

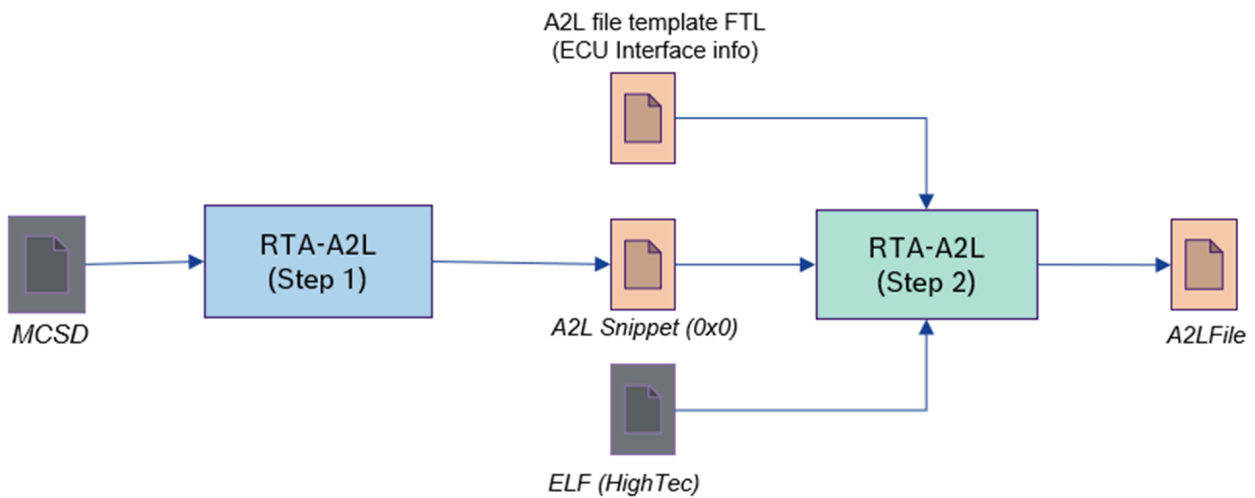


Figure 4.2. Translating MCSD to A2L, with Address Instantiation from ELF

4.1.3 Translating MCSD to A2L, with Address Instantiation from ADX

The MCSD file is provided to RTA-A2L, along with the ADX file, to generate an A2L file with addresses fetched from the ADX file. During **Step 1**, the AUTOSAR A2L Snippet is generated with 0x0 addresses. In **Step 2**, the AUTOSAR A2L file and A2L file template are merged by replacing actual addresses from the ADX file to generate a final A2L file.

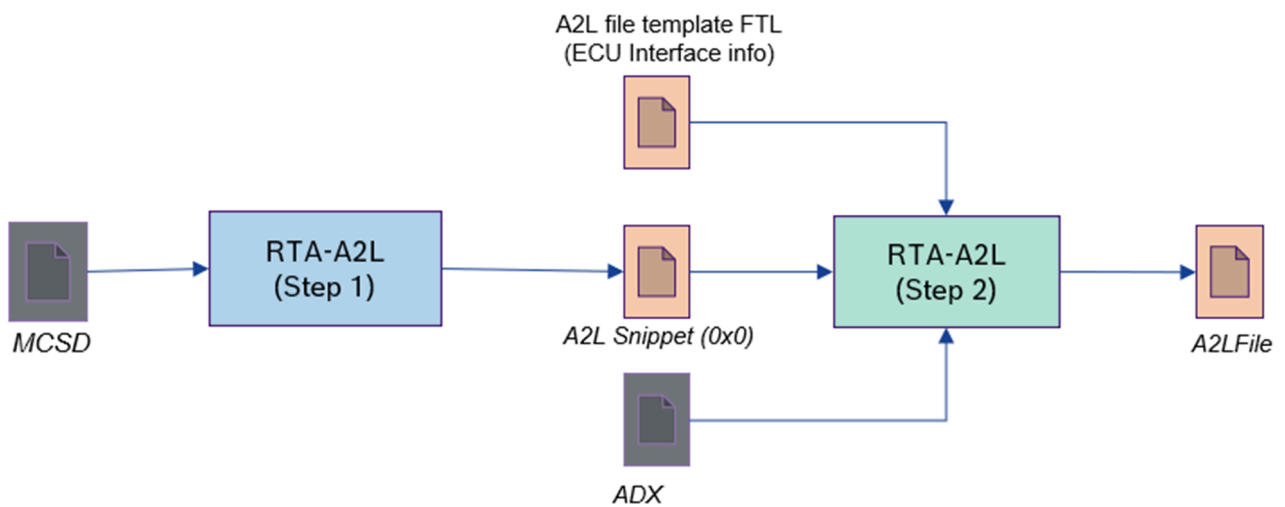


Figure 4.3. Translating MCSD to A2L, with Address Instantiation from ADX

4.1.4 Translating MCSD to A2L, with Address Instantiation from Symbols

The MCSD file is provided to RTA-A2L, along with Symbols, to generate an A2L file with addresses fetched from Symbols and it is run in one step only.

A2L file template FTL
(ECU Interface info)

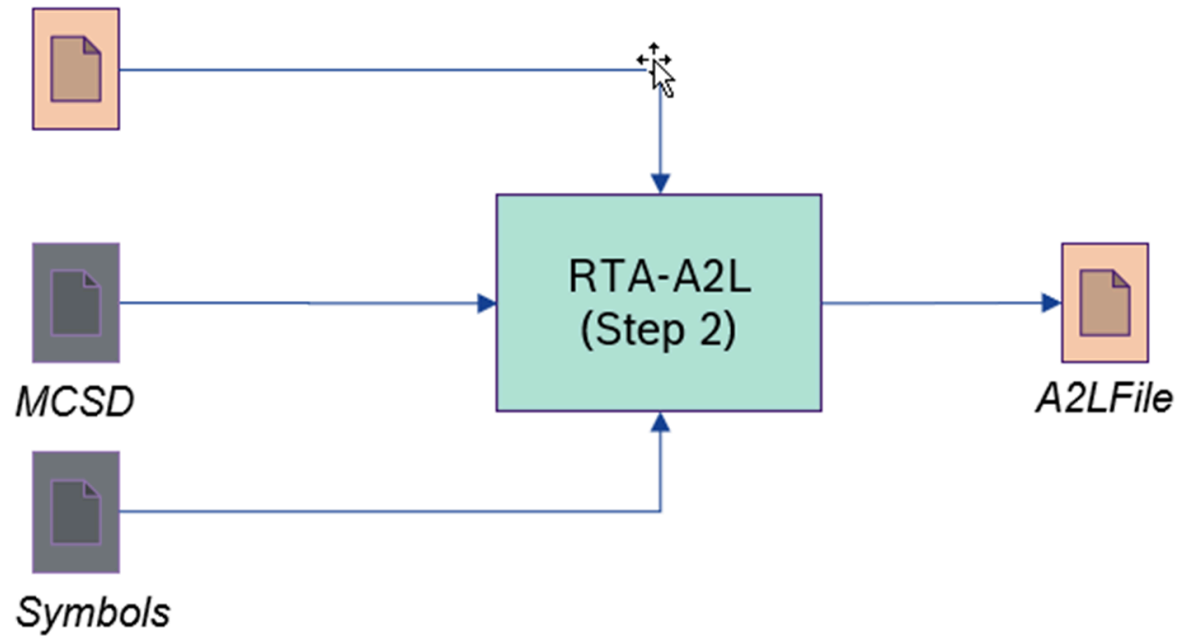


Figure 4.4. Translating MCS to A2L, with Address Instantiation from Symbols

4.1.5 RTA-A2L Code Generator steps for Translating MCSD to A2L

Step 1: RTA-A2L

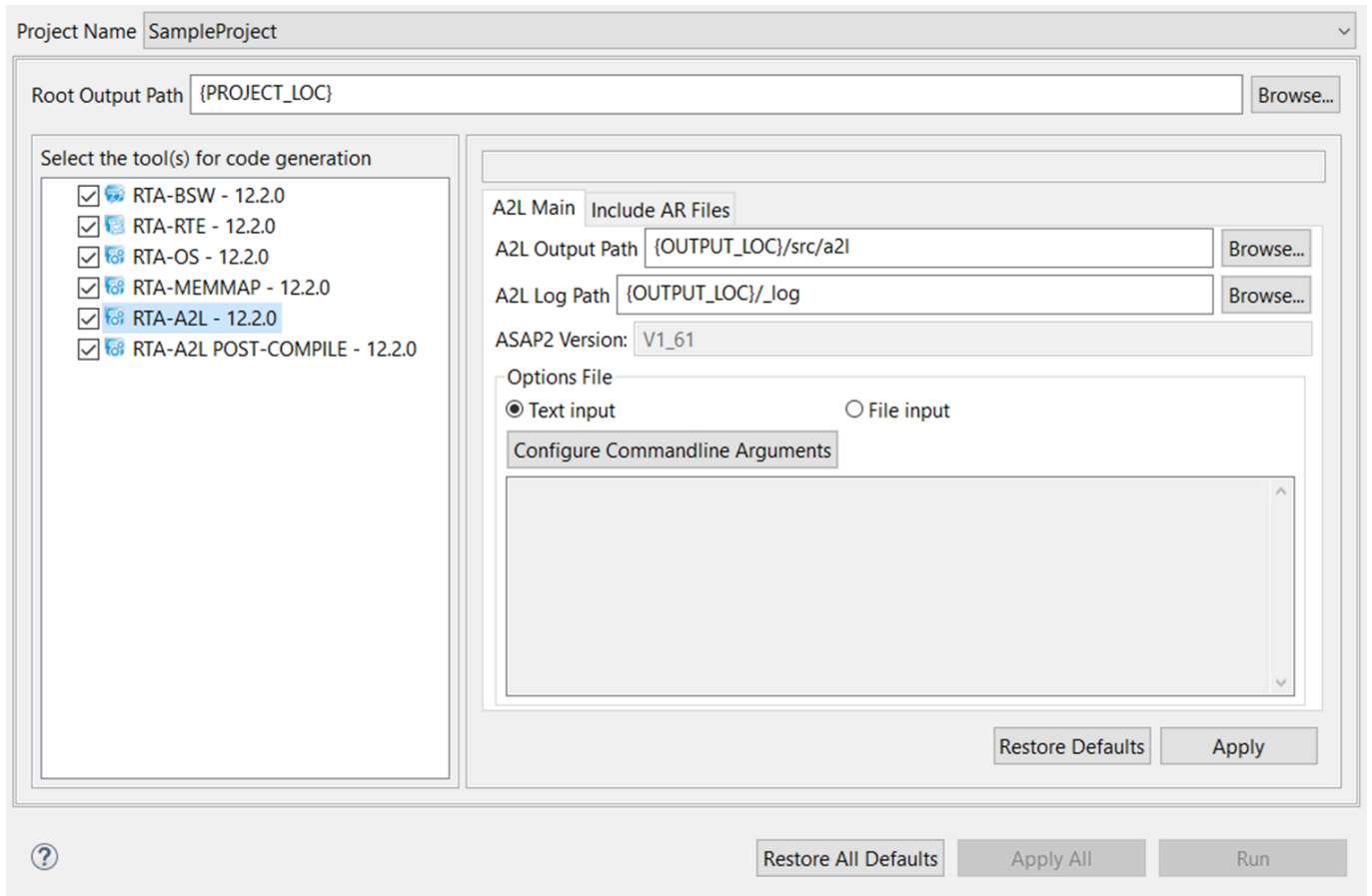


Figure 4.5.Step 1: RTA-A2L

Step 2: RTA-A2L POST-COMPILER

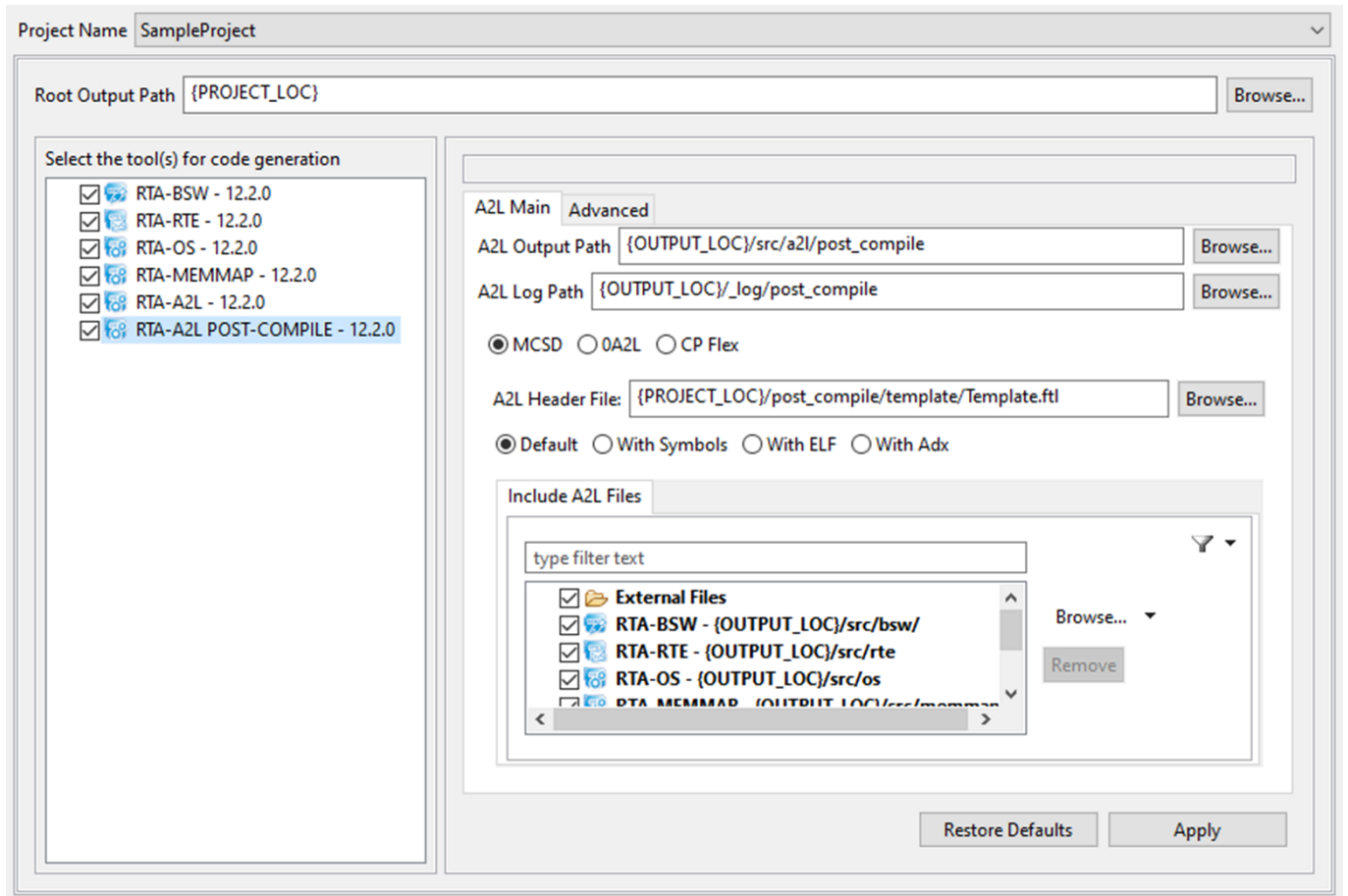


Figure 4.6.Step 2: RTA-A2L POST-COMPILER

4.2 Translating Zero A2L to A2L

The final A2L file is generated by using the Symbols/ELF/ADX file, which consists of addresses for the A2L blocks. Zero A2L to A2L involves only one step, where an A2L snippet is provided to RTA-A2L to generate the final A2L with actual addresses. The list of options required to run Zero A2L to A2L (Symbols/ELF/ADX) is provided below:

Step 2:

```
--elffile (input)
--symbols (input)
--adxfile (input)
--prj_root (input)
--mode (input)
--compiler (input)
--custadllist (input)
--a2lheadertemplate (input)
--internalara2l_filelist (input)
--adxoutput (output)
--a2l_out (output)
--log (Optional)
--toollog (output)
--force (Optional)
--adcskipDupLabels (Optional)
```

Please refer to Section 3.2 for more details.

4.2.1 Translating Zero A2L to A2L, with Address Instantiation from ELF

An A2L file with zero addresses is provided to RTA-A2L, along with the ELF file, to generate an A2L file with addresses fetched from the ELF file. The ELF file is generated by the compiler, from which the addresses can be extracted through the HighTec Dumper. In **Step 2**, the Dumper runs internally to generate an ADX file, where actual addresses are extracted from ELF. Later, the actual addresses are added to the final A2L file.

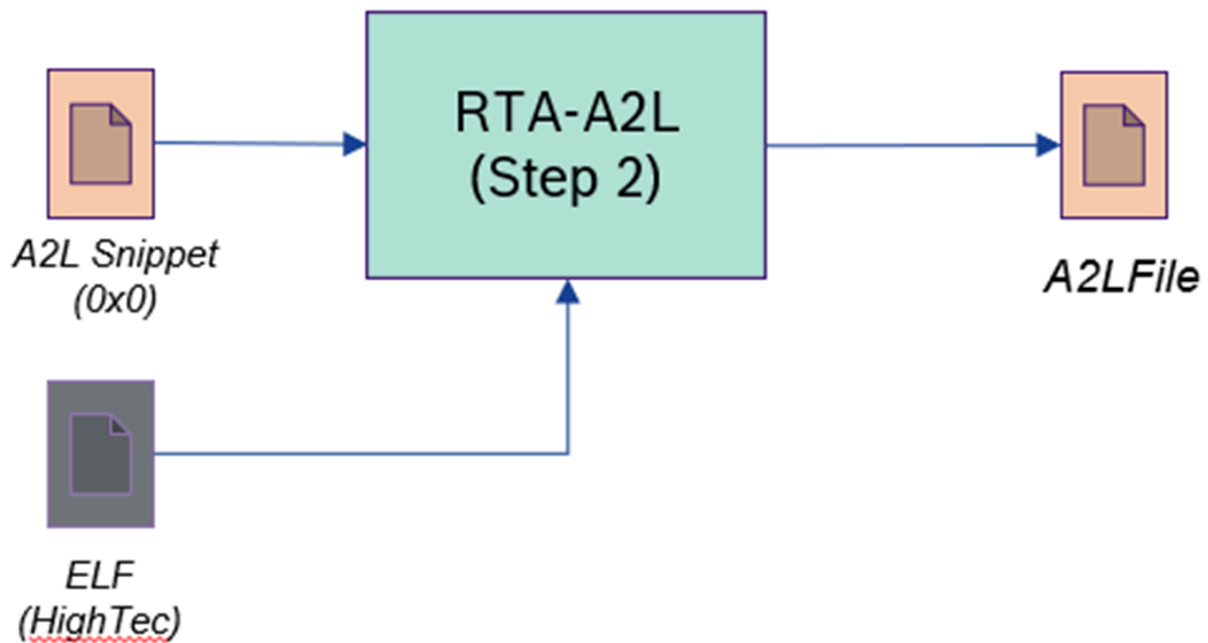


Figure 4.7. Translating Zero A2L to A2L, with Address Instantiation from ELF

4.2.2 Translating Zero A2L to A2L, with Address Instantiation from ADX

An A2L file with zero addresses is provided to RTA-A2L, along with an ADX file, to generate an A2L file with addresses fetched from the ADX file. The addresses are fetched from the ADX file and replaced in the final A2L file.

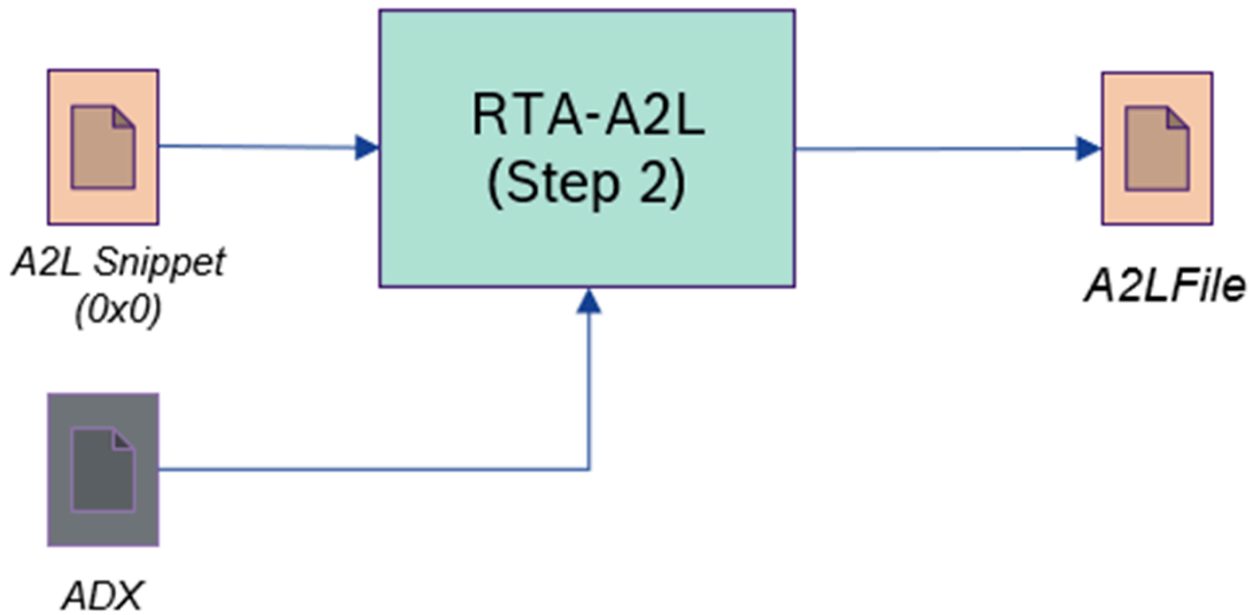


Figure 4.8. Translating Zero A2L to A2L, with Address Instantiation from ADX

4.2.3 Translating Zero A2L to A2L, with Address Instantiation from Symbols

An A2L file with zero addresses is provided to RTA-A2L, along with Symbols, to generate an A2L file with addresses fetched from Symbols.

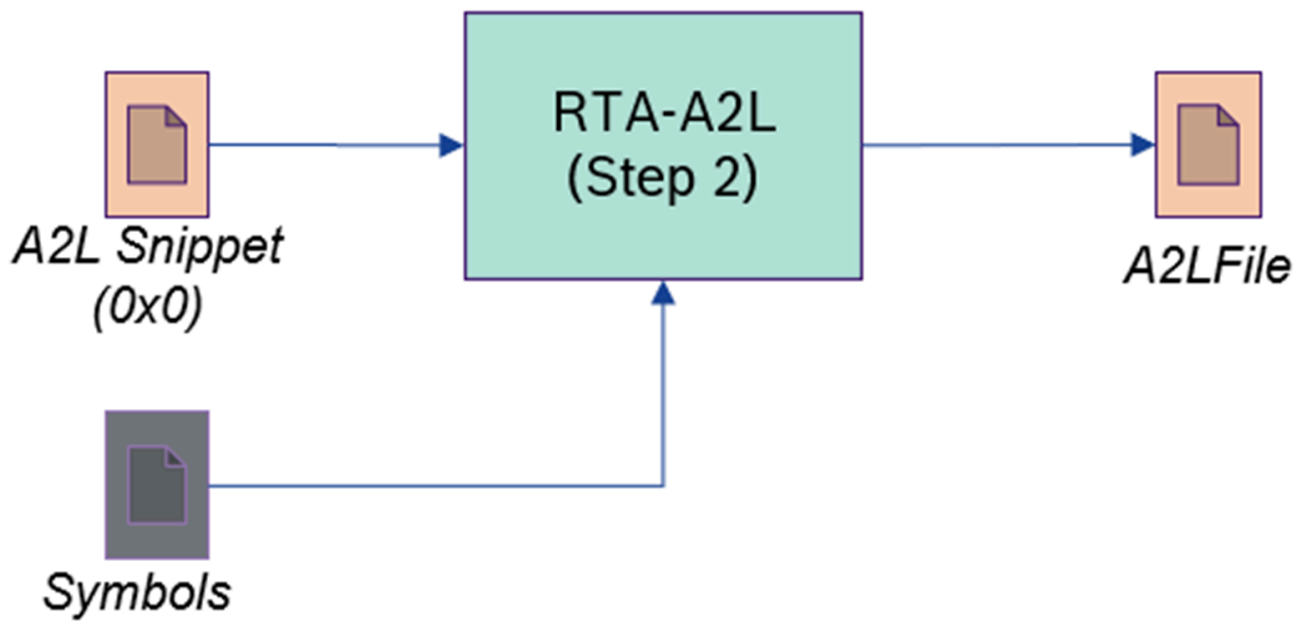


Figure 4.9. Translating Zero A2L to A2L, with Address Instantiation from Symbols

4.2.4 RTA-A2L Code Generator steps for Translating Zero A2L to A2L

Step 2: RTA-A2L POST-COMPILE

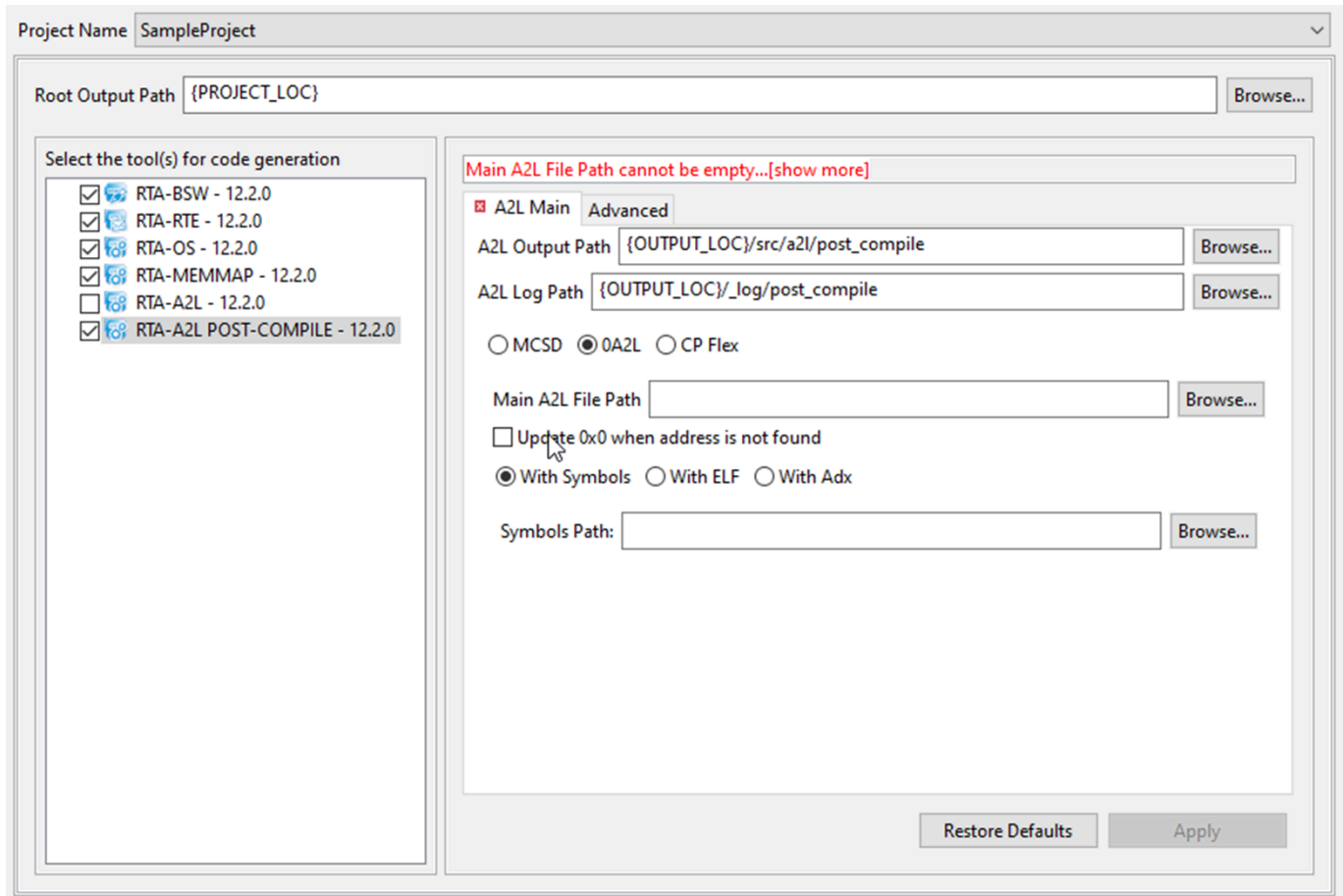


Figure 4.10. RTA-A2L POST-COMPILE

4.3 Translating ELF/ADX to A2L

4.3.1 Translating ADX to A2L

An A2L file is generated by using the ADX file generated from the *Address Dumper*, along with inputs like *adx_parameters/adx_signals*.

The list of options required to translate ADX to A2L is provided below:

Step 1:

```
--adx_parameters (input)
--adx_signals (input)
--adxfile (input)
--outputa2lfromADX (output)
--format (optional)
--expand_arrays (optional)
--byte_order (optional)
--recLay_indexmode (optional)
--asap2_version (optional)
--a2l_tool_log (output)
--prj_root (input)
```

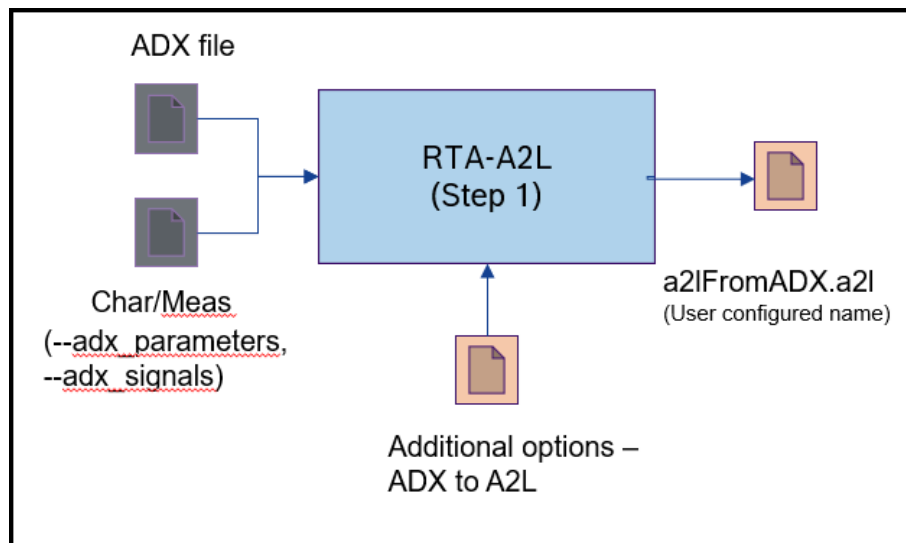


Figure 4.11. Translating ADX to A2L

4.3.2 Translating ELF to A2L

The parameter/signals list file is provided to RTA-A2L, along with the ELF file, to generate an A2L file with addresses fetched from the ELF file.

The ELF file is generated by the compiler, from which the addresses can be extracted through the *HighTec Dumper*. In **Step 2**, the Dumper runs internally to generate an ADX file, where the A2L content and actual addresses are extracted from ELF.

Step 2:

```
--adx_parameters (input)
--adx_signals (input)
--elffile (input)
--mode (input)
--compiler (input)
--custadllist (input)
--a2lheadertemplate (input)
--internalara2l_filelist (input)
--adxoutput (output)
--a2l_out (output)
--format (optional)
--expand_arrays (optional)
--byte_order (optional)
--recLay_indexmode (optional)
--asap2_version (optional)
--a2l_tool_log (output)
```

--prj_root (input)

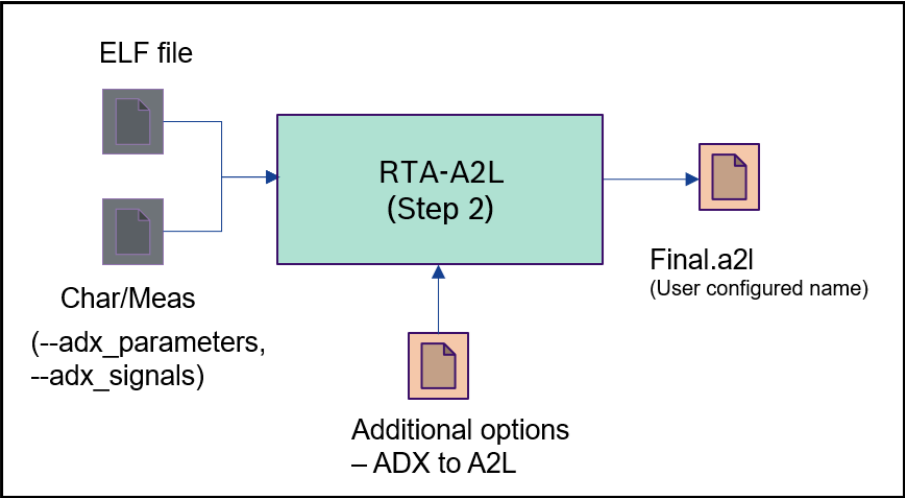


Figure 4.12. Translating ELF to A2L

5 RTA-A2L Generator

5.1 Introduction

RTA-A2L contains functionality to generate an A2L file snippet (valid as per ASAP specification with default addresses 0x0 for *CHARACTERISTICS* and *MEASUREMENTS*) out of AUTOSAR 4.x components.

The RTA-A2L Generator is developed as an Eclipse-based application. It can be used as a standalone tool (i.e. in headless mode). The Eclipse executable and an options file containing the required arguments will be used to invoke the RTA-A2L tool.

The input for the A2L and the ADX files will be provided from multiple *McSupport-*.arxml** files. These are AUTOSAR files and (mainly) contain *CHARACTERISTIC* and *MEASUREMENT* information relevant to the ECU software.

These input files can be listed and passed via an options file (a *.txt* file) to the RTA-A2L executable at the command line. The paths of the output artifacts i.e. the A2L file, ADX file and the log files are, alongside the input *McSupport* files, passed through the options file.

5.2 A2L Generation Overview

The A2L Generator is a standalone tool which creates the AUTOSAR A2L and mapping ADX files by processing the input AUTOSAR files.

Mapping ADX files are needed to fetch addresses created for source variable names during the ECU development phase. The following diagram shows the extension of A2L file generation for AR4 components.

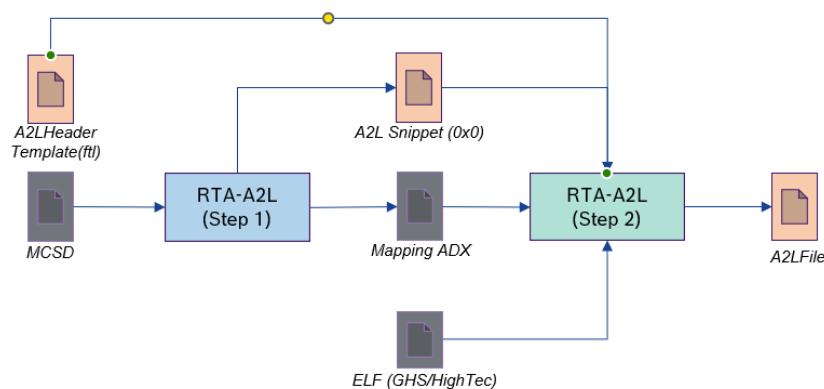


Figure 5.1.A2L file generation

5.3 File Generations Mode

If an A2L file already exists, File Generations Mode ensures that its contents are checked against those of the new file:

- If the contents are different, the A2L file is written again with the new

contents (obtained after processing the input files). The timestamp of the file will also be changed.

- If the contents are the same, the A2L file is not written again. The timestamp of the file will not change.

By default, the *INCREMENTAL* type of the A2L file generation is enabled.

5.4 RTA-A2L Template Handling

The RTA-A2L Generator uses FreeMarker templates to create the ADX file. These are provided with the tool i.e. the user need not supply them.

RTA-A2L uses the *ifDataBlock.ftl* template file for generating the *ETK* and *XCP ifData* blocks. It should be passed with the *CLI* option *--ifData_template_fileList*.

The user should add the name and the *dataRecord* for each *MEASUREMENT* and *CHARACTERISTIC* block in the form (below) to generate the *ifData* blocks for the corresponding *MEASUREMENTS/CHARACTERISTICS*:

```
<#list allMeasurements as measurement>
<#if a2LIfDataTemplateUtil.isCSECodeExists(measurement) && a2LIfDataTemplateUtil.isCSEFactorExists(measurement)>
<#assign newIfDataObject = a2LIfDataTemplateUtil.createIfData(measurement)>
<#assign cseCodeNormalised = a2LIfDataTemplateUtil.getCSECodeNormalized(measurement)>
<#assign cseCodeFactorNormalised= a2LIfDataTemplateUtil.getCSECodeFactorNormalized(measurement)>
<#if cseCodeNormalised != -1 && cseCodeFactorNormalised !=-1>
/*sample way to use the normalised csecode and the csecodefactor*/
<#if cseCodeNormalised == 1 && cseCodeFactorNormalised==1>
${a2LIfDataTemplateUtil.addIfData(newIfDataObject,"/begin IF DATA ETK DEFAULT_RASTERS 1 /end DATA line 1")}}
<#elseif cseCodeNormalised == 2 && cseCodeFactorNormalised==2>
${a2LIfDataTemplateUtil.addIfData(newIfDataObject,"/begin IF DATA ETK DEFAULT_RASTERS 2 /end DATA line 1")}}
/* Case where a 2 combinations leads to a single value*/
<#elseif (cseCodeNormalised==6 && cseCodeFactorNormalised==40) || (cseCodeNormalised==3 &&
cseCodeFactorNormalised==405)>
${a2LIfDataTemplateUtil.addIfData(newIfDataObject,"/begin IF DATA ETK DEFAULT_RASTERS 3 /end DATA line 1")}}
</#if>
</#if>
</#if>
</#list>
<#list allCharacteristics as characteristic>
<#if a2LIfDataTemplateUtil.isCSECodeExists(characteristic) && a2LIfDataTemplateUtil.isCSEFactorExists(characteristic)>
<#assign newIfDataObject = a2LIfDataTemplateUtil.createIfData(characteristic)>
<#assign cseCodeNormalised = a2LIfDataTemplateUtil.getCSECodeNormalized(characteristic)>
<#assign cseCodeFactorNormalised= a2LIfDataTemplateUtil.getCSECodeFactorNormalized(characteristic)>
<#if cseCodeNormalised != -1 && cseCodeFactorNormalised !=-1>
/*sample way to use the normalised csecode and the csecodefactor*/
<#if cseCodeNormalised == 1 && cseCodeFactorNormalised==1>
${a2LIfDataTemplateUtil.addIfData(newIfDataObject,"/begin IF DATA ETK DEFAULT_RASTERS 1 /end DATA line 1")}}
<#elseif cseCodeNormalised == 2 && cseCodeFactorNormalised==2>
${a2LIfDataTemplateUtil.addIfData(newIfDataObject,"/begin IF DATA ETK DEFAULT_RASTERS 2 /end DATA line 1")}}
/* Case where a 2 combinations leads to a single value*/
<#elseif (cseCodeNormalised==6 && cseCodeFactorNormalised==40) || (cseCodeNormalised==3 &&
cseCodeFactorNormalised==405)>
${a2LIfDataTemplateUtil.addIfData(newIfDataObject,"/begin IF DATA ETK DEFAULT_RASTERS 3 /end DATA line 1")}}
</#if>
</#if>
</#if>
</#list>
```

Figure 5.2.Sample *IfDataTemplate.ftl* file

5.5 Input File Content and Processing

The *McSupport* files will first be validated in the RTA-A2L Generator. Before loading the input file, therefore, its contents i.e. the paths of the *McSupport* files, are checked. If a file path is invalid, an error message will be logged in both the tool log file and the user log files and the A2L file generation will abort. (In the case of test mode, an error message will be logged and the RTA-A2L Generator will continue.)

5.5.1 McSupport File Handling

Once the *McSupport* file is loaded, it is checked for well-formedness and validated as per the specified AUTOSAR schema.

The RTA-A2L Generator then performs the following operations:

1. If the *McSupport* file is not well-formed (e.g. if there is no proper end tag) then an error message is logged and the RTA-A2L Generator aborts.
2. If the *McSupport* file is well-formed but not valid as per the specified AUTOSAR schema (e.g. if some mandatory attributes are missing) then an error message is logged and RTA-A2L Generator continues generating an A2L file.

For AUTOSAR-specific log messages, refer to the RTA-A2L Error Description Document.

5.5.2 Fetching of AUTOSAR Elements

The RTA-A2L Generator creates an A2L file that can contain the following AUTOSAR elements:

1. **Collection of <MC-DATA-INSTANCE> of type <MC-PARAMETER-INSTANCES>**

If the element is an instance of <BSW-IMPLEMENTATION>, then that element might contain <MC-SUPPORT> data. This <MC-SUPPORT> will contain the <MC-PARAMETER-INSTANCES>. All the <MC-DATA-INSTANCE>s which appear in this <MC-PARAMETER-INSTANCES> will be collected for the A2L *CHARACTERISTIC* block creation. If multiple entries of the *McDataInstances* exist with the same name, then an error message will be thrown.

Multiple entries of <MC-DATA-INSTANCE> exist with the <SHORT-NAME> "[{ShortNameValue}]". But only the first entry will be considered for the A2L file generation.

Based on the category of the <MC-DATA-INSTANCE>s, the *ShortName* of the A2L blocks will be altered.

2. **Collection of <MC-DATA-INSTANCE> of type <MC-VARIABLE-INSTANCES>**

<MC-SUPPORT> data also contains <MC-VARIABLE-INSTANCES>, which contains

MEASUREMENTs information in the form of <MC-DATA-INSTANCE>s. These <MC-DATA-INSTANCE>s will be collected for the A2L *MEASUREMENT* block creation.

If multiple entries of <MC-DATA-INSTANCE>s exist with the same name, then an error message will be thrown:

Multiple entries of <MC-DATA-INSTANCE> exist with the <SHORT-NAME> "{{ShortNameValue}}". Only the first entry will be considered for the A2L file generation.

Based on the category of the <MC-DATA-INSTANCE>s, the *ShortName* of the A2L blocks will be altered.

3. Collection of <UNIT>

In the input file, <ELEMENTS>/<UNIT> information is used to create the A2L *Unit* block. The RTA-A2L Generator supports duplicate <UNIT> objects in different *ARPackages*. *uniqueNames* will be created for the duplicate elements.

4. Collection of <COMPU-METHOD>

In the input file, <ELEMENTS>/<COMPU-METHOD> information is used to create the A2L *CompuMethod* block. RTA-A2L supports the duplicate <COMPU-METHOD> objects in different *ARPackages*. *uniqueNames* will be created for the duplicate elements.

5. Collection of <SW-RECORD-LAYOUT>

In the input file, <ELEMENTS>/<SW-RECORD-LAYOUT> information is used to create the A2L *RecordLayout* block. RTA-A2L supports the duplicate <SW-RECORD-LAYOUT> objects in different *ARPackages*. *uniqueNames* will be created for the duplicate elements.

6. Collection of <SW-BASE-TYPE>

In the input file, <ELEMENTS>/<SW-BASE-TYPE> information is used to create the A2L *DataType* block. RTA-A2L supports the duplicate <SW-BASE-TYPE> objects in different *ARPackages*. *uniqueNames* will be created for the duplicate elements.

7. Collection of <DATA-CONSTR>

In the input file, <ELEMENTS>/<DATA-CONSTR> information is used to create the A2L *Limits*. RTA-A2L supports the duplicate <DATA-CONSTR> objects in different *ARPackages*. *uniqueNames* will be created for the duplicate elements.

8. Collection of <PHYSICAL-DIMENSION>

In the input file, <ELEMENTS>/<PHYSICAL-DIMENSION> information is used to create the A2L *SI-EXPONENTS* for the A2L *Units*. The RTA-A2L supports the duplicate <PHYSICAL-DIMENSION> objects in different *ARPackages*. *uniqueNames* will be created for the duplicate elements.

9. Collection of <SW-SYSTEMCONST>

If the element is an instance of `<SW-SYSTEMCONST>`, then it is used to create the A2L *SystemConstant* block. RTA-A2L supports the duplicate `<SW-SYSTEMCONST>` objects in different *ARPackages*. *uniqueNames* will be created for the duplicate elements.

10. Collection of `<SW-SYSTEMCONSTANT-VALUE-SET>`

If the element is an instance of `<SW-SYSTEMCONSTANT-VALUE-SET>`, it represents the *SwSystemConstantValueSet* information which contains the list of *SwSystemConstValues* of the ARTOP model.

These *SystemConstant* values will, in turn, contain the *SwSystemConstReferences*, which refer to the *SystemConstants*. These are used to create the RTA-A2L *System_Constant* entries.

5.5.3 uniqueName Creation for Duplicate Entries

Except for the `<MC-DATA-INSTANCE>` element, the RTA-A2L Generator supports duplicate entries for the other AUTOSAR elements e.g. `<UNIT>`, `<COMPU-METHOD>`, `<SW-RECORD-LAYOUT>`, `<SW-SYSTEM-CONSTANT>`.

Duplicate elements appear in an `<AR-PACKAGE>` when the *ShortName* of each object is the same but the hash codes are different.

To make the blocks unique, RTA-A2L ensures that the first occurring object will have same name, whereas from the second occurrence onwards, the *_running* number will be added and created as a new *Unit* block. Moreover, all references to the object will be updated.

Sample Message

Duplicate entry of `<COMPU-METHOD>` with name "CompuMethodsUnits_CM_LindaPerInstPara2" exists in the same package `/RB/DataTypes/CompuMethods/CompuMethodsUnits_CM_LindaPerInstPara2` of file `input\McSupport_files\RB_McSupport_FrTrcv_BSW_Curve_Map_ComAxis.arxml`.

The figure below shows the generation of unique *Unit* blocks when the *Unit* block is a duplicate entry:


```

/begin UNIT
  Unit1
    "double quotes"
    "m/s"
    EXTENDED_SI
    SI_EXPONENTS 0 0 0 1 0 0 -1
/end UNIT

```

```

/begin UNIT
  Unit1__1
    "double quotes"
    "m/s"
    EXTENDED_SI
    SI_EXPONENTS 0 0 0 1 0 0 -1
/end UNIT

```

```

/begin UNIT
  Unit1__2
    "double quotes"
    "m/s"
    EXTENDED_SI
    SI_EXPONENTS 0 0 0 1 0 0 -1
/end UNIT

```

Figure 5.3.A2L Unique *Unit* blocks

If a *Unit* block with the *ShortName Unit1* has a duplicate entry, then the following blocks will be generated in the A2L file: *Unit1*, *Unit1__1*, *Unit1__2*, *Unit1__3*.

And if a <COMPU-METHOD> element refers to the duplicated *Unit* block (*Unit1__2*) the reference will be updated:

```

/begin COMPU_METHOD
  FlxXcvr_DeviceSelection_RatFunc
  "Longname for FlxXcvr_DeviceSelection_RatFunc"
  RAT_FUNC
  "%0.3"
  "m/s"
  COEFFS 0.0 1000.0 0.0 0.0 0.0 1.0
  REF_UNIT Unit1__2
/end COMPU_METHOD

```

Figure 5.4. *CompuMethod_With_Duplicated_UnitRef*

However, if the same AUTOSAR element duplicates *and, in this case the hash code of the duplicate elements is the same*, then that object will *not* be considered for the RTA-A2L block creation and the exception below will be thrown:

Duplicate entries of the {{ARBlock}} with the name '{{ShortName}}' have been found in the same package {{PackageName}}. Hence, this entry will not be considered for the A2L block generation.

Please refer to the *RTA-A2L Error Description* for the Log Entry for creating the *uniqueName* for an AUTOSAR element.

5.6 <MC-DATA-INSTANCE> Supported Categories

The RTA-A2L Generator supports the following data types and categories:

5.6.1 <MC-PARAMETER-INSTANCE> Supported Categories

- The *STRING*, *VALUE*, *VAL_BLK*, Simple *CURVE*, Simple *MAP*, Grouped *CURVE*, Simple *CUBOID*, Simple *CUBE_4* and Simple *CUBE_5* categories of primitive *McDataInstances*.
- The *STRUCTURE* and *ARRAY* Complex categories with combinations of the primitive types up to any level of nesting.
- If the <SW-CALIBRATION-ACCESS> of the <MC-DATA-INSTANCE> is of type *NOT-ACCESSIBLE* or <SW-CALIBRATION-ACCESS> is unavailable (the default value is *NOT-ACCESSIBLE*), then it will not appear in the A2L or ADX files.
- If the <SW-CALIBRATION-ACCESS> of the *Parent* element of the *STRUCTURE* or *ARRAY* element is of type *NOT-ACCESSIBLE* or <SW-CALIBRATION-ACCESS> is unavailable (the default value is *NOT-ACCESSIBLE*), then no A2L or ADX entries will be generated for the *Parent* or child elements. If the child element is of type *NOT-ACCESSIBLE*, or <SW-CALIBRATION-ACCESS> is unavailable (the default value is *NOT-ACCESSIBLE*) then, only for that element, no A2L or ADX entries will be generated.

5.6.2 <MC-VARIABLE-INSTANCE> Supported Categories

- The *VALUE* type of primitive *McDataInstances*.
- The *STRUCTURE* and *ARRAY* Complex categories with combinations of the primitive types up to any level of nesting.
- If the <SW-CALIBRATION-ACCESS> of the <MC-DATA-INSTANCE> is of type *NOT-ACCESSIBLE* or <SW-CALIBRATION-ACCESS> is unavailable (the default value is *NOT-ACCESSIBLE*), then it will not appear in the A2L or ADX files.
- If the <SW-CALIBRATION-ACCESS> of the *Parent* element of the *STRUCTURE* or *ARRAY* element is of type *NOT-ACCESSIBLE* or <SW-CALIBRATION-ACCESS> is unavailable (the default value is *NOT-ACCESSIBLE*), then no A2L or ADX entries will be generated for the *Parent* or child elements. If the child element is of type *NOT-ACCESSIBLE* or <SW-CALIBRATION-ACCESS> is unavailable (the default value is *NOT-ACCESSIBLE*) then, only for that element, no A2L or ADX entries will be generated.

5.6.3 Naming the MEASUREMENT and CHARACTERISTIC Blocks

Based on the category of the *McDataInstance*, the name of the A2L blocks will be as follows:

- If the *McDataInstance* category is *STRING/VALUE/VAL_BLK/CURVE/MAP/COM_AXIS/CUBOID/CUBE_4/CUBE_5*, a single A2L block will be created with name equal to the *ShortName* of the *McDataInstance*:

A2L Name: ShortName of the McDataInstance
 ADX Entry:
 <LABEL-NAME>: Symbol Information of the McDataInstance
 <MAP-LABEL-NAME>: ShortName of the McDataInstance

```
<MC-DATA-INSTANCE>
  <SHORT-NAME>ValueExample</SHORT-NAME>
  <LONG-NAME>
    <L-4 L="FOR-ALL">Longname for ValueExample</L-4>
  </LONG-NAME>
  <CATEGORY>VALUE</CATEGORY>
  <RESULTING-PROPERTIES>
    <SW-DATA-DEF-PROPS-VARIANTS>
      ...
    </SW-DATA-DEF-PROPS-VARIANTS>
  </RESULTING-PROPERTIES>
  <SYMBOL>FlxrXcvr_DeviceSelection_C</SYMBOL>
</MC-DATA-INSTANCE>
```

Figure 5.5.A Simple AUTOSAR block

A2L Name: VALUE Example

```

ADX Entry:
<LABEL-NAME>: FlxrXcvr_DeviceSelection_C
<MAP-LABEL-NAME>: VALUE Example

```

- If the category of the *Parent McDataInstance* is *STRUCTURE* then all the *subElements* will be collected and, for each *subElement*, the *ShortName* will be checked. If the *ShortName* exists for the *subElement*, then that *McDataInstance* will be considered. Then, based on the category of the *subElement McDataInstance*, the *A2LName* will be created.

If the *subElement* contains the <SYMBOL> information, then the *subElement* <SYMBOL> information will be considered for the ADX file generation. Or else, then the *Parent* element <SYMBOL> information will be considered.

- If the *subElement* is of category *VALUE/VAL_BLK/CURVE/MAP*, then one A2L block will be created for each *subElement* with the following details:

```
A2L Name: ShortName(Parent McDataInstance).ShortName(Child McDataInstance)
```

The following ADX entries will be created - one for the *Parent Structure McDataInstance* and one for each child *McDataInstance*:

```

<LABEL-NAME>: = Symbol(Parent McDataInstance)
<MAP-LABEL-NAME>: = ShortName(Parent McDataInstance) One entry for each child McDataInstance
if(Symbol(Child McDataInstance)) then
<LABEL-NAME>: = Symbol(Parent McDataInstance).Symbol(Child McDataInstance)
else <LABEL-NAME>: = Symbol(Parent McDataInstance).ShortName(Child McDataInstance)
<MAP-LABEL-NAME>: = ShortName(Parent McDataInstance).ShortName(ChildMcDataInstance)

```

The diagram below shows the *STRUCTURE* element having the *VALUE* as the *subElement*:

```

<MC-DATA-INSTANCE>
  <SHORT-NAME>CanIf_ControllerState</SHORT-NAME>
  <CATEGORY>STRUCTURE</CATEGORY>
  ...
  <SUB-ELEMENTS>
    <MC-DATA-INSTANCE>
      <SHORT-NAME>CanIf_ControllerModeType</SHORT-NAME>
      <CATEGORY>VALUE</CATEGORY>
      <RESULTING-PROPERTIES>
        ...
      </RESULTING-PROPERTIES>
      <SYMBOL>FirstChildElementSymbol</SYMBOL>
    </MC-DATA-INSTANCE>
  </SUB-ELEMENTS>
  <SYMBOL>CanIf_ControllerStateSymbol</SYMBOL>
</MC-DATA-INSTANCE>

```

Figure 5.6.AUTOSAR *STRUCTURE VALUE* Example

The A2L block will be created only for the primitive *McDataInstance* with the following naming conventions:

A2L Name for CanIf_ControllerModeType

CanIf_ControllerState.CanIf_ControllerModeType

The following ADX entries will be created for the Parent CanIf_Controller-State

```
<LABEL-NAME>: = CanIf_ControllerStateSymbol
<MAP-LABEL-NAME>: = CanIf_ControllerState
```

And for CanIf_ControllerModeType

```
<LABEL-NAME>: = CanIf_ControllerStateSymbol.CanIf_ControllerModeType
<MAP-LABEL-NAME>: = CanIf_ControllerState.CanIf_ControllerModeType
```

- If the *subElement* is of category *STRUCTURE*, one A2L block will be created for all the leaf-level primitive *McDataInstances* with the following A2L naming conventions:

A2L Name

```
ShortName(First Parent McDataInstance).ShortName(Second Parent McDataInstance) .....ShortName(leaf
level Primitive McDataInstance)
```

ADX Entries

One ADX Entry for each *Parent* *McDataInstance* and one entry for each leaf-level Primitive *McDataInstances*.

Parent McDataInstance

```
<LABEL-NAME>: = Symbol(First Parent McDataInstance).ShortName(Second Parent McDataInstance)
<MAP-LABEL-NAME>: = ShortName(First Parent McDataInstance).ShortName(Second Parent McDataInstance)
```

Child primitive McDataInstance

```
if(Symbol(subElement)) then,
<LABEL-NAME>: = Symbol(First Parent McDataInstance).Symbol (Second Parent McDataInstance)...Symbol(leaf
level Primitive McDataInstance)
Else
<LABEL-NAME>: = Symbol(First Parent McDataInstance).ShortName(Second Parent
McDataInstance)...ShortName(leaf level Primitive McDataInstance)
<MAP-LABEL-NAME>: = ShortName(First Parent McDataInstance).ShortName(Second Parent
McDataInstance)...ShortName(leaf level Primitive McDataInstance)
```

The diagram below shows the *STRUCTURE* element having the *STRUCTURE-VALUE* as the *subElements*:

```

<MC-DATA-INSTANCE>
  <SHORT-NAME>EngNControl1</SHORT-NAME>
  <CATEGORY>STRUCTURE</CATEGORY>
  ...
  <SUB-ELEMENTS>
    <MC-DATA-INSTANCE>
      <SHORT-NAME>EngNControl2</SHORT-NAME>
      <CATEGORY>STRUCTURE</CATEGORY>
      ...
      <SUB-ELEMENTS>
        <MC-DATA-INSTANCE>
          <SHORT-NAME>EngNControl23</SHORT-NAME>
          ...
        </MC-DATA-INSTANCE>
      </SUB-ELEMENTS>
    </MC-DATA-INSTANCE>
  </SUB-ELEMENTS>
  <SYMBOL>EpmNEngRecAr_MaxComplicated_Symbol</SYMBOL>
</MC-DATA-INSTANCE>

```

Figure 5.7. AUTOSAR *STRUCTURE VALUE* Example**A2L block Name for EngControl123**

EngNControl1.EngNControl2.EngNControl23

ADX Entry for EngConto11

<LABEL-NAME>: = EpmNEngRecAr_MaxComplicated_Symbol <MAP-LABEL-NAME>: = EngNControl1
--

ADX Entry for EngConto12

<LABEL-NAME>: = EpmNEngRecAr_MaxComplicated_Symbol.EngNControl2<MAP-LABEL-NAME>: = EngNControl1.EngNControl2

ADX Entry for EngConto123

<LABEL-NAME>: = EpmNEngRecAr_MaxComplicated_Symbol.EngNControl2.EngNControl23 <MAP-LABEL-NAME>: = EngNControl1.EngNControl2.EngNControl23
--

- If the *subElement* contains the <SYMBOL> information, then the *subElement* Symbol will be considered. Or else, the *Parent* element Symbol information will be considered.

In the example below, the *subElement* contains the <SYMBOL> information, which will be used for the ADX entries creation:

```

<MC-DATA-INSTANCE>
  <SHORT-NAME>CanIf_ControllerState</SHORT-NAME>
  <CATEGORY>STRUCTURE</CATEGORY>
  ...
  <SUB-ELEMENTS>
    <MC-DATA-INSTANCE>
      <SHORT-NAME>CanIf_ControllerModeType</SHORT-NAME>
      <CATEGORY>VALUE</CATEGORY>
      <RESULTING-PROPERTIES>
        ...
      </RESULTING-PROPERTIES>
      <SYMBOL>FirstChildElementSymbol</SYMBOL>
    </MC-DATA-INSTANCE>
  </SUB-ELEMENTS>
  <SYMBOL>CanIf_ControllerStateSymbol</SYMBOL>
</MC-DATA-INSTANCE>

```

Figure 5.8. *STRUCTURE VALUE* with <SYMBOL> information

AdxEntry for CanIf_ControllerState

```

<LABEL-NAME>: = CanIf_ControllerStateSymbol
<MAP-LABEL-NAME>: = CanIf_ControllerState

```

AdxEntry for CanIf_ControllerModeType

```

<LABEL-NAME>: = CanIf_ControllerStateSymbol.FirstChildElementSymbol
<MAP-LABEL-NAME>: = CanIf_ControllerState.CanIf_ControllerModeType

```

- If the *subElement* is of category *ARRAY*, the number of A2L blocks created is equal to the *ARRAY-SIZE* value of the *McDataInstances*. If the *ARRAY-SIZE* is not specified in the input file, the number of *subElements* will be considered as the *ARRAY-SIZE*.

For the CHARACTERISTIC block: A2L blockName of the Primitive Child element

```

ShortName(First Parent McDataInstance).ShortName (Array Parent McDataInstance)[i] , where i=0 to
ARRAY_SIZE-1

```

For the MEASUREMENT block: A2L blockName of the Primitive Child element

```

ShortName(First Parent McDataInstance).ShortName(Array Parent McDataInstance) with the ARRAY_SIZE
attribute

```

ADX Entries

```

if(Symbol(Array SubElementmnt)) then
<LABEL-NAME>: = Symbol(First Parent McDataInstance).Symbol(Array Parent McDataInstance)[i] , where i=0
to ARRAY_SIZE-1 else
<LABEL-NAME>: = Symbol(First Parent McDataInstance).ShortName (Array Parent McDataInstance)[i] , where
i=0 to ARRAY_SIZE-1
<MAP-LABEL-NAME>: = ShortName(First Parent McDataInstance).ShortName(Array Parent McDataInstance)[i] ,
where i=0 to ARRAY_SIZE-1

```

The diagram below shows the *STRUCTURE* element having the *ARRAY-VALUE* as the *subElements*:

```

<MC-DATA-INSTANCE>
  <SHORT-NAME>EngN_Record_t</SHORT-NAME>
  <CATEGORY>STRUCTURE</CATEGORY>
  ...
  <SUB-ELEMENTS>
    <MC-DATA-INSTANCE>
      <SHORT-NAME>EngN_Array_t</SHORT-NAME>
      <CATEGORY>ARRAY</CATEGORY>
      <ARRAY-SIZE>2</ARRAY-SIZE>
      ...
      <SUB-ELEMENTS>
        <MC-DATA-INSTANCE>
          <SHORT-NAME>element_0</SHORT-NAME>
          <LONG-NAME>
            <L-4 L="FOR-ALL">Longname for element_0</L-4>
          </LONG-NAME>
          <CATEGORY>VALUE</CATEGORY>
          ...
        </MC-DATA-INSTANCE>
      </SUB-ELEMENTS>
    </MC-DATA-INSTANCE>
  </SUB-ELEMENTS>
  <SYMBOL>EpmNEngRecAr_MaxComplicated_Symbol</SYMBOL>
</MC-DATA-INSTANCE>

```

Figure 5.9.AUTOSAR *STRUCTURE ARRAY VALUE* Example

In the above example the A2L blocks will be created for the *element_0* and *element_1* blocks.

The following CHARACTERISTIC blocks will be created: A2L block Name for **element_0**

```
EngN_Record_t.EngN_Array_t[0]
```

A2L block Name for element_1

```
EngN_Record_t.EngN_Array_t[1]
```

The following MEASUREMENT blocks will be created: A2LBlockName

```
EngN_Record_t.EngN_Array_t with the ARRAY_SIZE = 2
```

ADX entry for EngN_Record_t

```

<LABEL-NAME>: = EngN_Record_t
<MAP-LABEL-NAME>: = EpmNEngRecAr_MaxComplicated_symbol

```

ADX Entry for EngN_Array_t

```

<LABEL-NAME>: = EngN_Record_t.EngN_Array_t
<MAP-LABEL-NAME>: = EpmNEngRecAr_MaxComplicated_symbol.EngN_Array_t

```

ADX Entry for element_0

```

<LABEL-NAME>: = EngN_Record_t.EngN_Array_t[0]
<MAP-LABEL-NAME>: = EpmNEngRecAr_MaxComplicated_symbol.EngN_Array_t[0]

```

ADX Entry for element_1


```
<LABEL-NAME>: = EngN_Record_t.EngN_Array_t[1]
<MAP-LABEL-NAME>: = EpmEngRecAr_MaxComplicated_symbol.EngN_Array_t[1]
```

In the case of *ARRAY*, if the *subElement* contains the <SYMBOL> information then the *subElement* symbol will NOT be considered as all the *ARRAY subElements* are represented with the *Parent* entries. For the first leaf-level *subElement*, therefore, a warning message will be logged.

In the below example, the *ARRAY subElement* Symbol information will not be considered for the generation of ADX file entries:

```
<MC-DATA-INSTANCE>
  <SHORT-NAME>EngN_Record_t</SHORT-NAME>
  <CATEGORY>STRUCTURE</CATEGORY>
  ...
  <SUB-ELEMENTS>
    <MC-DATA-INSTANCE>
      <SHORT-NAME>EngN_Array_t</SHORT-NAME>
      <CATEGORY>ARRAY</CATEGORY>
      <ARRAY-SIZE>2</ARRAY-SIZE>
      ...
      <SUB-ELEMENTS>
        <MC-DATA-INSTANCE>
          <SHORT-NAME>element_0</SHORT-NAME>
          <LONG-NAME>
            <L-4 L="FOR-ALL">Longname for element_0</L-4>
          </LONG-NAME>
          <CATEGORY>VAL_BLK</CATEGORY>
          ...
          <SYMBOL>element_0_Symbol</SYMBOL>
        </MC-DATA-INSTANCE>
      </SUB-ELEMENTS>
    </MC-DATA-INSTANCE>
    <SYMBOL>EngN_Array_t_Symbol</SYMBOL>
  </SUB-ELEMENTS>
  <SYMBOL>EngN_Record_t_Symbol</SYMBOL>
</MC-DATA-INSTANCE>
```

Figure 5.10. *STRUCTURE-ARRAY-VALUE* with *subElement* Symbol

```
AdxEntry (MapLabelName,LabelName)
AdxEntry (EngN_Record_t, EngN_Record_t_Symbol)
AdxEntry (EngN_Record_t.EngN_Array_t,EngN_Record_t_Symbol.EngN_Array_t_Symbol)
AdxEntry (EngN_Record_t.EngN_Array_t[0],EngN_Record_t_Symbol.EngN_Array_t_Symbol[0])
AdxEntry (EngN_Record_t.EngN_Array_t[1],EngN_Record_t_Symbol.EngN_Array_t_Symbol[1])
```

- If the type of the *McDataInstance* is *ARRAY*:

1. The number of A2L blocks is equal to the *ARRAY_SIZE* attribute of the *McDataInstance*. If the *ARRAY_SIZE* is not mentioned, the number of *subElements* will be considered as the *ARRAY_SIZE* value.
2. No A2L Entry will be created for the *Parent* element.
3. Based on the category of the *subElement*, the names of the A2L blocks will be updated.
4. In the case of *ARRAY*, if the *subElement* contains the <SYMBOL> information, then the *subElement* Symbol will NOT be considered for the ADX file entries

generation, since all of the *ARRAY subElements* are represented with the *Parent* entries and for the first leaf-level *subElement*. (A warning message will be logged.)

5. If the category of *subElement* is *VALUE/VAL_BLK/CURVE/MAP*:

MEASUREMENT block creation

For the Simple *ARRAY* category, only one block will be created and the *ARRAY_SIZE* attribute will be updated for the A2L block.

A2L block Name

ShortName(Parent McDataInstance), with the ARRAY_SIZE attribute ADX Entry:
<LABEL-NAME>: = Symbol(Parent McDataInstance)[i] , where i=0 to ARRAY_SIZE-1
<MAP-LABEL-NAME>: = ShortName(Parent McDataInstance)[i] , where i=0 to ARRAY_SIZE-1

CHARACTERISTIC block creation

The number of the *CHARACTERISTICS* blocks created is equal to the value of the *ARRAY_SIZE*.

A2L block Name

ShortName(Parent McDataInstance)[i], where i=0 to ARRAY_SIZE-1

ADX Entry

<LABEL-NAME>: = Symbol(Parent McDataInstance)[i], where i=0 to ARRAY_SIZE-1
<MAP-LABEL-NAME>: = ShortName(Parent McDataInstance)[i], where i=0 to ARRAY_SIZE-1

The diagram below shows the *ARRAY* element having the *VALUE* as the *subElements*:

```

<MC-DATA-INSTANCE>
  <SHORT-NAME>EpmNEngArray_Tina</SHORT-NAME>
  <CATEGORY>ARRAY</CATEGORY>
  <ARRAY-SIZE>2</ARRAY-SIZE>
  ...
  <SUB-ELEMENTS>
    <MC-DATA-INSTANCE>
      <SHORT-NAME>element_0</SHORT-NAME>
      <CATEGORY>VALUE</CATEGORY>
      ...
    </MC-DATA-INSTANCE>
    <MC-DATA-INSTANCE>
      <SHORT-NAME>element_1</SHORT-NAME>
      <CATEGORY>VALUE</CATEGORY>
      ...
    </MC-DATA-INSTANCE>
  </SUB-ELEMENTS>
  <SYMBOL>EpmNEngArray_Tina_Symbol</SYMBOL>
</MC-DATA-INSTANCE>

```

Figure 5.11.ARRAY VALUE Example

In the above example, the following naming conventions will be followed:

A2L block Name for the MEASUREMENT block: EpmNEngArray_Tina
A2L block Names for the CHARACTERISTIC blocks

EpmNEngARRAY_Tina [0], EpmNEngARRAY_Tina[1]

ADX Entries

<LABEL-NAME>: = EpmNEngArray_Tina_symbol <MAP-LABEL-NAME>: = EpmNEngArray_Tina <LABEL-NAME>: = EpmNEngArray_Tina_symbol[i], where i=0 to ARRAY_SIZE-1 <MAP-LABEL-NAME>: = EpmNEngArray_Tina[i] where, i=0 to ARRAY_SIZE-1
--

- If the category of the *subElement* is *ARRAY*, the *ARRAY_SIZE* value of the sub-*ARRAY* element will be fetched and added to the *ShortNameList*. If the *ARRAY_SIZE* attribute doesn't exist in the input file, then the *ARRAY_SIZE* will be considered as the *subElements* size.

A2L block Name

1st Parent ShortName[i][j], where i is 0 to 1st Parent (ARRAY_SIZE-1) and j is 0 to 2nd Parent (ARRAY_SIZE-1).
--

ADX Entries

<LABEL-NAME>: = 1st Parent Symbol[i][j] where, i is 0 to 1st Parent (ARRAY_SIZE-1) and j is 0 to 2nd Parent (ARRAY_SIZE-1).
 <MAP-LABEL-NAME>: = 1st Parent ShortName[i][j], where i is 0 to 1st Parent(ARRAY_SIZE-1) and j is 0 to 2nd Parent (ARRAY_SIZE-1).

The diagram below shows the *ARRAY* element having the *ARRAY-VALUE* as the *subElements*:

```
<MC-DATA-INSTANCE>
  <SHORT-NAME>CoEngStArray</SHORT-NAME>
  <CATEGORY>ARRAY</CATEGORY>
  <ARRAY-SIZE>3</ARRAY-SIZE>
  ...
  <SUB-ELEMENTS>
    <MC-DATA-INSTANCE>
      <SHORT-NAME>element_0</SHORT-NAME>
      <CATEGORY>ARRAY</CATEGORY>
      <ARRAY-SIZE>2</ARRAY-SIZE>
      ...
      <SUB-ELEMENTS>
        <MC-DATA-INSTANCE>
          <SHORT-NAME>element_0_value</SHORT-NAME>
          <CATEGORY>VALUE</CATEGORY>
          ...
        </MC-DATA-INSTANCE>
        <MC-DATA-INSTANCE>
          <SHORT-NAME>element_1_value</SHORT-NAME>
          <CATEGORY>VALUE</CATEGORY>
          ...
        </MC-DATA-INSTANCE>
      </SUB-ELEMENTS>
    </MC-DATA-INSTANCE>
  </SUB-ELEMENTS>
  <SYMBOL>CoEngStArray_Symbol</SYMBOL>
</MC-DATA-INSTANCE>
```

Figure 5.12.AUTOSAR *ARRAY VALUE*

For the example above, the *Names* will be altered as follows:

A2L blocks Name

{CoEngStArray[0][0], CoEngStArray[0][1], CoEngStArray[0][2], CoEngStArray[1][0], CoEngStArray[1][1], CoEngStArray[1][2]}, CoEngStArray[2][0], CoEngStArray[2][1], CoEngStArray[2][2]}

ADX Entry for the ARRAY Parent McDataInstance

<LABEL-NAME>: = CoEngStArray_Symbol <MAP-LABEL-NAME>: = CoEngStArray

ADX Entries for the ARRAY Parent elements

<LABEL-NAME>: = CoEngStArray_Symbol[0]
 <MAP-LABEL-NAME>: = CoEngStArray[0]
 <LABEL-NAME>: = CoEngStArray_Symbol[1]

```

<MAP-LABEL-NAME>: = CoEngStArray[1]
<LABEL-NAME>: = CoEngStArray_Symbol[2]
<MAP-LABEL-NAME>: = CoEngStArray[2]

```

ADX Entries for the Child elements

```

<LABEL-NAME>: = CoEngStArray_Symbol[0][0]
<MAP-LABEL-NAME>: = CoEngStArray[0][0]

<LABEL-NAME>: = CoEngStArray_Symbol[0][1]
<MAP-LABEL-NAME>: = CoEngStArray[0][1]

<LABEL-NAME>: = CoEngStArray_Symbol[0][2]
<MAP-LABEL-NAME>: = CoEngStArray[0][2]

<LABEL-NAME>: = CoEngStArray_Symbol[1][0]
<MAP-LABEL-NAME>: = CoEngStArray[1][0]

<LABEL-NAME>: = CoEngStArray_Symbol[1][1]
<MAP-LABEL-NAME>: = CoEngStArray[1][1]

<LABEL-NAME>: = CoEngStArray_Symbol[1][2]
<MAP-LABEL-NAME>: = CoEngStArray[1][2]

<LABEL-NAME>: = CoEngStArray_Symbol[2][0]
<MAP-LABEL-NAME>: = CoEngStArray[2][0]

<LABEL-NAME>: = CoEngStArray_Symbol[2][1]
<MAP-LABEL-NAME>: = CoEngStArray[2][1]

<LABEL-NAME>: = CoEngStArray_Symbol[2][2]
<MAP-LABEL-NAME>: = CoEngStArray[2][2]

```

- If the category of the *subElement* is *STRUCTURE*, then the number of *Short-Name* entries will be equal to the *ARRAY_SIZE* number of Structure *subElements*.

A2L block Name

```
ParentArrayName[i].ChildName(Structure element), where i=0 to Parent ARRAY_SIZE-1
```

ADX Entry

One entry will be created for each *ARRAY* and Structure *Parent* element and one entry for the *subElements*.

The following naming conventions apply for the child elements:

```

<LABEL-NAME>: = ParentSymbol[i].ChildName(Structure element), where i=0 to Parent ARRAY_SIZE-1
<MAP-LABEL-NAME>: = ParentName[i].ChildName(Structure element), where i=0 to Parent ARRAY_SIZE-1

```

The diagram below shows the *ARRAY* element having the *STRUCTURE-VALUE* as the *subElements*:

```

<MC-DATA-INSTANCE>
  <SHORT-NAME>CoEngStArray</SHORT-NAME>
  <CATEGORY>ARRAY</CATEGORY>
  <ARRAY-SIZE>3</ARRAY-SIZE>
  ...
  <SUB-ELEMENTS>
    <MC-DATA-INSTANCE>
      <SHORT-NAME>element_0</SHORT-NAME>
      <CATEGORY>STRUCTURE</CATEGORY>
      ...
      <SUB-ELEMENTS>
        <MC-DATA-INSTANCE>
          <SHORT-NAME>EngN</SHORT-NAME>
          <CATEGORY>VALUE</CATEGORY>
          ...
        </MC-DATA-INSTANCE>
      </SUB-ELEMENTS>
    </MC-DATA-INSTANCE>
    ...
  </SUB-ELEMENTS>
  <SYMBOL>CoEngStArray_Symbol</SYMBOL>
</MC-DATA-INSTANCE>

```

Figure 5.13.AUTOSAR ARRAY STRUCTURE VALUE Example

In the example above, the following naming conventions apply:

A2L block Name for EngN: Since the ARRAY_SIZE value is 2, two A2L blocks of EngN will be created with the following Names

{CoEngStArray[0].EngN, CoEngStArray[1].EngN}
--

ADX Entries

<LABEL-NAME>: = CoEngStArray_Symbol <MAP-LABEL-NAME>: = CoEngStArray <LABEL-NAME>: = CoEngStArray_Symbol[0] <MAP-LABEL-NAME>: = CoEngStArray[0] <LABEL-NAME>: = CoEngStArray_Symbol[1] <MAP-LABEL-NAME>: = CoEngStArray[1] <LABEL-NAME>: = CoEngStArray_Symbol[0].EngN <MAP-LABEL-NAME>: = CoEngStArray[0].EngN <LABEL-NAME>: = CoEngStArray_Symbol[1].EngN <MAP-LABEL-NAME>: = CoEngStArray[1].EngN

Please refer to the *RTA-A2L Error Description* document for the list of log entries supported for the creation of A2L Name for the *CHARACTERISTIC MEASUREMENT* elements.

5.6.4 D-SERAP Structure Elements

- The input <MC-DATA-INSTANCE> blocks can have the *D-SERAP STRUCTURE*, which the A2L blocks generated will also have if the *Parent STRUCTURE* Name is ignored.
- An additional validation has been added for *D-SERAP STRUCTURES*. They are generated if *calibrationMethod* = *SINGLE_POINTERED2*. Or else, then normal *STRUCTURES* are generated.
- If the <SW-CALIBRATION-ACCESS> of the <MC-DATA-INSTANCE> is of type *NOT-ACCESSIBLE*, or if <SW-CALIBRATION-ACCESS> is unavailable (the default value is *NOT-ACCESSIBLE*), then that entry will not appear in the A2L or ADX files.
- If the <SW-CALIBRATION-ACCESS> of the *Parent* of the *STRUCTURE* element is of type *NOT-ACCESSIBLE*, or if <SW-CALIBRATION-ACCESS> is unavailable (the default value is *NOT-ACCESSIBLE*), then no A2L or ADX entries will be generated either for the *Parent* or the child elements. If the child element is of type *NOT-ACCESSIBLE* or <SW-CALIBRATION-ACCESS> is unavailable (the default value is *NOT-ACCESSIBLE*) then, only for that element, no A2L or ADX entries will be generated.

The diagram below shows the sample *D-SERAP STRUCTURE* example which contains the *VALUE* as the *subElement*:

```

<MC-DATA-INSTANCE>
  <SHORT-NAME>Rte_CGi_BSWImp17</SHORT-NAME>
  <CATEGORY>STRUCTURE</CATEGORY>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="ETAS-RTARTE">
        <SD GID="MC-DATA-CLASS">CALPRM-GROUP</SD>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
  ...
  <SUB-ELEMENTS>
    <MC-DATA-INSTANCE>
      <SHORT-NAME>CanIf_Acq_JDD_CAN_BSI_772_D1c</SHORT-NAME>
      <CATEGORY>VALUE</CATEGORY>
      <ADMIN-DATA>
        <SDGS>
          <SDG GID="ETAS-RTARTE">
            <SD GID="MC-DATA-CLASS">CALPRM</SD>
          </SDG>
        </SDGS>
      </ADMIN-DATA>
      ...
    </MC-DATA-INSTANCE>
    ...
  </SUB-ELEMENTS>
  <SYMBOL>Rte_CGi_BSWImp17_BSWIMPL_Can_CALIB_UNSPECIFIED</SYMBOL>
</MC-DATA-INSTANCE>

```

Figure 5.14. *D-SERAP STRUCTURE* example (A *D-SERAP STRUCTURE* with a *VALUE*)

Here, the *Parent STRUCTURE* belongs to the *MC-DATA-CLASS* of type *CALPRM-GROUP* and the child element belongs to the *MC-DATA-CLASS* of type *Calprm*. The *Parent* element's category should be *STRUCTURE* and the *subElements* could be of type *STRUCTURE*, *ARRAY*, *VALUE*, *CURVE*, *MAP* or *COM_AXIS*. The *MC-DATA-CLASS* of type *Calprm-GROUP* must appear only for the root element.

A2L block Name

```
CanIf_Acq_JDD_CAN_BSI_772_D1c
```

ADX Entry

```

<LABEL-NAME>: = Rte_CGi_BSWImp17_BSWIMPL_Can_CALIB_UNSPECIFIED.CanIf_Acq_JDD_CAN_BSI_772_D1c
<MAP-LABEL-NAME>: = CanIf_Acq_JDD_CAN_BSI_772_D1c

```

The complete A2L block appears as below:


```

/begin CHARACTERISTIC
  CanIf_Acq_JDD_CAN_BSI_772_D1c
  "This is the longname for CanIf_Acq_JDD_CAN_BSI_772_D1c"
  VALUE
    0x0
    Value_uint8
    4.0
    CanIf_Var_Compu
    0.0
    4.
/end CHARACTERISTIC

```

The name will not be referenced by the parent name

Figure 5.15. *D-SERAP* A2L block

Since the child element will not contain the Symbol information, the <LABEL-NAME> will still be referenced from the *Parent* Symbol, whereas the <MAP-LABEL-NAME> will contain just the Child Name.

```

<MEMORY-ELEMENT>
  <LABEL-NAME>Rte_CGi_BSWImpl7_BSWIMPL_Can_CALIB_UNSPECIFIED.CanIf_Acq_JDD_CAN_BSI_772_D1c</LABEL-NAME>
  <MAP-LABEL-NAME>CanIf_Acq_JDD_CAN_BSI_772_D1c</MAP-LABEL-NAME>
</MEMORY-ELEMENT>

```

Figure 5.16. Mapping ADX Entries supported for the *D-SERAP STRUCTURE*

5.6.4.1 Dserap_Structure_Structure_Value Example

The example below contains the *D-SERAP STRUCTURE*, with another Structure element as the *subElement*, which in turn contains the *VALUE* as the child element:

```

<MC-DATA-INSTANCE>
  <SHORT-NAME>EpmNEngRecAr</SHORT-NAME>
  <CATEGORY>STRUCTURE</CATEGORY>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="ETAS-RTARTE">
        <SD GID="MC-DATA-CLASS">CALPRM-GROUP</SD>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
  ...
  <SUB-ELEMENTS>
    <MC-DATA-INSTANCE>
      <SHORT-NAME>EngNControl2</SHORT-NAME>
      <CATEGORY>STRUCTURE</CATEGORY>
      <ADMIN-DATA>
        <SDGS>
          <SDG GID="ETAS-RTARTE">
            <SD GID="MC-DATA-CLASS">CALPRM</SD>
          </SDG>
        </SDGS>
      </ADMIN-DATA>
      ...
    </MC-DATA-INSTANCE>
    <SUB-ELEMENTS>
      <MC-DATA-INSTANCE>
        <SHORT-NAME>EngNControl23</SHORT-NAME>
        <CATEGORY>VALUE</CATEGORY>
        <ADMIN-DATA>
          <SDGS>
            <SDG GID="ETAS-RTARTE">
              <SD GID="MC-DATA-CLASS">CALPRM</SD>
            </SDG>
          </SDGS>
        </ADMIN-DATA>
        ...
      </MC-DATA-INSTANCE>
    </SUB-ELEMENTS>
  </MC-DATA-INSTANCE>
  <SYMBOL>EpmNEngRecAr_Symbol</SYMBOL>
</MC-DATA-INSTANCE>

```

Figure 5.17.Dserap_Structure_Structure_Value Example

The following naming conventions apply:

A2L block Name

EngNControl12.EngNControl23

ADX Entry

<LABEL-NAME>: = EpmNEngRecAr_Symbol.EngNControl12 <MAP-LABEL-NAME>: = EngNControl12 <LABEL-NAME>: = EpmNEngRecAr_Symbol.EngNControl12.EngNControl23 <MAP-LABEL-NAME>: = EngNControl12.EngNControl23
--

5.6.4.2 D-SERAP STRUCTURE_ARRAY_VALUE Example

The example below contains the *D-SERAP STRUCTURE*, with another Structure element as the *subElement*, which in turn contains the *VALUE* as the child element:

```
<MC-DATA-INSTANCE>
  <SHORT-NAME>EngN_Record_t</SHORT-NAME>
  <CATEGORY>STRUCTURE</CATEGORY>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="ETAS-RTARTE">
        <SD GID="MC-DATA-CLASS">CALPRM-GROUP</SD>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
  ...
  <SUB-ELEMENTS>
    <MC-DATA-INSTANCE>
      <SHORT-NAME>EngN_Array_t</SHORT-NAME>
      <CATEGORY>ARRAY</CATEGORY>
      <ADMIN-DATA>
        <SDGS>
          <SDG GID="ETAS-RTARTE">
            <SD GID="MC-DATA-CLASS">CALPRM</SD>
          </SDG>
        </SDGS>
      </ADMIN-DATA>
      <ARRAY-SIZE>2</ARRAY-SIZE>
    </MC-DATA-INSTANCE>
    ...
    <SUB-ELEMENTS>
      <MC-DATA-INSTANCE>
        <SHORT-NAME>element_0</SHORT-NAME>
        <CATEGORY>VALUE</CATEGORY>
        <ADMIN-DATA>
          <SDGS>
            <SDG GID="ETAS-RTARTE">
              <SD GID="MC-DATA-CLASS">CALPRM</SD>
            </SDG>
          </SDGS>
        </ADMIN-DATA>
        ...
      </MC-DATA-INSTANCE>
      <MC-DATA-INSTANCE>
        <SHORT-NAME>element_1</SHORT-NAME>
        ...
      </MC-DATA-INSTANCE>
    </SUB-ELEMENTS>
  </MC-DATA-INSTANCE>
  <SYMBOL>EpmNEngRecAr_Symbol</SYMBOL>
</MC-DATA-INSTANCE>
```

Figure 5.18.DSERAP_Structure_Array_Value Example

The following naming conventions apply:

A2L block Name

EngN_Array_t

ADX Entry

<pre> <LABEL-NAME>: = EpmNEngRecAr_symbol[0] <MAP-LABEL-NAME>: = EngN_Array_t[0] <LABEL-NAME>: = EpmNEngRecAr_symbol[1] <MAP-LABEL-NAME>: = EngN_Array_t[1] </pre>
--

**NOTE**

If the <SD> tags are missing in the <ADMIN-DATA>, then behavior will be as a normal Structure element.

5.6.5 RAM-BLOCK STRUCTURE Elements

- *RAM-BLOCK STRUCTURES* are generated when the *calibrationMethod* is provided as *INITIALIZED_RAM*. If the *calibrationMethod* is other than *INITIALIZED_RAM*, then normal structures are generated.
- The input <MC-DATA-INSTANCE> blocks can be *RAM-BLOCK STRUCTURES*, for which the A2L blocks will be generated by ignoring the parent *STRUCTURE Names*.
- If the <SW-CALIBRATION-ACCESS> of the *STRUCTURE* parent element is of type *NOT-ACCESSIBLE*, or <SW-CALIBRATION-ACCESS> is unavailable (the default value is *NOT-ACCESSIBLE*), then no A2L or ADX entries will be generated, either for the parent or child elements.

If only the child element is of type *NOT-ACCESSIBLE*, or <SW-CALIBRATION-ACCESS> is unavailable (the default value is *NOT-ACCESSIBLE*) then, only for that element, no A2L or ADX entries will be generated.

- If the direct *SUB-ELEMENTS* of the *RAM-BLOCK STRUCTURES* do not fall under the category of *STRUCTURE*, then those elements will not be generated in an A2L or ADX file.
- The *MC-DATA-CLASS* type of the *RAM-BLOCK SUB-ELEMENTS* must be categorized as *CALPRM-GROUP* and should belong to the *STRUCTURE* category. In addition, the *MC-DATA-CLASS* for the *SUB-ELEMENTS* within the *CALPRM-GROUP* must also be of type *CALPRM*. If these conditions are not met, the elements will not be generated in the A2L and ADX files.

5.6.5.1 RAM-BLOCK STRUCTURES with Value as Leaf Element

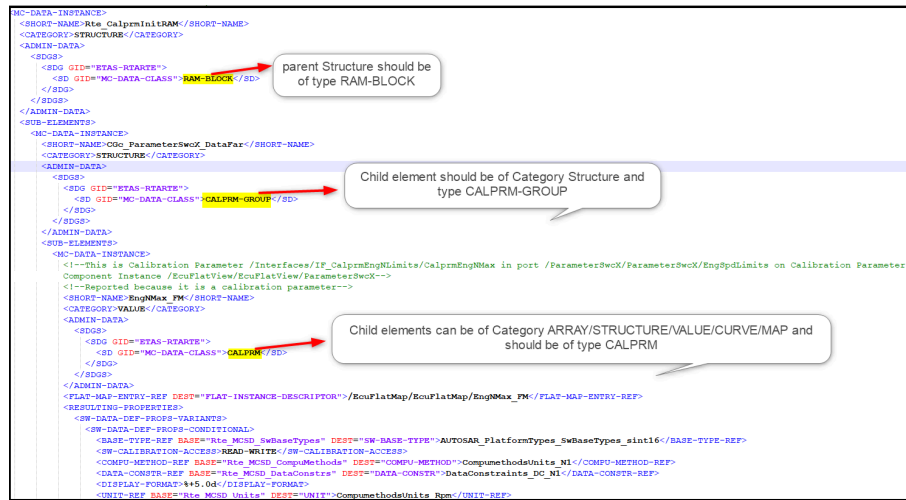


Figure 5.19. RAM-BLOCK STRUCTURE with Value as Leaf Element

- Where the parent *STRUCTURE* belongs to the *MC-DATA-CLASS* type *RAM-BLOCK*, and all the immediate *SUB-ELEMENTS* are of category *STRUCTURE* and the *MC-DATA-CLASS* type *CALPRM-GROUP*. The *SUB-ELEMENTS* of the child *STRUCTURE* should belong to *MC-DATA-CLASS* type *CALPRM*.

The following data will be generated in the ADX file as:

```
<LABEL-NAME>: Rte_CalprminitRAM .CGc_ParameterSwCx_DataFar.EngNMax_FM
<MAP-LABEL-NAME>: EngNMax_FM
```

An A2L Snippet with *shortName EngNMax_FM* is generated as below:

```
/begin CHARACTERISTIC
EngNMax_FM
  ""
  VALUE
  0x0
  DefaultValueRecordLayout_AUTOSAR_PlatformTypes_SwBaseTypes_sint16
  16583.5
  CompuMethodsUnits_N1
  -200.00000
  16383.50000
  FORMAT "%10.5"
  PHYS UNIT "rpm"
/end CHARACTERISTIC
```

Figure 5.20. A2L Snippet with shortName EngNMax_FM

5.6.5.2 RAM-BLOCK STRUCTUREs with Array as Leaf Element



Figure 5.21. RAM-BLOCK STRUCTURE with Array as Leaf Element

The following data will be generated in the ADX file:

```

<LABEL-NAME>: Rte_CalprmInitRAM.CGi_BSWImpl1_BSWIMPL_Gerlinde_DataNear.GerlindePerInstParaArray
<MAP-LABEL-NAME>: GerlindePerInstParaArray

```

An A2L Snippet with *shortName* GerlindePerInstParaArray is generated as below:

```

begin CHARACTERISTIC
  GerlindePerInstParaArray
  ""
  VAL_BLK
  0x0
  DefaultValueRecordLayout_AUTOSAR_PlatformTypes_SwBaseTypes_sint16
  22644.3
  CompumethodsUnits_CM_SignalDummy2
  -299.30000
  22345.00000
  FORMAT "%10.5"
  PHYS_UNIT "Display_SignalDummy2"
  NUMBER 3
/end CHARACTERISTIC

```

Figure 5.22. A2L Snippet with shortName GerlindePerInstParaArray

5.6.5.3 RAM-BLOCK STRUCTURE with STRUCTURE as Leaf Element

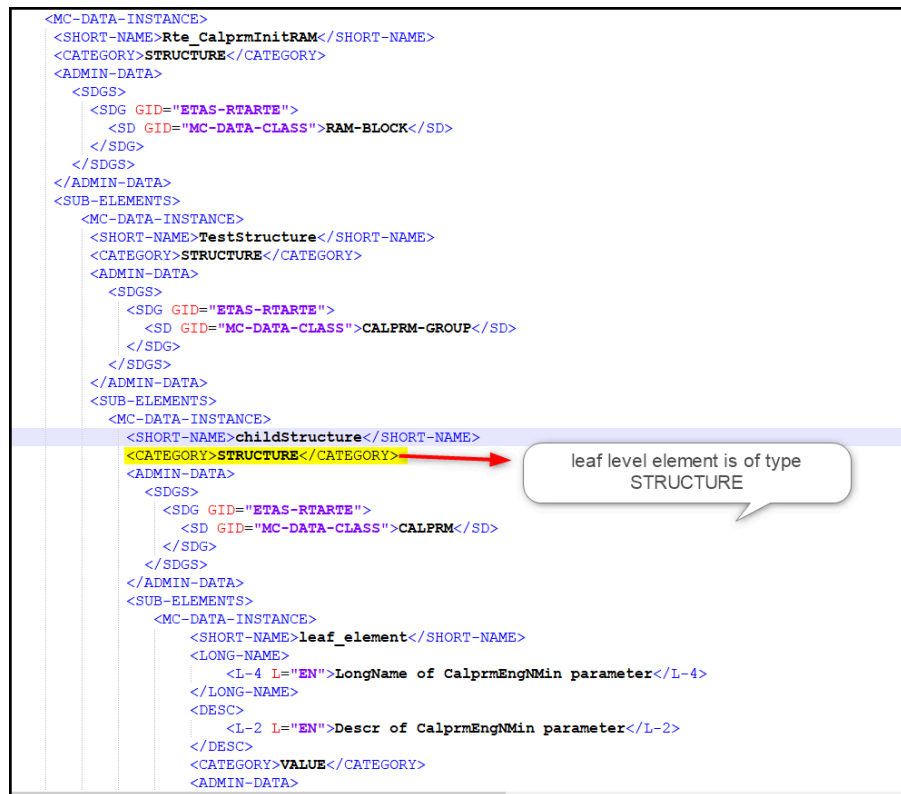


Figure 5.23.RAM-BLOCK STRUCTURE with STRUCTURE as Leaf Element

The following data will be generated in the ADX file:

```

<LABEL-NAME>: Rte_CalprmInitRAM.Test_Symbol.childStructureSymbol.leaf
<MAP-LABEL-NAME>: childStructure.leaf_element

```

An A2L Snippet with *shortName* *childStructure.leaf_element* is generated as below:

```

/begin CHARACTERISTIC
  childStructure.leaf_element
    "/LongName of CalprmEngNMin parameter"
    VALUE
    0x0
    DefaultValueRecordLayout_AUTOSAR_PlatformTypes_SwBaseTypes_sint16
    16583.5
    CompuMethodsUnits_N1
    -200.00000
    16383.50000
    FORMAT "%10.5"
    PHYS UNIT "rpm"
  /begin ANNOTATION
    ANNOTATION LABEL "Description (coming from AUTOSAR DESC field)"
    /begin ANNOTATION_TEXT
      "Descr of CalprmEngNMin parameter\r\n"
    /end ANNOTATION_TEXT
  /end ANNOTATION
/end CHARACTERISTIC

```

Figure 5.24.A2L Snippet with shortName childStructure.leaf_element

5.6.5.4 RAM-BLOCK STRUCTURES with Complex STRUCTURES as SUB-ELEMENTS

In the case of Complex *STRUCTURES* at the leaf level (which constitutes the third-level *SUB-ELEMENT* or *SUB-ELEMENT* of *CALPRM-GROUP STRUCTURES*) the naming convention is consistent with that of standard Complex *STRUCTURES* as described in *Naming the MEASUREMENT and CHARACTERISTIC Block* (please refer to Section 5.6.3 for more details).

In this convention, the leaf-level element serves as the parent element for the purposes of generating the *ShortName* and *Symbol*. e.g.:

```

<MC-DATA-INSTANCE>
  <SHORT-NAME>Rte_CalprmInitRAM</SHORT-NAME>
  <CATEGORY>STRUCTURE</CATEGORY>
  <ADMIN-DATA>
    <SDGS><SDG GID="ETAS-RTARTE"><SD GID="MC-DATA-CLASS">RAM-BLOCK</SD></SDG></SDGS>
  </ADMIN-DATA>
  <SUB-ELEMENTS>
    <MC-DATA-INSTANCE>
      <SHORT-NAME>CGc_ParameterSwCx_DataFar</SHORT-NAME>
      <CATEGORY>STRUCTURE</CATEGORY>
      <ADMIN-DATA>
        <SDGS><SDG GID="ETAS-RTARTE"><SD GID="MC-DATA-CLASS">CALPRM-GROUP</SD></SDG></SDGS>
      </ADMIN-DATA>
      <SUB-ELEMENTS>
        <MC-DATA-INSTANCE>
          <SHORT-NAME>EngNMax_FM</SHORT-NAME>
          <CATEGORY>ARRAY</CATEGORY>
          <ADMIN-DATA>
            <SDGS><SDG GID="ETAS-RTARTE"><SD GID="MC-DATA-CLASS">CALPRM</SD></SDG></SDGS>
          </ADMIN-DATA>
          <ARRAY-SIZE>2</ARRAY-SIZE>
          <SUB-ELEMENTS>
            <MC-DATA-INSTANCE>
              <SHORT-NAME>element_0</SHORT-NAME>
              <CATEGORY>STRUCTURE</CATEGORY>
              <SUB-ELEMENTS>
                <MC-DATA-INSTANCE>
                  <SHORT-NAME>EngNMin0_FM0</SHORT-NAME>
                  <LONG-NAME>
                    <L-4 L="EN">LongName of CalprmEngNMin parameter</L-4>
                  </LONG-NAME>
                  <CATEGORY>VALUE</CATEGORY>
                  <ADMIN-DATA>
                    <SDGS><SDG GID="ETAS-RTARTE"><SD GID="MC-DATA-CLASS">CALPRM</SD></SDG></SDGS>
                  </ADMIN-DATA>
                  <FLAT-MAP-ENTRY-REF DEST="FLAT-INSTANCE-DESCRIPTOR">/EcuFlatMap/EcuFlatMap/EngNMin_FM</FLAT-MAP-ENTRY-REF>
                  <RESULTING-PROPERTIES>
                    <SW-DATA-DEF-PROPS-VARIANTS>
                      <SW-DATA-DEF-PROPS-CONDITIONAL>
                        <BASE-TYPE-REF BASE="Rte_MCSD_SwBaseTypes" DEST="SW-BASE-TYPE">AUTOSAR_PlatformTypes_SwBaseTypes_sint16</BASE-TYPE-REF>
                        <SW-CALIBRATION-ACCESS>READ-WRITE</SW-CALIBRATION-ACCESS>
                        <COMPU-METHOD-REF BASE="Rte_MCSD_CompuMethods" DEST="COMPU-METHOD">CompumethodsUnits_N1</COMPU-METHOD-REF>
                        <DATA-CONSTR-REF BASE="Rte_MCSD_DataConstrs" DEST="DATA-CONSTR">DataConstraints_DC_N1</DATA-CONSTR-REF>
                      </SW-DATA-DEF-PROPS-CONDITIONAL>
                    </SW-DATA-DEF-PROPS-VARIANTS>
                  </RESULTING-PROPERTIES>
                </MC-DATA-INSTANCE>
              </SUB-ELEMENTS>
            </MC-DATA-INSTANCE>
          </SUB-ELEMENTS>
        </MC-DATA-INSTANCE>
      </SUB-ELEMENTS>
    </MC-DATA-INSTANCE>
  </SUB-ELEMENTS>
</MC-DATA-INSTANCE>

```

Figure 5.25. RAM-BLOCK STRUCTURE with Complex STRUCTURE as SUB-ELEMENT

An A2L Snippet for the above data is generated as below:


```
/begin CHARACTERISTIC
EngNMax_FM[0].EngNMin0_FM0
"/LongName of CalprmEngNMin parameter"
VALUE
0x0
DefaultValueRecordLayout_AUTOSAR_PlatformTypes_SwBaseTypes_sint16
16583.5
CompumethodsUnits_N1
-200.00000
16383.50000
FORMAT "%10.5"
PHYS_UNIT "rpm"
/begin ANNOTATION
  ANNOTATION_LABEL "Description (coming from AUTOSAR DESC field)"
  /begin ANNOTATION_TEXT
    "Descr of CalprmEngNMin parameter\r\n"
  /end ANNOTATION_TEXT
/end ANNOTATION
/end CHARACTERISTIC

/begin CHARACTERISTIC
EngNMax_FM[1].EngNMin0_FM0
"/LongName of CalprmEngNMin parameter"
VALUE
0x0
DefaultValueRecordLayout_AUTOSAR_PlatformTypes_SwBaseTypes_sint16
16583.5
CompumethodsUnits_N1
-200.00000
16383.50000
FORMAT "%10.5"
PHYS_UNIT "rpm"
/begin ANNOTATION
  ANNOTATION_LABEL "Description (coming from AUTOSAR DESC field)"
  /begin ANNOTATION_TEXT
    "Descr of CalprmEngNMin parameter\r\n"
  /end ANNOTATION_TEXT
/end ANNOTATION
/end CHARACTERISTIC
```

Figure 5.26.A2L Snippet for the above data

5.7 A2L File Contents

5.7.1 A2LProject Details

The A2LProject *STRUCTURE* supported by the RTA-A2L Generator is as follows:

```
/begin PROJECT Name LongIdentifier
      [-> HEADER]
      {-> MODULE}*
/end PROJECT
```

where:

Name	The "AR" has been considered as the <i>Name</i> for the <i>A2LProject</i> .
LongIdentifier	The "AR_A2L_Gen" has been considered as the <i>LongIdentifier</i> . The same value will be appearing as the <i>VERSION</i> information in the <i>MOD_PAR</i> block.
HEADER	The <i>A2LHeader</i> will be created and set to the <i>A2LProject</i> . See section <i>A2LHeader Details</i> (below).

<i>MODULE</i>	The <i>A2LModule</i> is created and all the module contents are loaded. The same <i>A2LModule</i> is added to the <i>A2LProject</i> . The <i>A2LProject</i> may contain zero or more <i>A2LModules</i> . The RTA-A2L Generator supports the single <i>MODULE</i> and all the <i>McSupport</i> files contents are mapped to the single <i>MODULE</i> .
---------------	---

5.7.2 A2LHeader Details

The A2LHeader has the following *STRUCTURE*:

```
/begin HEADER Comment
[ -> VERSION]
[ -> PROJECT_NO]
/end HEADER
```

Where:

<i>Comment</i>	No Header comment will be set by the RTA-A2L. Hence, by default, "" will be set.
<i>VERSION</i>	The header version will be set to <i>AR_A2L_1.0</i> .
<i>PROJECT_NO</i>	The Project number has been set to <i>AR_A2L_Gen_1_0</i> . The <i>A2LHeader</i> will be added to the <i>A2LProject</i> .

5.7.3 A2LModule Details

The A2LModule has the following *STRUCTURE*:

```
/begin MODULE Name LongIdentifier
[ -> MOD_PAR]
{ -> CHARACTERISTIC}*
{ -> MEASUREMENT}*
{ -> COMPU_METHOD}*
{ -> COMPU_TAB }*
{ -> COMPU_VTAB}*
{ -> RECORD_LAYOUT}*
{ -> UNIT}*
/end MODULE
```

<i>Name</i>	The Name of the module has been set to <i>AR</i> .
<i>LongIdentifier</i>	Currently, the RTA-A2L Generator does not set any <i>LongIdentifier</i> to the <i>A2LModule</i> . Hence, by default, "" will be set.

The *A2LModule* is then loaded with *CHARACTERISTICS*, *MEASUREMENTS*, *CompuMethod*, *CompuTab*, *CompuVTab*, *Units* and *RecordLayouts*.

5.7.4 MOD_PAR Block Details

For each Module, only one *MOD_PAR* block will appear in the A2L file. RTA-A2L currently supports the following attributes in the *MOD_PAR* block:

```
/begin MOD_PAR
Comment
VERSION
{ ->System_Constant} *
/end MOD_PAR
```

<i>Comment</i>	Currently, no information has been set for the comment of the <i>MOD_PAR</i> block. Hence, by default, " " value will be set.
<i>VERSION</i>	"AR_A2L_Gen" has been set for the <i>MOD_PAR</i> block and this should be equal to the Project's <i>LongIdentifier</i> .
<i>System_Constant</i>	The AUTOSAR file usually provides the <i>SystemConstant</i> details in two blocks. i.e. <i>SwSystemConstantValueSet</i> , which usually contains the <i>SystemConstant</i> reference to the <i>SystemConstant</i> , value and <i>SystemConstant</i> block.

A sample *SwSystemConstantValueSet*, which contains the *SwSystemConstantRef* and the *VALUE* tag has been provided below:

```
<SW-SYSTEMCONSTANT-VALUE-SET>
  <SW-SYSTEMCONSTANT-VALUES>
    <SW-SYSTEMCONST-VALUE>
      <SW-SYSTEMCONST-REF DEST="SW-
SYSTEMCONST">/RB/SwSystemConstants/StableSystemConstants/SystemConstant_Stable</SW-SYSTEMCONST-REF>
      <VALUE>0xFF</VALUE>
    </SW-SYSTEMCONST-VALUE>
  </SW-SYSTEMCONSTANT-VALUES>
</SW-SYSTEMCONSTANT-VALUE-SET>
```

Figure 5.27.AUTOSAR *SystemConstant VALUE* Set Example block

The *SwSystemConstantValueSet* above refers to the *SystemConstant* block below:

```
<?xml version="1.0" encoding="UTF-8"?>
<AUTOSAR xmlns="http://autosar.org/schema/r4.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://autosar.org/schema/r4.0 AUTOSAR_4-0-3.xsd">
  <ADMIN-DATA>
    <LANGUAGE>EN</LANGUAGE>
    <USED-LANGUAGES>
      <L-10 L="EN"
        xml:space="default">English</L-10>
    </USED-LANGUAGES>
  </ADMIN-DATA>
  <AR-PACKAGES>
    <AR-PACKAGE>
      <SHORT-NAME>RB</SHORT-NAME>
      <AR-PACKAGES>
        <AR-PACKAGE>
          <SHORT-NAME>SwSystemConstants</SHORT-NAME>
          <AR-PACKAGES>
            <AR-PACKAGE>
              <SHORT-NAME>StableSystemConstants</SHORT-NAME>
              <CATEGORY>STABLE</CATEGORY>
              <ELEMENTS>
                <SW-SYSTEMCONST>
                  <SHORT-NAME>SystemConstant_Stable</SHORT-NAME>
                  <SW-DATA-DEF-PROPS>
                    <SW-DATA-DEF-PROPS-VARIANTS>
                      <SW-DATA-DEF-PROPS-CONDITIONAL>
                        <SW-CALIBRATION-ACCESS>READ-WRITE</SW-CALIBRATION-ACCESS>
                      </SW-DATA-DEF-PROPS-CONDITIONAL>
                    </SW-DATA-DEF-PROPS-VARIANTS>
                  </SW-DATA-DEF-PROPS>
                </SW-SYSTEMCONST>
              </ELEMENTS>
            </AR-PACKAGE>
          </AR-PACKAGES>
        </AR-PACKAGE>
      </AR-PACKAGES>
    </AR-PACKAGE>
  </AR-PACKAGES>
</AUTOSAR>
```

Figure 5.28.STRUCTURE of the AUTOSAR SystemConstant

The *SystemConstant* will be created based on the *SW-CALIBRATION-ACCESS*. If the calibration access is of type *NOT-ACCESSIBLE*, or *SW-CALIBRATION-ACCESS* is unavailable (the default is *NOT-ACCESSIBLE*), then that *SystemConstant* will not be populated in the A2LModule and will not be created in the A2L file. If the input file doesn't contain the *SW-CALIBRATION-ACCESS*, by default, the *NOT-ACCESSIBLE* type will be considered as the access type.

In the A2L file, the *SystemConstant* will appear as follows:

System_Constant Name VALUE

Where the *SystemConstant* entry is created based on the order in which the *SwSystemConstantValueSet* has been provided. For each complete entry of the <SW-SYSTEMCONST-VALUE>, one *SystemConstant* entry will be created.

Name	The tag <SW-SYSTEMCONST-REF> will be fetched from the <SW-SYSTEMCONST-VALUE>. Corresponding to this AUTOSAR <i>systemConst</i> object, the <i>uniqueName</i> will be fetched and set as the <i>Name</i> of the <i>SystemConstant</i> .
------	--

VALUE	The value of the <VALUE> tag. If it is hexadecimal, then it will be converted to decimal. In AUTOSAR, only the physical values are supported. Hence, the value of the <i>SystemConstant</i> will always contain the physical value.
-------	---

Please refer to the *RTA-A2L Error Description* document for the list of log entries supported for *SystemConstant*.

5.7.5 A2L Unit Details

RTA-A2L supports the following *Unit* details:

```
/begin UNIT Name LongIdentifier Display Type
[ ->SI-EXPONENTS]
[ ->REF_UNIT]
[ ->UNIT_CONVERSION]
/end UNIT
```

Name	The value of the <i>uniqueName</i> created for the <UNIT> block.
LongIdentifier	Will be created by the value of the <LONG-NAME> tag.
Display	The value taken from the <DISPLAY-NAME> tag. If the <DISPLAY-NAME> tag is missing, or its value is empty or missing, then it will default to "" (empty string).
Type	Decided based on the <i>FactorToSiUnit</i> and <i>OffsetToSiUnit</i> values.
	If <i>FactorSiToUnit</i> and <i>OffsetSiToUnit</i> = 1.0 and 0.0 respectively, then <i>EXTENDED_SI Unit</i> will be set.
	If <i>FactorSiToUnit</i> and <i>OffsetSiToUnit</i> ≠ 1.0 and 0.0 respectively, then <i>DERIVED Unit</i> will be generated.
SI_EXPONENTS	If the AUTOSAR <UNIT> contains the <PHYSICAL-DIMENSION-REF>, then it will be converted to <i>SI-EXPONENTS</i> for the A2L <i>Unit</i> block.

The following tag names are referenced from the corresponding <PHYSICAL-DIMENSION> and mapped to the A2LModule:

<LENGTH-EXP>	length
<MASS-EXP>	Mass
<TIME-EXP>	Time
<CURRENT-EXP>	Electric Current
<TEMPERATURE-EXP>	Temperature
<MOLAR-AMOUNT-EXP>	Amount of Substance
<LUMINOUS-INTENSITY-EXP>	Luminous Intensity

If any of the *PhysicalDimension* values are not present in the input file, the default value "0" will be assigned to the corresponding A2LEntry of *SI_EXPONENT*.

If any of the input values is invalid (e.g. *FLOAT*, string, special characters) than a default value "0" will be assigned to the corresponding A2LEntry of the *SI_EXPO-*

NENT and a warning message will be logged.

RTA-A2L also supports the hexadecimal value for *PHYSICAL-DIMENSIONS*.

SL_EXPONENT is applicable for *Unit* Type *EXTENDED_SI*.

<i>REF_UNIT</i>	If the <i>Unit</i> Type is <i>DERIVED</i> , then the <REF_UNIT> tag will be set. The <i>Unit</i> will be decided and fetched from <i>UnitGroups</i> .
<i>UNIT_CONVERSION</i>	If the <i>Unit</i> Type is <i>DERIVED</i> , then the <i>UNIT_CONVERSION</i> is set from the <i>FactorToSiUnit</i> and <i>OffsetToSiUnit</i> tags.

 **NOTE**

The types *DERIVED*, *REF_UNIT* and *UNIT_CONVERSION* are applicable only in case the *Unit* is part of *UnitGroups*. Please refer to the *RTA-A2L Error Description* document for the log entry details for the A2L *Units*.

5.7.6 A2L CompuMethod Details

All the *CompuMethods* present in the *McSupport* files are written in the A2L file if the category is: *RAT_FUNC*, *TAB_VERB*, *TAB_INTP*, *TEXTTABLE*, *TAB_NOINTP*, *IDENTICAL*, *LINEAR* and *SCALE_LINEAR_AND_TEXTTABLE*.

Currently the RTA-A2L supports the following *CompuMethod* details:

```
/begin COMPU_METHOD
  Name
  LongIdentifier
  ConversionType
  Format Unit
  [-> COEFFS]
  [-> COEFFS_LINEAR]
  [-> FORMULA]
  [-> COMPU_TAB_REF]
  [-> STATUS_STRING_REF]
  [-> REF_UNIT]
/end COMPU_METHOD
```

<i>Name</i>	The value of the <i>uniqueName</i> corresponding to the AUTOSAR <i>Compu</i> Object.
<i>LongIdentifier</i>	Created by the value of <LONG-NAME> tag of the AUTOSAR input.
<i>ConversionType</i>	Currently, the RTA-A2L supports the <i>RAT_FUNC</i> , <i>TEXTTABLE</i> , <i>TAB_NOINTP</i> , <i>IDENTICAL</i> , <i>LINEAR</i> and <i>SCALE_LINEAR_AND_TEXTTABLE</i> types of <i>CompuMethods</i> , from AUTOSAR.
	It also supports the <i>TAB_VERB</i> and <i>TAB_INTP</i> <i>CompuMethods</i> , which are of type <i>MSR</i> . Warning messages will be provided.
	The <i>TEXTTABLE</i> type of <i>CompuMethods</i> from AUTOSAR is handled as the <i>TAB_VERB</i> type and the <i>TAB_VERB</i> will be appearing as the <i>ConversionType</i> in the generated A2L file.

	The <i>IDENTICAL</i> and the <i>LINEAR</i> type of <i>CompuMethods</i> from AUTOSAR are handled as the <i>RAT_FUNC</i> type of <i>CompuMethod</i> and the <i>RAT_FUNC</i> will be appearing as the <i>ConversionType</i> in the generated A2L file.
	The <i>SCALE_LINEAR_AND_TEXTABLE</i> type of <i>CompuMethod</i> from AUTOSAR has both the features of <i>TEXTABLE</i> and the <i>LINEAR CompuMethod</i> . <i>LINEAR</i> will be appearing as the <i>ConversionType</i> in the generated A2L file.
<i>Format</i>	The value from the <DISPLAY-FORMAT> will be fetched and set as the <i>Format</i> for the <i>CompuMethod</i> .
<i>Unit</i>	The <i>Unit</i> will be fetched from the <UNIT-REF> tag of the <i>CompuMethod</i> . If an A2L <i>Unit</i> block already appears in the A2L module, the corresponding A2L <i>Unit</i> block will be considered.
	The Display from the A2L <i>Unit</i> block will be set as the <i>Unit</i> for the <i>CompuMethod</i> .
	If the <UNIT-REF> tag is missing/proxy, or the referenced <i>Unit</i> is missing in the A2L module, then by default "" (empty string) will be set as the <i>Unit</i> .

COEFS

- The *COEFS* will be set to the *CompuMethod* of type *RAT_FUNC/IDENTICAL/LINEAR*.
- The *COEFS* will be set for the *RAT_FUNC* type *CompuMethod* block in the A2L file, if the *CompuContent* exists within the <COMPU-PHYS-TO-INTERNAL> or with in the <COMPU-INTERNAL-TO-PHYS> for the *RAT_FUNC* and for the *LINEAR* type of *CompuMethods*.
- The *COEFS* values will be fetched from the <COMPU-PHYS-TO-INTERNAL> or <COMPU-INTERNAL-TO-PHYS>/<COMPU-SCALES>/<COMPU-SCALE>/<COMPU-RATIONAL-COEFS>/<COMPU-NUMERATOR>/<V> tags and <COMPU-DENOMINATOR>/<V> tags.
- The *CompuMethods* cannot have a single <COMPU-NUMERATOR>/<V> tag and <COMPU-DENOMINATOR>/<V> tag. (Either <COMPU-NUMERATOR> or <COMPU-DENOMINATOR> or both should have 2 <V> tags.)
- The *CompuMethods* with a single <COMPU-DENOMINATOR>/<V> tag cannot have a denominator value equal to zero.
- The hexadecimal values will be converted into the decimal values, before writing to the A2L file.
- For the *IDENTICAL* type of *CompuMethods*, the *CompuScales* defined on the *SwCompuScales* will be ignored and the default *COEFS* are always set i.e. 0, 1, 0, 0, 0, 1.
- If the type of the *CompuMethod* is *IDENTICAL*, and if it contains the missing *CompuContent*, then the default behavior of one-to-one *COEFS* are created and assigned to the A2L *CompuMethod* i.e. *COEFS* are set to the default values 0, 1, 0, 0, 0, 1.

The *COEFFS* values will be set as *COEFFS a b c d e f*.

The figure below shows the *COEFFS* generated for the *RAT_FUNC* type of *CompuMethod*, when the *compuContent* exists within the `<COMPU-PHYS--TO-INTERNAL>` and `<COMPU-INTERNAL-TO-PHYS>`.

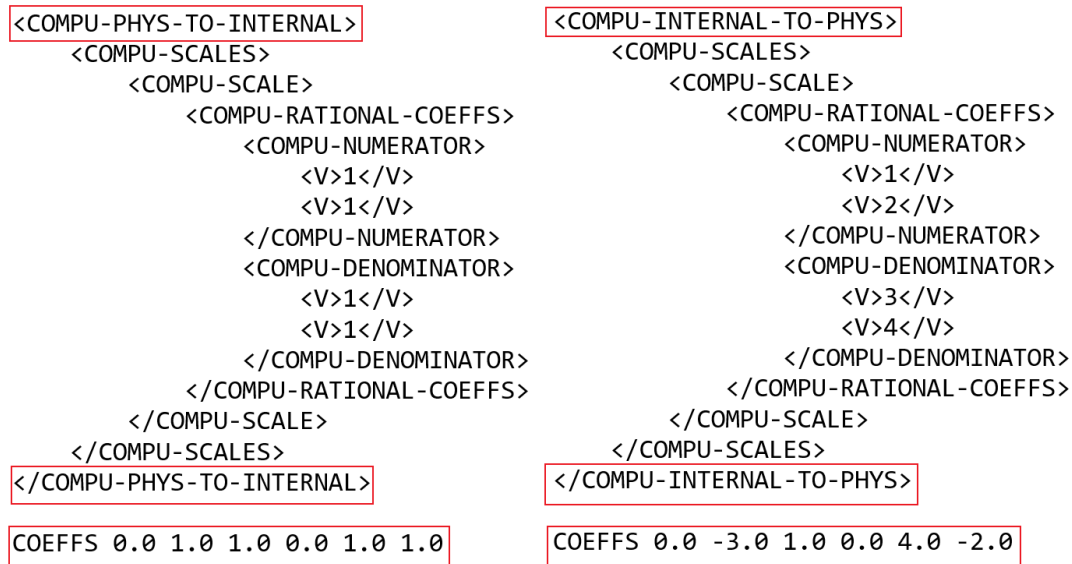


Figure 5.29. *COEFFS* for *RAT_FUNC*

COEFFS_LINEAR

- The *COEFFS_LINEAR* value will be set for the *CompuMethods* whose category is of type *SCALE_LINEAR_AND_TEXTABLE*.
- The *COEFFS_LINEAR* values will be fetched from the `<COMPU-PHYS-TO-INTERNAL>` or `<COMPU-INTERNAL-TO-PHYS>`/`<COMPU-SCALES>`/`<COMPU-SCALE>`/`<COMPU-RATIONAL-COEFFS>`/`<COMPU-NUMERATOR>`/`<V>` tags.

FORMULA

The *FORMULA* is fetched from the `<SW-DATA-DEPENDENCY-FORMULA>` tag. (Please refer to Section 5.7.6.2 for more details.)

COMPU_TAB_REF

If the *CompuMethod* is of type *TAB_VERB/TEXT_TABLE/TAB_INTP/TAB_NOINTP*, the corresponding *COMPU_TAB* or *COMPU_VTAB* will be created and provided as the reference in the *COMPU_METHOD*, with the *COMPU_TAB_REF* value.

STATUS_STRING_REF

If the *CompuMethod* is of type *SCALE_LINEAR_AND_TEXTABLE*, the corresponding *COMPU_TAB* or *COMPU_VTAB* will be created and provided as the reference in the *COMPU_METHOD*, with a *STATUS_STRING_REF* value.

REF_UNIT

- The *Unit* will be fetched from the <UNIT-REF> tag.
- Corresponding to this AUTOSAR *Unit* object, the *A2LUnit* object will be fetched from the A2LModule. This *A2LUnit* will be set as the *Unit* for the *CompuMethod*.
- This attribute will not be created in the A2L file if the <UNIT-REF> tag is missing/proxy/invalid.

Please refer to the *RTA-A2L Error Description* document for the list of log entries supported for the *CompuMethod* block.

5.7.6.1 Creation of the Default CompuMethod Block

If the *VALUE* type and *STRING* type of <MC-DATA-INSTANCE> of the <MC-PARAMETER-INSTANCES>/<MC-VARIABLE-INSTANCES> contain the missing/invalid/proxy <COMPU-METHOD-REF>, then a default *CompuMethod* block will be created in the A2L file.

The default *CompuMethod* will follow the *STRUCTURE* of the *CompuMethod* of type AUTOSAR *IDENTICAL*. The MSR OneToOne *CompuMethod STRUCTURE* has been followed to create the default *CompuMethod* Type.

The Name of the *CompuMethod* will be set to *DefaultIdenticalCompu*. The *long-Name* will be considered as an empty string "".

The type will be considered as the *RAT_FUNC*. The *Unit* has been set to the empty display string (""). The *Format* has been set to the default value %10.5.

The *COEFFS* are set to the default values 0, 1, 0, 0, 0, 1.

5.7.6.2 Creation of FORM CompuMethod Block

If the *McVariableInstance* contains <SW-IS-VIRTUAL>1</SW-IS-VIRTUAL> and <SW-DATA-DEPENDENCY>, then a *FORM CompuMethod* block will be created in the A2L file.

The *FORM CompuMethod* will follow the *STRUCTURE* of type AUTOSAR FORM:

- The *ShortName* of the *CompuMethod* will be set as <ShortNameOf McDataInstance>_CompuMethod_RTAA2L.
- The *longName* will be considered as an empty string "".
- The type will be considered as *FORM*.
- The display format has been set to the default value %10.5.
- The *Unit* reference has been set to the empty string "".
- The formula to be written in the *CompuMethod* is fetched from the *McDataInstance*.

5.7.7 COMPU_TABLE Details

The *COMPU_TAB* will be created if the *CompuMethod* type is *TAB_INTP* or *TAB_NOINTP*. *COMPU_VTAB* will be created if the *CompuMethod* category is *TAB_VERB*, *TEXTTABLE* or *SCALE_LINEAR_AND_TEXTTABLE*. They will not be

created for the types *RAT_FUNC*, *IDENTICAL* or *LINEAR*. *COMPU_TAB* or *COMPU_VTAB* will be created at the same time as the *CompuMethod* block.

RTA-A2L currently supports the following *CompuTable* details:

```
/begin COMPU_TAB/COMPU_VTAB Name LongIdentifier
      ConversionType
      NumberValuePairs
      {InVal OutVal}*
/end COMPU_TAB/COMPU_VTAB
```

<i>Name</i>	The value of the <i>uniqueName</i> created for the input <COMPU-METHOD> block.
<i>LongIdentifier</i>	Created by the <LONG-NAME> tag of the input <COMPU-METHOD>.
<i>ConversionType</i>	If the input <i>CompuMethod</i> type is of type <i>TAB_VERB</i> , <i>TEXT-TABLE</i> or <i>SCALE_LINEAR_AND_TEXTTABLE</i> , then the <i>ConversionType</i> will also be set as <i>TAB_VERB</i> .
	If the input <i>CompuMethod</i> type is <i>TAB_INTP</i> then the <i>ConversionType</i> will be set as <i>TAB_INTP</i> .
	If the <i>CompuMethod</i> type is <i>TAB_NOINTP</i> then the <i>ConversionType</i> will also be appeared as <i>TAB_NOINTP</i> .
<i>NumberValuePairs</i>	This will be the total Number of { <i>InVal</i> , <i>OutVal</i> } pairs.
{ <i>InVal OutVal</i> }	See sections below.

5.7.7.1 The Mapping of A2L COMPU_TAB InVal and OutVal Pairs

For *COMPU_TAB*, the {*InVal*, *OutVal*} pairs will be considered only if the <COMPU-SCALE>s are of type <COMPU-PHYS-TO-INTERNAL>:

<i>InVal</i>	For <i>COMPU_TAB</i> , <i>InVal</i> will be the value of <VT> from the <COMPU-SCALE>.
<i>OutVal</i>	The <i>OutVal</i> will be the value of <LOWER-LIMIT>.

5.7.7.2 The Mapping of A2L COMPU_VTAB InVal and OutVal Pairs

For *COMPU_VTAB*, the {*InVal*, *OutVal*} pairs will be considered only if the <COMPU-SCALE>s are of type <COMPU-INTERNAL-TO-PHYS>:

<i>InVal</i>	For <i>COMPU_VTAB</i> , the <i>InVal</i> will be the value of <LOWER-LIMIT> from the <COMPU-SCALE>.
<i>OutVal</i>	The <i>OutVal</i> will be the value of <VT>.

Please refer to the *RTA-A2L Error Description* document for the log entry details for the A2L *CompuTable* block.

5.7.8 RECORD_LAYOUT Details

The *RECORD_LAYOUT* will be created and written to the A2L file only if it has been referenced from any of the *McParameter* instances and only if that *RecordLayout*

exists.

If the `<MC-PARAMETER-INSTANCE>/<MC-DATA-INSTANCE>` of type *VALUE* references a *RecordLayout*, which does not exist in the input file, then a default *RecordLayout* will be created. For the `<MC-PARAMETER-INSTANCE>` of type *CURVE* or *MAP*, the default *RecordLayout* is not created/mapped and the *CHARACTERISTIC* block will not be generated in the A2L file.

The *STRUCTURE* of the *RECORD_LAYOUT* in the A2L file is as shown below:

```

/begin RECORD_LAYOUT      name
    RESERVED              <Pos> <DatTypW>
    NO_AXIS_PTS_X         <Pos> <DatTypW>
    AXIS_PTS_X            <Pos> <DatTypW> INDEX_DECR DIRECT
    FNC_VALUES            <Pos> <DatTypW> COLUMN_DIR DIRECT
    RESERVED              <Pos> <DataSize>
/end RECORD_LAYOUT

```

<i>name</i>	Depends on the <code><SHORT-NAME></code> of the input <code><SW-RECORD-LAYOUT></code> and the <code><BASE-TYPE-REF></code> s of the <code><MC-DATA-INSTANCE></code> and their <code><SW-CAL-PRM-AXIS></code> .
	If the <code><RECORD-LAYOUT-REF></code> does not exist in the <code><MC-DATA-INSTANCE></code> , then the name will be set to <code><DefaultValueRecordLayout>_<ShortName></code> of the <i>baseType</i> . For the <i>VALUE</i> type of the <code><MC-DATA-INSTANCE></code> , and for other types, an error message will be thrown.
<code><Pos></code>	Decided according to the order in which the <i>SwRecordLayoutVs</i> appears in the <i>RecordLayout</i> of the <i>McSupport</i> file.
<code><DatTypW></code>	Will be set if the <code><BASE-TYPE-REF></code> exists in the <i>McDataInstance</i> of the <i>McParameter</i> instances. Or else then, by default, <i>UBYTE</i> will be set.
	The referenced <code><BASE-TYPE></code> will be fetched and converted to the A2L <i>DataType</i> type and set as the <i>RecordLayout</i> <code><DatTypW></code> .
<code><DataSize></code>	Will be set if the <code><BASE-TYPE-REF></code> exists in the <i>McDataInstance</i> of the <i>McParameter</i> instances. Or else then, by default, <i>UBYTE</i> will be set.
	The referenced <code><BASE-TYPE></code> will be fetched and converted to the A2L <i>DataSizeType</i> and set as the <i>RecordLayout</i> .
<i>COLUMN_DIR</i>	Will be set as <i>COLUMN_DIR</i> for the <i>VALUE</i> type of <i>RECORD_LAYOUT</i> .
<i>DIRECT</i>	Will be set as <i>DIRECT</i> itself for the <i>VALUE</i> type of <i>RecordLayout</i> .
<i>INDEX_DECR/INDEX_INCR</i>	Is decided according to the <code><SW-RECORD-LAYOUT--GROUP-STEP></code> value. If the value is <code>-1</code> , then it is <i>INDEX_DECR</i> . Otherwise, it is <i>INDEX_INCR</i> .

<i>STATIC_RECORD_LAYOUT</i>	This keyword is generated for a <i>RecordLayout</i> block if the switch <i>enableStaticRecordLayouts</i> = <i>TRUE</i> or the switch <i>Asap2Version</i> is set as <i>1.60</i> or <i>1.61</i> .
<i>STATIC_ADDRESS_OFFSETS</i>	This keyword is generated for each <i>RecordLayout</i> . If the switch <i>enableStaticAddressOffsets</i> = <i>TRUE</i> or <i>Asap2Version</i> is set as <i>1.70</i> .

```

FNC_VALUES <Pos> <DatTypW> COLUMN_DIR DIRECT
FNC_VALUES <Pos> <DatTypW> COLUMN_DIR DIRECT
FNC_VALUES <Pos> <DatTypW> ROW_DIR DIRECT
FNC_VALUES <Pos> <DatTypW> COLUMN_DIR DIRECT
FNC_VALUES <Pos> <DatTypW> COLUMN_DIR DIRECT
IDENTIFICATION <Pos> <DatTypW>
AXIS_PTS_X <Pos> <DatTypX> <SstRicht> DIRECT
AXIS_PTS_Y <Pos> <DatTypY> <SstRicht> DIRECT
AXIS_PTS_Z <Pos> <DatTypZ> <SstRicht> DIRECT
AXIS_PTS_4 <Pos> <DatTyp4> <SstRicht> DIRECT
AXIS_PTS_5 <Pos> <DatTyp5> <SstRicht> DIRECT
SRC_ADDR_X <Pos> <AdrTypX>
SRC_ADDR_Y <Pos> <AdrTypY>
RIP_ADDR_X <Pos> <AdrTypX>
RIP_ADDR_Y <Pos> <AdrTypY>
NO_AXIS_PTS_X <Pos> <DatTyp>
NO_AXIS_PTS_Y <Pos> <DatTyp>
NO_AXIS_PTS_Z <Pos> <DatTyp>
NO_AXIS_PTS_4 <Pos> <DatTyp>
NO_AXIS_PTS_5 <Pos> <DatTyp>
OFFSET_X <Pos> <DatTyp>
OFFSET_Y <Pos> <DatTyp>
SHIFT_OP_X <Pos> <DatTyp>
SHIFT_OP_Y <Pos> <DatTyp>
DISTANCE_OP_X <Pos> <DatTyp>
DISTANCE_OP_Y <Pos> <DatTyp>
RESERVED <Pos> <DataSize>

```

Of the above parameters, only those in the table below are implemented. The corresponding data types are generated according to the Axis to which they belong.

<i>FNC_VALUES</i>	Generated from the value of the <SW-RECORD-LAYOUT--GROUP>, which contains <i>VALUE</i> in the <SW-RECORD-LAYOUT-V-PROP>. The <i>VALUE</i> Axis (i.e. <i>0</i> or <i>V</i>) is used.
<i>NO_AXIS_PTS_X</i>	Generated from the value of the <SW-RECORD-LAYOUT--GROUP>, which contains <i>COUNT</i> in the <SW-RECORD-LAYOUT-V-PROP>. The <i>X</i> -Axis (i.e. <i>1</i> or <i>X</i>) is used.
<i>NO_AXIS_PTS_Y</i>	Generated from the value of the <SW-RECORD-LAYOUT--GROUP>, which contains <i>COUNT</i> in the <SW-RECORD-LAYOUT-V-PROP>. The <i>y</i> -Axis (i.e., <i>2</i> or <i>Y</i>) is used.
<i>NO_AXIS_PTS_Z</i>	Generated from the value of the <SW-RECORD-LAYOUT--GROUP>, which contains <i>COUNT</i> in the <SW-RECORD-LAYOUT-V-PROP>. The <i>Z</i> -Axis or <i>z1</i> Axis (i.e., <i>3</i> or <i>Z</i> or <i>Z1</i>) are used.
<i>NO_AXIS_PTS_4</i>	Generated from the value of the <SW-RECORD-LAYOUT--GROUP>, which contains <i>COUNT</i> in the <SW-RECORD-LAYOUT-V-PROP>. The <i>z2</i> Axis (i.e., <i>4</i> or <i>Z2</i>) is used.
<i>NO_AXIS_PTS_5</i>	Generated from the value of the <SW-RECORD-LAYOUT--GROUP>, which contains <i>COUNT</i> in the <SW-RECORD-LAYOUT-V-PROP>. The <i>z3</i> Axis (i.e., <i>5</i> or <i>Z3</i>) is used.
<i>AXIS_PTS_X</i>	Generated from the value of the <SW-RECORD-LAYOUT--GROUP>, which contains <i>VALUE</i> in the <SW-RECORD-LAYOUT-V-PROP>. The <i>X</i> -Axis (i.e., <i>1</i> or <i>X</i>) is used.
<i>AXIS_PTS_Y</i>	Generated from the value of the <SW-RECORD-LAYOUT--GROUP>, which contains <i>VALUE</i> in the <SW-RECORD-LAYOUT-V-PROP>. The <i>y</i> -Axis (i.e., <i>2</i> or <i>Y</i>) is used.

<i>AXIS_PTS_Z</i>	Generated from the value of the <SW-RECORD-LAYOUT--GROUP>, which contains <i>VALUE</i> in the <SW-RECORD-LAYOUT-V-PROP>. The <i>Z-Axis</i> or <i>z1 Axis</i> (i.e., <i>3</i> or <i>Z</i> or <i>Z1</i>) are used.
<i>AXIS_PTS_4</i>	Generated from the value of the <SW-RECORD-LAYOUT--GROUP>, which contains <i>VALUE</i> in the <SW-RECORD-LAYOUT-V-PROP>. The <i>z2 Axis</i> (i.e., <i>4</i> or <i>Z2</i>) is used.
<i>AXIS_PTS_5</i>	Generated from the value of the <SW-RECORD-LAYOUT--GROUP>, which contains <i>VALUE</i> in the <SW-RECORD-LAYOUT-V-PROP>. The <i>z3 Axis</i> (i.e., <i>5</i> or <i>Z3</i>) is used.
<i>RESERVED</i>	Generated from the value of the <SW-RECORD-LAYOUT--GROUP>, which contains <i>GAPS</i> in the <SW-RECORD-LAYOUT-V-PROP>. As this does not belong to any specific <i>Calprm</i> Axis description, the data type (actually <i>DataSet</i> type) is taken from the <BASE-TYPE-REF> inside <SW-RECORD-LAYOUT-V>.
	A <SW-RECORD-LAYOUT-V> must specify <BASE-TYPE-REF> as mandatory. Also, according to ASAP specification only <i>BYTE</i> , <i>WORD</i> and <i>LONG</i> are valid data size values. A <i>DataSet</i> (<i>UBYTE</i> , <i>SBYTE</i> , <i>UWORD</i> etc.) cannot be used in place of the data size.
	Validations are made for the <i>baseType</i> fetched from the <SW-RECORD-LAYOUT-V> and error messages are thrown i.e. <i>A2L_MCS_001</i> and <i>A2L_MCS_016</i> from the table in <i>The List of Log Entries Supported for The RecordLayouts</i> .

5.7.8.1 Naming Convention for a RecordLayout

When generating *RecordLayouts* in AUTOSAR, *baseTypes* of <SW-RECORD-LAYOUT> are considered for generating data types of the parameters inside the *RecordLayout*. If one or many *baseTypes* are missing/invalid in the <SW-RECORD-LAYOUT> then *baseTypes* from <MC-DATA-INSTANCE> and its *CalprmAxis* are considered for generating data types of the parameters inside the *RecordLayout*.

- In the case of valid *baseTypes* present in the <SW-RECORD-LAYOUT>, the name of the *RecordLayout* generated in the A2L file is same as the *Short-Name* of the <SW-RECORD-LAYOUT>.
- In the case of *baseTypes* missing/invalid in the <SW-RECORD-LAYOUT>, then if two <MC-DATA-INSTANCE>s refer to the same <SW-RECORD-LAYOUT>, and if the <BASE-TYPE>s of the <MC-DATA-INSTANCE> are different, then it overrides the first *RecordLayout* with the second *RecordLayout*, making the wrong *RecordLayout* for the first <MC-DATA-INSTANCE>. Therefore, the name should be unique for different *McDataInstances* with different *baseTypes*.
- Also in the case of *baseTypes* missing/invalid in the <SW-RECORD-LAYOUT>, the name of the A2L *RecordLayout* is created by appending the <SHORT-NAME> of the <SW-RECORD-LAYOUT> with the <BASE-TYPE>s of the <MC-DATA-INSTANCE>, which references the *RecordLayout*, and their

Axis separated with a single “_”.

For example, if a *CHARACTERISTIC* of type *MAP* and *baseType sint32* references a *RecordLayout* with the name *RecordLayoutExample* as well as the *baseTypes uint16* and *uint32* from *<SW-CALPRM-AXIS>*, then the *RecordLayout* name generated is “RecordLayoutExample_sint32_uint16_uint32”.

If the *baseType* is missing in the *<SW-CALPRM-AXIS>*, then the *baseType* of the reference variable which is referenced by the *<SW-CALPRM-AXIS>* is taken for the corresponding *RecordLayout* Axis. Preference is given to the *baseType* of inside *<SW-CALPRM-AXIS>*.

Validations are made for all the *baseTypes* fetched from *<SW-RECORD-LAYOUT>* and if a *baseType(s)* is invalid/missing then the *baseTypes* from *<SW-CALPRM-AXIS>* are validated and error messages are thrown i.e. *A2L_MCS_001* and *A2L_MCS_016* from the table in *The List of Log Entries Supported for The RecordLayouts*.

If the *RecordLayout* is duplicated in the different *ARPackages* then, for the second entry, the *_running* number will be appended and the same will be updated in the referenced *CHARACTERISTIC* block.

A detailed *RecordLayout* behavior with example is given in the section *RecordLayout Behavior with Examples*.

Please refer to the *RTA-A2L Error Description* document for the list of log entries supported for the *RecordLayouts*.

5.7.8.2 Creation of a Default RecordLayout

If the *VALUE* and *STRING* types of *<MC-DATA-INSTANCE>* of the *<MC-PARAMETER-INSTANCES>* contain a proxy/invalid/missing *<SW-RECORD-LAYOUT-REF>*, a default *RecordLayout* will be created and assigned to the *CHARACTERISTIC* block. The default *RecordLayout* will be created with the following details:

<i>Name</i>	DefaultValueRecordLayout_<SHORT-NAME> of the <BASE--TYPE> (<BASE--TYPE> is the value from the <BASE-TYPE-REF> of the <MC-DATA-INSTANCE>).
<Pos>	The <Pos> value has been assumed to be 1.
<DatType>	This value will be set if the <BASE-TYPE-REF> exists in the <i>McDataInstance</i> of the <i>McParameter</i> Instances. Or else then, by default, <i>UBYTE</i> will be set. The referenced <BASE-TYPE> will be fetched and converted to the A2L <i>DataType</i> and set as the <i>RecordLayout</i> <DataType>.
<i>COLUMN_DIR</i>	This value will be set as <i>COLUMN_DIR</i> for the <i>VALUE</i> type of <i>RecordLayout</i> .
<i>DIRECT</i>	This value will be set as <i>DIRECT</i> for the <i>VALUE</i> type of <i>RecordLayout</i> .

5.7.8.3 RecordLayout Behaviour with Examples

- If a Calprm<MC-DATA-INSTANCE> references a <SW-RECORD-LAYOUT> *RecLayout* with all *baseTypes* valid, then the *RecordLayout* is generated with the name *RecLayout* and the *baseTypes* are taken from <SW-RECORD-LAYOUT>.
- If, for a *RecordLayout RLayout*, the *baseTypes* are taken from <SW-RECORD-LAYOUT>, then the name convention is *RLayout* without any *baseType* postfix. And if, for a *RecordLayout RLayout*, the *baseTypes* are taken from <MC-DATA-INSTANCE>, then the name convention is *RLayout_basetypesFromMcDataInstance* (e.g. *RLayout_uint8_sint8*).
- If a Calprm<MC-DATA-INSTANCE> references a <SW-RECORD-LAYOUT> *RecLayout* with any/many invalid *baseTypes*, then the *RecordLayout* is generated with the name *RecLayout_basetypesMcDataInstance*. The *baseTypes* are taken from <MC-DATA-INSTANCE>.

e.g. if a Calprm<MC-DATA-INSTANCE> of type *CURVE* references a <SW-RECORD-LAYOUT> *RecLayout* and <MC-DATA-INSTANCE>, with *baseType uint8* and X-Axis *baseType uint16*, then the *RecordLayout* generated will have the name "RecLayout_uint8_unit16".

The McDataInstances and the RecordLayouts axes required are provided as below:

- A *RecordLayout*, which has been referenced by the *VALUE* type of *McDataInstance*, should support at least one V-Axis with the correct *baseType*. If the *baseType* is missing/invalid at the *RecordLayout* level, then the *baseType* will be fetched from the *Parent McDataInstance*.
- A *RecordLayout*, which is referenced from the *COM_AXIS*, should support one Axis (X/Y). If any of these axes contains missing/invalid *baseTypes*, then all the *baseTypes* will be fetched from the *McDataInstance Axis*.
- A *RecordLayout*, which has been referenced by the *SIMPLE CURVE*, should support 2 axes (V, X). If either of these axes contains missing/invalid *baseTypes*, then the X/Y *baseTypes* will be fetched from the *McDataInstance Axis* and, for V, from the *Parent McDataInstance*.
- A *RecordLayout*, which has been referenced from the *SIMPLE MAP*, should support 3 axes (V, X, Y). If any of these axes contains missing/invalid *baseTypes* then, for the X-Axis, they will be fetched from the *McDataInstance 1st Axis baseType* and, for the Y-Axis, from the *McDataInstance 2nd Axis baseType*. For V, the *baseTypes* will be fetched from the *Parent McDataInstance*.
- A *RecordLayout*, which has been referenced from the *SIMPLE CUBOID*, should support 4 axes (V, X, Y, Z). If any of these axes contains the missing /invalid *baseTypes* then: for the X-Axis, they will be fetched from the *McDataInstance 1st Axis baseType*; for the Y-Axis, from the *McDataInstance 2nd Axis baseType*; and for the Z-Axis, from the *McDataInstance 3rd Axis baseType*. For V, the *baseTypes* will be fetched from the *Parent*

McDataInstance.

- A *RecordLayout*, which has been referenced from the *SIMPLE CUBE_4*, should support 5 axes (V, X, Y, Z1, Z2). If any of these axes contains the missing/invalid *baseTypes* then: for the X-Axis, *baseTypes* will be fetched from the *McDataInstance* 1st Axis *baseType*; for the Y-Axis, from the *McDataInstance* 2nd Axis *baseType*; for the Z1 Axis, from the *McDataInstance* 3rd Axis *baseType*; and for the Z2 Axis, from the *McDataInstance* 4th Axis *baseType*. For V, the *baseTypes* will be fetched from the *Parent McDataInstance*.
- A *RecordLayout*, which has been referenced from the *SIMPLE CUBE_5*, should support 6 axes (V, X, Y, Z1, Z2, Z3). If any of these axes contains the missing/invalid *baseTypes* then: for the X-Axis, *baseTypes* will be fetched from the *McDataInstance* 1st Axis *baseType*; for the Y-Axis, from the *McDataInstance* 2nd Axis *baseType*; for the Z1 Axis, from the *McDataInstance* 3rd Axis *baseType*; for the Z2 Axis, from the *McDataInstance* 4th Axis *baseType*; and for the Z3 Axis, from the *McDataInstance* 5th Axis *baseType*. For V, the *baseTypes* will be fetched from the *Parent McDataInstance*.
- A *RecordLayout*, which has been referenced from the *GROUPED CURVE /GROUPED MAP*, should support 1 Axis (V). If this *baseType* is missing/invalid then, for V, it will be fetched from the *Parent McDataInstance*.

If the valid *baseTypes* don't exist at the *RecordLayout* or *McDataInstance* level, then no *RecordLayout* or *CHARACTERISTIC* blocks will be created.

5.7.8.4 FNC_VALUES Behaviour for McDataInstance of Category MAP

- If the *McDataInstance* is of type *MAP* and the referenced <SW-RECORD-LAYOUT> has multiple *SwRecordLayoutGroups*, then the type is checked to generate *FNC_VALUES*.
- If the type of *SwRecordLayoutGroup* is *COLUMN_DIR*, then *FNC_VALUES* is generated as *COLUMN_DIR*.
- If the type of *SwRecordLayoutGroup* is *ROW_DIR*, then *FNC_VALUES* is generated as *ROW_DIR*.
- If the type is other than *COLUMN_DIR* or *ROW_DIR*, then it is ignored.
- If the type is missing, then the order of the V-Axis and V-Index is calculated.
- If the first V-Axis is 1 and V-Index is X and if the next V-Axis is 2 and V-Index is Y, then the *FNC_VALUES* is generated as *COLUMN_DIR*.
- If the first V-Axis is 2 and V-Index is Y and if the next V-Axis is 1 and V-Index is X, then the *FNC_VALUES* is generated as *ROW_DIR*.

5.7.8.5 FNC_VALUES Behaviour for McDataInstance of Category CUBOID, CUBE_4 and CUBE_5

- If the *McDataInstance* is of type *CUBOID*, *CUBE_4* or *CUBE_5* and the referenced <SW-RECORD-LAYOUT> has multiple *SwRecordLayoutGroups*, then the type is checked to generate *FNC_VALUES*.

- If the type of *SwRecordLayoutGroup* is *COLUMN_DIR*, then *FNC_VALUES* is generated as *COLUMN_DIR*.
- If the type of *SwRecordLayoutGroup* is *ROW_DIR*, then *FNC_VALUES* is generated as *ROW_DIR*.
- If the type is other than *COLUMN_DIR* or *ROW_DIR*, then *FNC_VALUES* is generated as *COLUMN_DIR* by default.
- If the type is missing, then *FNC_VALUES* is generated as *COLUMN_DIR* by default.

5.7.9 CHARACTERISTIC Details

Currently the RTA-A2L supports *VALUE*, *VAL_BLK*, *CURVE* (Simple and Grouped), *MAP* (Simple and Grouped), *COM_AXIS*, *STRUCTURE*, *ARRAY*, *STRING* and *BOOLEAN* types of *McDataInstances* as input to create the *CHARACTERISTIC* blocks.

- If the input <MC-DATA-INSTANCE> is of type *NOT-ACCESSIBLE*, or <SW--CALIBRATION-ACCESS> is unavailable, then no *CHARACTERISTIC* blocks will be created.

In the case of the *GROUPED CURVE/MAP*, if <MC-DATA-INSTANCE>s are of type *NOT-ACCESSIBLE*, or <SW-CALIBRATION-ACCESS> is unavailable for the type *COM_AXIS*, then these *GROUPED CURVE/MAP CHARACTERISTIC* blocks will not be generated in the A2L file and an error message will be logged.

The following *CHARACTERISTIC* details are supported by RTA-A2L:

```

/begin CHARACTERISTIC
  Name LongIdentifier
  Type
  Address
  Deposit
  MaxDiff Conversion
  LowerLimit
  UpperLimit
  [-> FORMAT]
  [-> PHYS_UNIT]
  [-> READ-ONLY]
  [-> MAX_REFRESH]
  [-> BYTE_ORDER]
  {-> ANNOTATION} *
  {[> AXIS_DESCR]}*
  {[> BIT_MASK]}
  {[> IF_DATA]}
  {[> NUMBER]}
  {[> ENCODING]}
  {[> MATRIX_DIM]}
  {[> STEP_SIZE]}
/end CHARACTERISTIC

```

Name

The customized *ShortName* of the *McDataInstance*. Please refer to Section 5.6.3 for more details.

LongIdentifier

Will be created using the <LONG-NAME> tag of the *McDataInstance*. Please refer to Section 5.7.15.2 for more details.

Type

The *VALUE* fetched from the <CATEGORY> of the *McDataInstance*. Only the primitive types *VALUE*, *VAL_BLK*, *CURVE*, *MAP*, *CUBOID*, *CUBE_4*, *CUBE_5*, *STRING*, *BOOLEAN* have been supported, to create the *CHARACTERISTIC* block by the RTA-A2L. The complex types *STRUCTURE* and *ARRAY* are supported. For the *BOOLEAN* type of *McDataInstance*, the type in the A2L will appear as *VALUE*. For the *STRING* type of *McDataInstance*, the type in the A2L will appear as *ASCII*.

Address

The *Address VALUE* will be set to the dummy *VALUE 0x0000* by RTA-A2L.

Deposit

To set the *Deposit*, we would be referring the <SW-RECORD-LAYOUT-REF>. (Please refer to Section 5.7.8 for more details.).

This newly-created A2L *RecordLayout* object will be fetched from the A2LModule and assigned as the *Deposit* to the *CHARACTERISTIC* block.

If the <SW-RECORD-LAYOUT-REF> is missing/proxy/invalid for the *VALUE* / *VAL_BLK* / *BOOLEAN* types of *McDataInstance* in the input file, then a default *RecordLayout* object will be created with the name of the *RecordLayout* as "DefaultValueRecordLayout_<SHORT-NAME> of the referenced <BASE-TYPE>". It will be added to the A2LModule and assigned to the *CHARACTERISTIC* block.

MaxDiff

Calculated as the difference between the *UpperLimit* and *LowerLimit* of the *CHARACTERISTIC* block. Before setting the *MaxDiff* value, the *UpperLimit* and the *LowerLimit* values will be set to the *CHARACTERISTIC* block $\text{MaxDiff} = (\text{UpperLimit} - \text{LowerLimit})$.

When the *UpperLimit* and *LowerLimit* values are derived from the *baseType* rather than the *dataConstraint*, and the *baseType* is of type *float64*, the difference between the upper and lower limits will result in a very large value greater than the limit of float 64. In such instances, the *maxDiff* value will be set to the maximum limit of *float64*, and the value will be formatted according to the display format.

Please refer to *Generation of A2LLimits based on the input display format* in Section 5.7.22 for more details.

e.g.:

- If the display format is %8.4 then the *MaxDiff* will be generated as 1.7976E308.
- If the display format is %5.6 then the *MaxDiff* will be generated as 1.797693E308.

ENCODING

Specifies the type of *ENCODING* used when the *CHARACTERISTIC* is of type *ASCII*. If no *ENCODING* is specified, the content is interpreted as string of 8-Bit characters. The *ENCODING* keyword is supported from ASAP version 1.7. Possible values: *UTF8*; *UTF16*; *UTF32*.

Conversion

To set the *Conversion*, the <COMPU-METHOD-REF> tag is referenced. The *CompuMethod* object is fetched, along with the corresponding A2L *uniqueName*.

The corresponding *CompuMethod* with the name equal to the *uniqueName* will be searched in the A2LModule. If the *CompuMethod* exists with this name, then it will be fetched from the A2LModule and set as the *Conversion*. Or else, the <COMPU-METHOD-REF> is missing/invalid/proxy, and if the type of the *McDataInstance* is *VALUE* or *VAL_BLK*, then a default *CompuMethod* of type AUTOSAR *IDENTICAL* will be created and assigned to the *CHARACTERISTIC* block. The respective warning messages will be logged in the log file.

If the type is *BOOLEAN*, and if the <COMPU-METHOD-REF> is missing/invalid /proxy then, for that *CHARACTERISTIC* block, *NO_COMPU_METHOD* keyword is generated for the *Conversion*. Or else, then if the type is other than *VALUE*, *VAL_BLK* or *BOOLEAN*, and if the <COMPU-METHOD-REF> is missing/proxy /invalid, then that *CHARACTERISTIC* block will not be created.

LowerLimit

UpperLimit

This is one of the mandatory parameters for the *CHARACTERISTIC* block creation. To create it, there should be a <DATA-CONSTR-REF> in the <MC-DATA-INSTANCE>. If <DATA-CONSTR-REF> is missing/invalid, then <BASE-TYPE-REF> should exist to create the default limits.

In the case of a *BOOLEAN* type of *CHARACTERISTIC* block creation, if the *DataConstr* object is missing/invalid/proxy, *0* will be set as the default *LowerLimit* and *1* as default *UpperLimit*.

For a *CHARACTERISTIC* block type other than *BOOLEAN*, if the *DataConstr* object is missing, then by using the *baseTypeRef* object, the default limits will be set.

If the <BASE-TYPE-REF> of the *McDataInstance* is missing or invalid, then the *baseType* will be fetched from the provided *RecordLayout* V-Axis. The *baseType* from the *RecordLayout* should be valid. Also, the *CompuMethod* referenced by the corresponding *CHARACTERISTIC* is considered for generation of these parameters. A warning message is logged if the referenced *CompuMethods* type is other than *TEXTTABLE*, *IDENTICAL* or *TAB_VERB*.

If the *baseType* is present, then these parameters are generated from the *CompuMethod* referenced by the corresponding *CHARACTERISTIC*. A warning message is logged if the referenced *CompuMethod's* type is other than *TEXTTABLE*, *IDENTICAL* or *TAB_VERB*.

For the *FLOAT* type of *baseType*, the <DATA-CONSTR-REF> is mandatory.

To create the *LowerLimit*, the <LOWER-LIMIT> tag will be referenced from the <DATA-CONSTR>. To create the *UpperLimit*, the <UPPER-LIMIT> tag will be referenced from the <DATA-CONSTR>. (Please refer to Section 5.7.22 for more details.)

FORMAT

This will be created only if the <DISPLAY-FORMAT> value exists.

If the <DISPLAY-FORMAT> is supplied with format *%[ddd.dd]f*, then the format's *f*

value will be removed and only the `%[ddd.dd]` will be set.

If the `<DISPLAY-FORMAT>` is supplied with format `%[ddd.dd]`, `%[.dd]`, `%[ddd]` i.e. only digits, then the format is set as given in the input. (Please refer to Section 5.7.21 for more details.)

PHYS_UNIT

If the `<MC-DATA-INSTANCE>` has a valid `<UNIT-REF>` tag, then *PHYS_UNIT* will be generated for the *CHARACTERISTIC* block in the A2L file.

The *PHYS_UNIT* will be generated in the A2L file as *PHYS_UNIT <UNIT> displayName*.

e.g. *PHYS_UNIT m/s*

If the `<UNIT>` referenced by the `<MC-DATA-INSTANCE>` does not have a `<DISPLAY-NAME>` or an empty `<DISPLAY-NAME>`, then *PHYS_UNIT* will not be generated in the A2L file for the *CHARACTERISTIC* and a warning message will be logged with message ID: *A2L_MCS_013* in the *tool.log* file.

e.g.

ICE Message ID: A2L_MCS_013, WARN: `<DISPLAY-NAME>` in `<UNIT>` 'RB_PT_PCT_ComRx_HBatLim_Units_Ampr' is Missing/Empty, Hence Physical Unit will not be generated.

If the `<UNIT>` referenced by the `<COMPU-METHOD>` of the `<MC-DATA-INSTANCE>` is different from the `<UNIT>` referenced by the `<MC-DATA-INSTANCE>`, then *PHYS_UNIT* will be generated and a warning message will be logged with message ID: *A2L_MCS_WARN_038* in the *tool.log* file.

e.g.

ICE Message ID: A2L_MCS_WARN_038, WARN: `<UNIT-REF>` RB_PT_PCT_ComRx_HBatLim_Units_Ampr and compuMethod `<UNIT-REF>` RB_PT_PCT_ComRx_HBatLim_Units_NoUnit which are referred in `<MC-DATA-INSTANCE>` Com_iLimBatTempMin_C are not compatible.

READ_ONLY

If the `<SW-CALIBRATION-ACCESS>` of the `<MC-DATA-INSTANCE>` is set to *READ-ONLY*, then this attribute will be generated for the *CHARACTERISTIC* blocks.

MAX_REFRESH

This will be created only if the `<SW-REFRESH-TIMING>` value exists. (Please refer to Section 5.7.20 for more details.)

BYTE_ORDER

If the `<BYTE-ORDER>` value of the `<SW-BASE-TYPE>` is specified, the `<MC-DATA-INSTANCE>` referencing this `<SW-BASE-TYPE>` will include a *BYTE_ORDER* keyword with the corresponding value in the A2L file.

Only *MOST-SIGNIFICANT-BYTE-FIRST* and *MOST-SIGNIFICANT-BYTE-LAST* values are supported for the *BYTE-ORDER* in A2L file generation. Consequently, *MSB-FIRST* and *MSB-LAST* will be produced in the A2L file.

The *OPAQUE* value for `<BYTE-ORDER>` is not supported for A2L generation.

ANNOTATION

This will be created only if the `<ANNOTATIONS>` exists. RTA-A2L supports the

creation of the *ANNOTATION_LABEL* and the *ANNOTATION_TEXT*. (Please refer to Section 5.7.19 for more details.)

AXIS_DESCR

This block is generated for the *CURVE* and *MAP* type of *CHARACTERISTICS*.

For a *CURVE CHARACTERISTIC*, a single *AXIS_DESCR* block (for the X-Axis) will be generated, whereas for the *MAP*, two *Axis_DESC* blocks (X and Y-Axes) will be generated. (Please refer to Section 5.7.13 for more details.)

NUMBER

This attribute is generated for the *VAL_BLK* and *ASCII* types of *CHARACTERISTICS*. It is also supported for single/multi-dimensional *ARRAYs* (*ARRAYs* of *VALUE* or *BOOLEAN* type) which are leaf *subElements* of *STRUCTURE*.

For a *VAL_BLK CHARACTERISTIC*, this attribute is generated with the value specified in <SW-VALUE-BLOCK-SIZE> of <MC-DATA-INSTANCE>. If this tag <SW-VALUE-BLOCK-SIZE> is missing in the <MC-DATA-INSTANCE>, then A2L throws an error and will not consider the <MC-DATA-INSTANCE> to have generated the corresponding *CHARACTERISTIC* block.

Single/multi-dimensional *ARRAYs* whose *subElements* are of type *VALUE* or type *BOOLEAN* are generated with *NUMBER*.

Structure leaf elements, which are single/multi-dimensional *ARRAYs* and are of type *VALUE* or *BOOLEAN*, are generated with *NUMBER*.

The dimensions are determined from *ARRAY_SIZE*. If *ARRAY_SIZE* is missing, the number of *subElements* are calculated to determine the *ARRAY_SIZE*. In the case of multi-dimensional *ARRAYs*, the product of the X, Y and Z dimensions is set against *NUMBER*.

This attribute is generated if the *MATRIX_DIM* switch is not enabled in the options file.

MATRIX_DIM

This is generated for the *VAL_BLK* and *ASCII* types of *CHARACTERISTICS*. It is also supported for single/multi-dimensional *ARRAYs* (*ARRAYs* of *VALUE* or *BOOLEAN* type) which are leaf *subElements* of *STRUCTURE*.

It is generated if the *MATRIX_DIM* switch is enabled in the options file, or if the *ASAP_VERSION* is >= 1.7.

The *MATRIX_DIM* keyword is not supported for the *ASCII* type.

For the *VAL_BLK CHARACTERISTIC*, only one-dimensional *ARRAYs* are supported and only the X dimension is generated. The Y and Z dimensions are generated as 1 if *ASAP_VERSION* < 1.7, or else only the X dimension is generated. The value of the tag <SW-VALUE-BLOCK-SIZE> is set as the X dimension of *MATRIX_DIM*.

Single/multi-dimensional *ARRAYs* whose *subElements* are of type *VALUE* or *BOOLEAN* are generated with *MATRIX_DIM*.

Structure leaf elements which are single/multi-dimensional *ARRAYs* and are of type *VALUE* or *BOOLEAN* are generated with *MATRIX_DIM*.

Single-dimensional *ARRAYs* are generated with the X dimension.

The dimensions are determined from the *ARRAY_SIZE*.

If the *ARRAY_SIZE* is missing, the number of *subElements* are calculated to determine the *ARRAY_SIZE*.

In the case of multi-dimensional *ARRAYs*, the respective *ARRAY_SIZE* of *subElements* would be set based on the availability of the X, Y and Z dimensions.

e.g. Single dimensional with *ARRAY_SIZE* as 18

<pre> If ASAP_VERSION < 1.7 If NUMBER 18 then MATRIX_DIM 18 1 1 If ASAP_VERSION >= 1.7 If NUMBER 18 then MATRIX_DIM 18 </pre>

BIT_MASK

RTA-A2L supports the *BIT_MASK* creation for *CHARACTERISTIC* blocks. These blocks are generated with the values specified in the tags <BIT-POSITION> and <NUMBER-OF-BITS>, which are present with the *Parent* tag <SW-BIT-REPRESENTATION>.

The *BIT_MASK* keyword can be used to mask out single bits of the value to be processed. The least significant bit in *BIT_MASK* determines how far the masked value is shifted to the right.

Input block:

```
<MC-DATA-INSTANCE>
  <SHORT-NAME>ValueExample</SHORT-NAME>
  <CATEGORY>VALUE</CATEGORY>
  <RESULTING-PROPERTIES>
  <SW-DATA-DEF-PROPS-VARIANTS>
  <SW-DATA-DEF-PROPS-CONDITIONAL> -----
  <SW-BIT-REPRESENTATION>
  <BIT-POSITION>12</BIT-POSITION>
  <NUMBER-OF-BITS>4</NUMBER-OF-BITS>
  </SW-BIT-REPRESENTATION> -----
  <BASE-TYPE-REF DEST="SW-BASE-TYPE">/RB/DataTypes/Swbasetypes/uint16</BASE-TYPE-REF> -----
  <SW-HOST-VARIABLE>
  <MC-DATA-INSTANCE-VAR-REF DEST="MC-DATA-
  INSTANCE">/RB/RTE/BswImplementation/FlxrXcvr_Var2_u32</MC-DATA-
  INSTANCE-VAR-REF>
  </SW-HOST-VARIABLE>
  -----
  <SW-RECORD-LAYOUT-REF DEST="SW-RECORD-LAYOUT">/RB/DataTypes/SwRecordLayouts/Value_uint32<
  /SW-RECORD-LAYOUT-REF>
  </SW-DATA-DEF-PROPS-CONDITIONAL>
  </SW-DATA-DEF-PROPS-VARIANTS>
  </RESULTING-PROPERTIES>
  <SYMBOL>ValueExample_SymbolName</SYMBOL>
</MC-DATA-INSTANCE>
```

Conditions are placed on the values of the tags `<BIT-POSITION>` and `<NUMBER-OF-BITS>`:

- The value of `<BIT-POSITION>` should always be $\leq (\text{<BASE-TYPE-SIZE>} - 1)$.
- The value of $(\text{<BIT-POSITION>} + \text{<NUMBER-OF-BITS>})$ should always be $\leq (\text{<BASE-TYPE-SIZE>})$.

An error message is thrown if these conditions are not met, and the corresponding block is not considered for A2L generation.

The above validation is performed against the *RecordLayout baseType* size.

If the *RecordLayout* is not present in the *Calprm*, then the validation is performed against the *Calprm baseType* size.

If the `<SW-HOST-VARIABLE>` tag is present for *BIT_MASK* and *RecordLayout* is referenced in *Calprm*, then the validation is performed against the *RecordLayout baseType* size.

If the `<SW-HOST-VARIABLE>` tag is present for *BIT_MASK* and *RecordLayout* is missing in *Calprm*, then the validation of *baseType* size is done against the *baseType* referenced in the host variable.

ADX ENTRY for BIT_MASK

An *ADX ENTRY* is created for every block which contains the tags `<BIT-POSITION>` and `<NUMBER-OF-BITS>`. e.g.:

```
<MEMORY-ELEMENT>
  <LABEL-NAME>ValueExample_SymbolName</LABEL-NAME>
  <MAP-LABEL-NAME>ValueExample</MAP-LABEL-NAME>
  <BIT-POSITION>12</BIT-POSITION>
  <NUMBER-OF-BITS>4</NUMBER-OF-BITS>
</MEMORY-ELEMENT>
```

If the `<SW-HOST-VARIABLE>` tag is present for *BIT_MASK*, then the *ADX ENTRY* generation will change.

If the host variable is provided, then the *Symbol Name* of the host variable will be added for `<LABEL-NAME>hostVariable_symbol</LABEL-NAME>`.

If the host variable is not present, then the *Symbol Name* of the bit variable is used for the generation of the <LABEL-NAME>. e.g.

```

<MEMORY-ELEMENT>
  <LABEL-NAME>ValueExample_SymbolName_of_HostVariable</LABEL-NAME>
  <MAP-LABEL-NAME>ValueExample</MAP-LABEL-NAME>
  <BIT-POSITION>12</BIT-POSITION>
  <NUMBER-OF-BITS>4</NUMBER-OF-BITS>
</MEMORY-ELEMENT>

```

IF_DATA

RTA-A2L supports *ETK* and *XCP IfData* blocks creation, using the *xpand* template files.

Currently, the *IfDataname* and the *Data* record contents are supported.

If the *IfDataname* is not set in the templates, then the RTA-A2L Generator sets the default name as below.

For *CalObjects*, the name is set as "defaultCharIfdata_[index]", where *index=0,1,2* etc.

Please refer to the *RTA-A2L Error Description* document for the list of log entries supported for *CHARACTERISTICS*.

STEP_SIZE

If the <MC-DATA-INSTANCE> of type *MC-PARAMETER-INSTANCE* consists of the <STEP-SIZE> tag, then this attribute will be generated for the *CHARACTERISTIC* blocks.

e.g. STEP_SIZE 0.025

5.7.10 AXIS_PTS Details

The <MC-DATA-INSTANCE> blocks with the <CATEGORY> set as *COM_AXIS* will be considered as the input for the *AXIS_PTS* blocks creation. If the <MC-DATA-INSTANCE> is of type *NOT-ACCESSIBLE*, then no *AXIS_PTS* blocks will be created in the A2L file and the blocks which refer to this *COM_AXIS* will also not be created.

The *SwCalprm* Axis with the *SW-AXIS-INDEX* as 1 will be considered for the *AXIS_PTS* block creation. If the *SW-AXIS-INDEX* is missing/invalid then, by default, the first Axis in the *SW-CALPRM-AXIS-SET* will be considered as the input for the *AXIS_PTS* block creation.

The following *AXIS_PTS* details are supported by the RTA-A2L:

```

/begin AXIS_PTS
  Name
  LongIdentifier
  Address
  InputQuantity
  Deposit
  Max-Diff
  Conversion
  MaxAxisPoints
  LowerLimit
  UpperLimit
  [-> ANNOTATION]*
  [-> MONOTONY]
  [-> FORMAT]
  [-> PHYS_UNIT]
  [-> READ-ONLY]
  [-> DEPOSIT]
  [-> IF_DATA]
  [-> STEP_SIZE]
  [-> BYTE_ORDER]
/end AXIS_PTS

```

Name

The *Name* is the customized *ShortName* of the *McDataInstance*. (Please refer to Section 5.6.3 for more details.)

LongIdentifier

The *LongIdentifier* will be created using the <LONG-NAME> tag of the *McDataInstance*.

Address

Currently, the address information is *0x0000*.

InputQuantity

The <SW-VARIABLE-REF>s will (if provided) be the input. There might be many <SW-VARIABLE-REF> occurrences: since the ASAP standard supports only one *InputQuantity*, the first <SW-VARIABLE-REF> will be considered.

From the <SW-VARIABLE-REF>, the <MC-DATA-INSTANCE_REF> will be considered, and the corresponding A2L *MEASUREMENT* block will be fetched from the A2LModule. This *MEASUREMENT* object will be set as the *InputQuantity*.

If no <SW-VARIABLE-REF> exists, or if the referenced variable is of type *NOT-ACCESSIBLE*, or <SW-CALIBRATION-ACCESS> is unavailable, or a proxy <SW-VARIABLE-REF> exists, or no *MEASUREMENT* block occurs in the A2LModule for the corresponding <MC-DATA-INSTANCE>, then the *NO_INPUT_QUANTITY* keyword will be set.

Deposit

To set the *Deposit*, RTA-A2L would be referencing both the <BASE-TYPE-REF> as well as the <SW-RECORD-LAYOUT-REF>.

The <BASE-TYPE-REF> is the mandatory attribute, to create the *Deposit* for the *CHARACTERISTIC* block. If the <BASE-TYPE-REF> is missing/proxy, then that *CHARACTERISTIC* block will not be created in the A2L file.

Along with the <BASE-TYPE-REF>, the <SW-RECORD-LAYOUT-REF> will also be referenced to create the *Deposit*. If the <SW-RECORD-LAYOUT-REF> exists, then a new *RecordLayout* object will be created by using the referenced <RECORD-LAYOUT>. The name of the created *RecordLayout* will be set as "<ShortName> of the referenced <RECORD-LAYOUT>_<SHORT-NAME> of the referenced <BASE-TYPE>" and it will be added to the A2LModule.

This newly-created A2L *RecordLayout* object will be fetched from the A2LModule and assigned as the *Deposit* to the *CHARACTERISTIC* block. (Please refer to Section 5.7.8 for more details.)

If the <SW-RECORD-LAYOUT-REF> is missing/proxy/invalid in the input file, then the *AXIS_PTS* block will not be created.

MaxDiff

The *MaxDiff* will be calculated as the difference between the *UpperLimit* and the *LowerLimit* of the *CHARACTERISTIC* block. Before setting the *MaxDiff* value, the *UpperLimit* and *LowerLimit* values will be set to the *CHARACTERISTIC* block. $MaxDiff = (UpperLimit - LowerLimit)$.

When the *UpperLimit* and *LowerLimit* values are derived from the *baseType* rather than the *dataConstraint*, and the *baseType* is of type *float64*, the difference between the upper and lower limits will result in a very large value greater than the limit of float 64. In such instances, the *maxDiff* value will be set to maximum limit of *float64*, and the value will be formatted according to the display format. (Please refer to *Generation of A2LLimits based on the input display format* in Section 5.7.22 for more details.)

Example: * If the display format is %8.4 then the *MaxDiff* will be generated as 1.7976E308. * If the display format is %5.6 then the *MaxDiff* will be generated as 1.797693E308.

Conversion

To set this attribute, RTA-A2L references both the <COMPU-METHOD-REF> and the <SW-VARIABLE-REF> tags.

If both the <COMPU-METHOD-REF> of the Axis and the <VARIABLE-REF> exist in the <SW-CALPRM-AXIS> (if *AXIS_DESCR* is set with the *InputQuantity*), then the definition of the *CompuMethod* of the <SW-CALPRM-AXIS>, as well as the <COMPU-METHOD> provided in the *Sw-Variable*, should be the same (except for the *ShortName* and *LongIdentifier/Desc*).

In this case, the <COMPU-METHOD-REF> from the Axis will be considered and the corresponding *A2LCompu* will be set as the *Conversion*. If the definitions are not the same, then a warning message will be provided and the *CompuMethod* of the Axis will be set. Or else, if only the <COMPU-METHOD-REF> exists in the Axis, then the corresponding *A2LCompu* object will be set as the *Conversion*.

If only the <SW-VARIABLE-REF> exists in the Axis, then the <COMPU-METHOD-REF> of the corresponding <MC-DATA-INSTANCE> will be fetched, and the corresponding *A2LCompu* object will be set as the *Conversion*.

If no <COMPU-METHOD-REF> and <SW-VARIABLE-REF> exist, then the *NO_COMPU_METHOD* keyword will be set.

If the proxy <COMPU-METHOD-REF> exists, then the *Conversion* will be set from the *VariableRef Compu* object (if it exists) or else the *NO_COMPU_METHOD* keyword will be set.

For checking the definition of the *CompuMethods*, only the attributes which are currently supported by the RTA-A2L for the *CompuMethod* will be considered i.e. *ConversionType*, *Format*, *Unit*, *COEEFS*, *COMPU_TAB*, *_REF** and *REF_UNIT*. If any new attributes are supported, then the check should be done for all feature-supporting attributes.

MaxAxisPoints

The <SW-MAX-AXIS-POINTS> tag is the input for the *MaxAxisPoints* creation.

If <SW-MAX-AXIS-POINTS> does not exist or is empty, then no *AXIS_DESCR* blocks will be created.

If the variation point occurs with the value *>= PRE_COMPILE_TIME*, along with the system condition and expressions in <SW-MAX-AXIS-POINTS>, then no *Axis_DESCR* blocks will be created.

LowerLimit**UpperLimit**

This is one of the mandatory parameters for the *AXIS_PTS* block creation.

To create this attribute the `<DATA-CONSTR-REF>` should exist in the `<MC--DATA-INSTANCE>`. If `<DATA-CONSTR-REF>` is missing/invalid, then `<BASE--TYPE-REF>` should exist to create the default limits.

The `<BASE-TYPE-REF>` and the *CompuMethod* referenced by the corresponding *AXIS_PTS* is considered for the generation of these parameters. A warning message is logged if the referenced *CompuMethod* type is other than *TEXTTABLE*, *IDENTICAL* and *TAB_VERB*.

If the *baseType* at the Axis Level does not exist or is invalid, then the *baseType* will be fetched from the provided *VariableRef* object. If no *VariableRef* exists, then the *baseType* will be fetched from the referenced *RecordLayout* X-Axis *baseType* for the X-Axis *AXIS_DESCR* block.

The *baseType* from the *RecordLayout* should also be valid. Also, the *CompuMethod* referenced by the corresponding *AXIS_PTS* is considered for generation of these parameters.

A warning message is logged if the referenced *CompuMethod*'s type is other than *TEXTTABLE*, *IDENTICAL* and *TAB_VERB*.

For the *FLOAT* type of *baseType* object, the `<DATA-CONSTR-REF>` is mandatory.

To create the *LowerLimit*, the `<LOWER-LIMIT>` tag will be referenced from the `<DATA-CONSTR>`. To create the *UpperLimit*, the `<UPPER-LIMIT>` tag will be referenced from the `<DATA-CONSTR>`.

Monotony

The `<DATA-CONSTR>` block referenced from the `<MC-DATA-INSTANCE>` / `<SW--CALPRM-AXIS>` will be used as input for the *Monotony* creation.

This attribute specifies the monotony constraint for an adjustable object (*CURVE*, *MAP*, or *CUBOID*) with respect to a specific Axis. It is only applicable to *CURVES* and *MAPs*.

Valid ARXML input values:

	A2L file Keywords
<i>DECREASING</i>	<i>MON_DECREASE</i>
<i>INCREASING</i>	<i>MON_INCREASING</i>
<i>MONOTONOUS</i>	<i>MONOTONOUS</i>
<i>NO-MONOTONY</i>	<i>NOT_MON</i>
<i>STRICT-MONOTONOUS</i>	<i>STRICT_MON</i>
<i>STRICTLY-INCREASING</i> (Default)	<i>STRICT_INCREASE</i>
<i>STRICTLY-DECREASING</i>	<i>STRICT_DECREASE</i>

Format

The `<DISPLAY-FORMAT>` tag of the `<SW-CALPRM-AXIS>` will be referenced to

set this value.

PHYS_UNIT

If the `<SW-AXIS-INDIVIDUAL>` tag present inside `<MC-DATA-INSTANCE>` has a valid `<UNIT-REF>` tag, then *PHYS_UNIT* will be generated for the *AXIS_PTS* block in the A2L file.

The *PHYS_UNIT* will be generated in the A2L file as *PHYS_UNIT <UNIT> displayName*.

e.g. *PHYS_UNIT m/s*

If the `<UNIT>` referenced by the `<SW-AXIS-INDIVIDUAL>` tag present inside `<MC-DATA-INSTANCE>` does not have a `<DISPLAY-NAME>` or an empty `<DISPLAY-NAME>`, then *PHYS_UNIT* will not be generated in the A2L file for the *AXIS_PTS* and a warning message will be logged with message ID: *A2L_MCS_013* in the *tool.log* file.

e.g.

ICE Message ID: A2L_MCS_013, WARN: <DISPLAY-NAME> in <UNIT> 'RB_PT_PCT_ComRx_HBatLim_Units_Ampr' is Missing/Empty, Hence Physical Unit will not be generated.

If the `<UNIT>` referenced in the `<COMPU-METHOD>` of the `<SW-AXIS-INDIVIDUAL>` tag inside `<MC-DATA-INSTANCE>` is different from the `<UNIT>` referenced by the `<SW-AXIS-INDIVIDUAL>` tag inside `<MC-DATA-INSTANCE>`, then *PHYS_UNIT* will be generated and a warning message will be logged with message ID: *A2L_MCS_WARN_038* in the *tool.log* file.

e.g.

ICE Message ID: A2L_MCS_WARN_038, WARN: <UNIT-REF> RB_PT_PCT_ComRx_HBatLim_Units_Ampr and compuMethod <UNIT-REF> RB_PT_PCT_ComRx_HBatLim_Units_NoUnit which are referred in <MC-DATA-INSTANCE> Com_iLimBatTempMin_C are not compatible.

ANNOTATION

This will be created only if the `<ANNOTATIONS>` exists. RTA-A2L supports the creation of the *ANNOTATION_LABEL* and the *ANNOTATION_TEXT*.

READ_ONLY

If `<SW-CALIBRATION-ACCESS>` is set to *READ-ONLY* at the `<SW-CALPRM-AXIS>`, then this attribute will be created for the *AXIS_PTS* blocks.

If the `<SW-CALIBRATION-ACCESS>` is NOT set at the `<SW-CALPRM-AXIS>` level, but the *Parent* `<MC-DATA-INSTANCE>` contains the *READ-ONLY* attribute for the `<SW-CALIBRATION-ACCESS>`, then this attribute will be created for the *AXIS_PTS* blocks.

If the `<SW-CALIBRATION-ACCESS>` is set to *NOT-ACCESSIBLE*, or `<SW-CALIBRATION-ACCESS>` is unavailable at the `<SW-CALPRM-AXIS>`, then this tag will be ignored.

DEPOSIT

This value will be set based on the `<SW-RECORD-LAYOUT-REF>` of the `<MC-DATA-INSTANCE>`.

If the referenced `<SW-RECORD-LAYOUT>` contains the `<SW-RECORD-LAY-`

OUT-V-PROP> as *LEFTDIFF* or *RIGHTDIFF*, then the *DEPOSIT* will be set as *DIFFERENCE*. Or else, the *DEPOSIT* will be set as *ABSOLUTE*.

The A2L *RecordLayout* will be created and set to the *CHARACTERISTIC* block.

IF_DATA

RTA-A2L supports the *ETK* and *XCP IfData* blocks creation, using the *xpand* template files.

Currently, the *IfDataname* and the Data record contents are supported.

If the *IfDataname* is not set in the templates, then RTA-A2L sets the default name as below.

For *CalObjects*, the name is set as "defaultCharIfdata_[index]", where *index* = 0,1,2 etc.

STEP_SIZE

If the <MC-DATA-INSTANCE> of type *MC-PARAMETER-INSTANCE* consists of a <STEP-SIZE> tag, then this attribute will be created for the *AXIS_PTS* blocks.

e.g. STEP_SIZE 0.025

BYTE_ORDER

If the <BYTE-ORDER> value of the <SW-BASE-TYPE> is specified, the <MC-DATA-INSTANCE> referencing this <SW-BASE-TYPE> will include a *BYTE_ORDER* keyword with the corresponding value in the A2L file.

Only *MOST-SIGNIFICANT-BYTE-FIRST* and *MOST-SIGNIFICANT-BYTE-LAST* values are supported for the *BYTE-ORDER* in A2L file generation. Consequently, *MSB_FIRST* and *MSB_LAST* will be produced in the A2L file.

The *OPAQUE* value for <BYTE-ORDER> is not supported for A2L generation.

5.7.11 MEASUREMENT Details

Currently, the RTA-A2L supports the *VALUE*, *BOOLEAN*, *ARRAY* and *STRUCTURE* types of variable *McDataInstances* as input for the creation of the *MEASUREMENT* blocks.

If the input *McDataInstance* is of type *NOT-ACCESSIBLE*, then no *MEASUREMENT* blocks will be created in the A2L file.

The following *MEASUREMENT* details are supported by the RTA-A2L Generator:

```
/begin MEASUREMENT
  Name LongIdentifier
  DataType
  Conversion
  Resolution
  Accuracy
  LowerLimit
  UpperLimit
  [-> FORMAT]
  [-> PHYS_UNIT]
  [-> ARRAY_SIZE]
  [-> MAX_REFRESH]
  [-> BYTE_ORDER]
  [-> READ_WRITE]
  {-> ANNOTATION} *
  {-> VIRTUAL}
  {[-> BIT_MASK]}
  {->IF_DATA}
  {->ARRAY_SIZE}
  {->MATRIX_DIM}
```

```
{ ->LAYOUT}
/end MEASUREMENT
```

Name

This is the customized *ShortName* of the *McDataInstance*. (Please refer to Section 5.6.3 for more details.)

LongIdentifier

This will be created using the <LONG-NAME> tag of the *McDataInstance*.

DataType

This will only be set if the <BASE-TYPE-REF> exists and is proper. Or else, no *DataType* will be set and no *MEASUREMENT* block be created.

The following *DataTypes* are supported: *UBYTE*, *SBYTE*, *UWORD*, *SWORD*, *ULONG*, *SLONG*, *FLOAT32_IEEE*, *FLOAT64_IEEE*. (Please refer to Section 5.7.16 for more details.)

Conversion

To set the *Conversion*, the <COMPU-METHOD-REF> will reference the *A2LCompu* with this name. That *CompuMethod* will then be fetched from the *A2LModule* and set as the *Conversion*.

Or else, if the <COMPU-METHOD-REF> is missing/invalid /proxy, and if the category of the *McDataInstance* is *VALUE*, then a default *CompuMethod* of type *AUTOSAR IDENTICAL* will be created and assigned to the *CHARACTERISTIC* block. The respective warning messages will be logged in the log file.

If the type is *BOOLEAN*, and if the <COMPU-METHOD-REF> is missing/proxy /invalid, then for that *MEASUREMENT* block the *NO_COMPU_METHOD* keyword is generated for the *Conversion*.

Or else, if the type is other than *VALUE* and *BOOLEAN*, and if the <COMPU-METHOD-REF> is missing/proxy/invalid, then that *MEASUREMENT* block will not be created.

Resolution

RTA-A2L always sets the *Resolution* of the *MEASUREMENT* block to 1.

Accuracy

RTA-A2L always sets the *Accuracy* of the *MEASUREMENT* block to 100.

LowerLimit

UpperLimit

This is one of the mandatory parameters for the *MEASUREMENT* block creation.

To create this attribute, there should be a <DATA-CONSTR-REF> in the <MC-DATA-INSTANCE>.

If <DATA-CONSTR-REF> is missing/invalid, then <BASE-TYPE-REF> should exist to create these attributes.

To create the *LowerLimit*, the <LOWER-LIMIT> tag will be referenced from the <DATA-CONSTR>.

To create the *UpperLimit*, the `<UPPER-LIMIT>` tag will be referenced from the `<DATA-CONSTR>`.

In the case of a *BOOLEAN* type of variable *McDataInstance*, if the *DataConstr* is missing/invalid, then *0* will be set as the default *LowerLimit* and *1* will be set as the default *UpperLimit*.

For the variable *McDataInstance*, apart from *BOOLEAN* type, if the *DataConstrRef* does not exist or the referenced *DataConstr* Object is invalid, then using the *baseType* object, the default limits will be set.

If the *baseType* is present, then these parameters are generated from the *CompuMethod* referenced by the corresponding *Measurement*. A warning message is logged if the referenced *CompuMethod* category is other than *TEXTTABLE*, *IDENTICAL* and *TAB_VERB*.

FORMAT

This will only be created if the `<DISPLAY-FORMAT>` value exists.

PHYS_UNIT

If the `<MC-DATA-INSTANCE>` has a valid `<UNIT-REF>` tag then *PHYS_UNIT* will be generated for the *MEASUREMENT* block in the A2L file.

The *PHYS_UNIT* will be generated in the A2L file as *PHYS_UNIT <UNIT> displayName*.

e.g. *PHYS_UNIT m/s*

If the `<UNIT>` referenced by the `<MC-DATA-INSTANCE>` does not have `<DISPLAY-NAME>` or an empty `<DISPLAY-NAME>`, then *PHYS_UNIT* will not be generated in the A2L file for the *MEASUREMENT* and a warning message will be logged with message ID: *A2L_MCS_013* in the *tool.log* file.

e.g.

ICE Message ID: A2L_MCS_013, WARN: <DISPLAY-NAME> in <UNIT> 'RB_PT_PCT_ComRx_HBatLim_Units_Ampr' is Missing/Empty, Hence Physical Unit will not be generated.

If the `<UNIT>` referenced in the `<COMPU-METHOD>` of the `<MC-DATA-INSTANCE>` is different from the `<UNIT>` referenced by the `<MC-DATA-INSTANCE>`, then *PHYS_UNIT* will be generated, and a warning message will be logged with message ID: *A2L_MCS_WARN_038* in the *tool.log* file.

e.g.

ICE Message ID: A2L_MCS_WARN_038, WARN: <UNIT-REF> RB_PT_PCT_ComRx_HBatLim_Units_Ampr and compuMethod <UNIT-REF> RB_PT_PCT_ComRx_HBatLim_Units_NoUnit which are referred in <MC-DATA-INSTANCE> Com_ilimBatTempMin_C are not compatible.

ARRAY_SIZE

The *MEASUREMENT* block will be generated for the *ARRAY* using the *ARRAY_SIZE* attribute, only in the case of a Simple *ARRAY MEASUREMENTs* block (e.g. element[0], element[1]) (rather than generating the *MEASUREMENT* block for each *ARRAY* element).

This attribute will be created for a single/multi-dimensional *ARRAY <MC-DATA-INSTANCE>*, or if the single/multi-dimensional (*ARRAY* of *VALUE* type or *BOOLEAN*

type) *ARRAY* <MC-DATA-INSTANCE> occurs at the leaf node. The *STRUCTURE* leaf elements, which are single/multidimensional *ARRAYs* and are of type *VALUE* or *BOOLEAN*, are generated with *ARRAY_SIZE*.

The dimensions are determined from *ARRAY_SIZE*.

If *ARRAY_SIZE* is missing, the number of *subElements* is calculated to determine the *ARRAY_SIZE*.

In the case of multi-dimensional *ARRAYs*, the product of X, Y and Z dimensions is set against *ARRAY_SIZE*. *ARRAY_SIZE* is generated if the *MATRIX_DIM* switch is not enabled in the options file.

MATRIX_DIM

This attribute is generated if the *MATRIX_DIM* switch is enabled in the options file and if the *ASAP_VERSION* is ≥ 1.7 .

MATRIX_DIM is also supported for single/multi-dimensional *ARRAYs* (*ARRAYs* of the *VALUE/BOOLEAN* type) including those which are leaf *subElements* of *STRUCTURE*.

Single/multi-dimensional *ARRAYs* whose *subElements* are of type *VALUE* or *BOOLEAN* are generated with *MATRIX_DIM*.

Structure leaf elements which are single/multi-dimensional *ARRAYs*, and are of type *VALUE*, are generated with *MATRIX_DIM*.

Single dimensional *ARRAYs* would be generated with the X dimension.

If the Y and Z dimensions are missing and *ASAP_VERSION* < 1.7 , then the Y and Z dimensions are generated as 1. The value of the tag *ARRAY_SIZE* is set as the X dimension of *MATRIX_DIM*.

In the case of multi-dimensional *ARRAYs*, the respective *ARRAY_SIZE* of *subElements* would be set based on the availability of the X, Y and Z dimensions.

The dimensions are determined from *ARRAY_SIZE*. If *ARRAY_SIZE* is missing, the number of *subElements* is calculated to determine the *ARRAY_SIZE*.

e.g. single dimensional with *ARRAY_SIZE* as 18

If <i>ASAP_VERSION</i> < 1.7	If <i>ARRAY_SIZE</i> 18	then <i>MATRIX_DIM</i> 18 1 1
If <i>ASAP_VERSION</i> ≥ 1	If <i>ARRAY_SIZE</i> 18	then <i>MATRIX_DIM</i> 18

LAYOUT

The *LAYOUT* keyword will be generated only if the *MEASUREMENT* <MC-DATA-INSTANCE> has a valid <SW-RECORD-LAYOUT-REF> and the referenced <SW-RECORD-LAYOUT> contains *COLUMN_DIR* or *ROW_DIR* from the category of <SW-RECORD-LAYOUT-GROUP>.

e.g.

The *Sample McDataInstance* is of type *Multidimensional Measurement* and is referencing the <SW-RECORD-LAYOUT> *SampleRecLayout*, which contains *COLUMN_DIR* and has a <SW-RECORD-LAYOUT-GROUP>.

The *LAYOUT* keyword will be generated in the A2L file as shown below:

```

LAYOUT COLUMN_DIR
<MC-DATA-INSTANCE>
  <SHORT-NAME>Sample</SHORT-NAME>
  <CATEGORY>ARRAY</CATEGORY>
  <ARRAY-SIZE>2</ARRAY-SIZE>
  <SUB-ELEMENTS>
    <MC-DATA-INSTANCE>
      <CATEGORY>ARRAY</CATEGORY>
      <ARRAY-SIZE>3</ARRAY-SIZE>
      <SUB-ELEMENTS>
        <MC-DATA-INSTANCE>
          <SHORT-NAME>element_0_Value</SHORT-NAM>
          ...
          <SW-RECORD-LAYOUT-REF DEST="SW-RECORD-LAYOUT">/RB/DataTypes/SwRecordLayouts
/SampleRecLayout</SW-RECORD-LAYOUT-REF>
        </MC-DATA-INSTANCE>
      </SUB-ELEMENTS>
    </SUB-ELEMENTS>
  </MC-DATA-INSTANCE>

<SW-RECORD-LAYOUT>
  <SHORT-NAME> SampleRecLayout </SHORT-NAME>
  <SW-RECORD-LAYOUT-GROUP>
    <SW-RECORD-LAYOUT-GROUP>
      <CATEGORY>COLUMN_DIR</CATEGORY>
      <SW-RECORD-LAYOUT-GROUP-AXIS>0</SW-RECORD-LAYOUT-GROUP-AXIS>
      <SW-RECORD-LAYOUT-GROUP-FROM>1</SW-RECORD-LAYOUT-GROUP-FROM>
      <SW-RECORD-LAYOUT-GROUP-TO>-1</SW-RECORD-LAYOUT-GROUP-TO>...
    </SW-RECORD-LAYOUT-GROUP>
  </SW-RECORD-LAYOUT-GROUP>
</SW-RECORD-LAYOUT>

```

MAX_REFRESH

This will be created only if the <SW-REFRESH-TIMING> value exists.

BYTE_ORDER

If the <BYTE-ORDER> value of the <SW-BASE-TYPE> is specified, the <MC-DATA-INSTANCE> referencing this <SW-BASE-TYPE> will include a *BYTE_ORDER* keyword with the corresponding value in the A2L file.

Only *MOST-SIGNIFICANT-BYTE-FIRST* and *MOST-SIGNIFICANT-BYTE-LAST* values are supported for the *BYTE-ORDER* in A2L file generation. Consequently, *MSB-FIRST* and *MSB-LAST* will be produced in the A2L file.

The *OPAQUE* value for <BYTE-ORDER> is not supported for A2L generation.

READ_WRITE

If the <SW-CALIBRATION-ACCESS> of the <MC-DATA-INSTANCE> is set as *READ-WRITE*, then the *READ_WRITE* keyword will be generated for the corresponding MEASUREMENT block in the generated A2L file.

ANNOTATION

This will be created only if the <ANNOTATIONS> exists. RTA-A2L supports the creation of the *ANNOTATION_LABEL* and the *ANNOTATION_TEXT*.

VIRTUAL

This will be created only if <SW-IS-VIRTUAL> and <SW-DATA-DEPENDENCY> exist.

BIT_MASK

RTA-A2L supports the *BIT_MASK* creation for *MEASUREMENTs* blocks. These blocks are generated with the values specified in the tags <BIT-POSITION> and <NUMBER-OF-BITS> present with the *Parent* tag <SW-BIT-REPRESENTATION>.

Input block:

```
<MC-DATA-INSTANCE>
  <SHORT-NAME>ValueExample</SHORT-NAME>
  <CATEGORY>VALUE</CATEGORY>
  <RESULTING-PROPERTIES>
    <SW-DATA-DEF-PROPS-VARIANTS>
      <SW-DATA-DEF-PROPS-CONDITIONAL> -----
      <SW-BIT-REPRESENTATION>
        <BIT-POSITION>12</BIT-POSITION>
        <NUMBER-OF-BITS>4</NUMBER-OF-BITS>
      </SW-BIT-REPRESENTATION> -----
      <BASE-TYPE-REF DEST="SW-BASE-TYPE"/>/RB/DataTypes/Swbasetypes/uint16</BASE-TYPE-REF> -----
      <SW-HOST-VARIABLE>
        <MC-DATA-INSTANCE-VAR-REF DEST="MC-DATA-INSTANCE"/>/RB/RTE/BswImplementation/FlxRxcvr_Var2_u32</MC-DATA-INSTANCE-VAR-REF>
      </SW-HOST-VARIABLE> -----
    </SW-DATA-DEF-PROPS-CONDITIONAL>
  </SW-DATA-DEF-PROPS-VARIANTS>
</RESULTING-PROPERTIES>
  <SYMBOL>ValueExample_SymbolName</SYMBOL>
</MC-DATA-INSTANCE>
```

There are restrictions on the values of the tags <BIT-POSITION> and <NUMBER-OF-BITS>:

- The value of <BIT-POSITION> must always be $\leq (\text{<BASE-TYPE-SIZE>} - 1)$.
- The value of $(\text{<BIT-POSITION>} + \text{<NUMBER-OF-BITS>})$ should always be $\leq (\text{<BASE-TYPE-SIZE>})$.

An error message is thrown if these conditions are not met, and the corresponding block is not considered for A2L generation.

If the <SW-HOST-VARIABLE> tag is present for the *BIT_MASK*, then the validation of the *baseType* size is done against the *baseType* referenced in the host variable.

ADX ENTRY for BIT_MASK

An *ADX ENTRY* is created for every block which contains the tags <BIT-POSITION> and <NUMBER-OF-BITS> e.g.:

```
<MEMORY-ELEMENT>
  <LABEL-NAME>ValueExample_SymbolName</LABEL-NAME>
  <MAP-LABEL-NAME>ValueExample</MAP-LABEL-NAME>
  <BIT-POSITION>12</BIT-POSITION>
  <NUMBER-OF-BITS>4</NUMBER-OF-BITS>
</MEMORY-ELEMENT>
```

If the <SW-HOST-VARIABLE> tag is present for the *BIT_MASK*, then the ADX entry generation would change. If the host variable is provided, the *Symbol Name* of the host variable will be added for <LABEL-NAME>hostVariable_symbol</LABEL-NAME>. If the host variable is not present, then the Symbol of the bit variable is used for the generation of the <LABEL-NAME>. e.g.:

```
<MEMORY-ELEMENT>
  <LABEL-NAME>ValueExample_SymbolName_of_HostVariable</LABEL-NAME>
  <MAP-LABEL-NAME>ValueExample</MAP-LABEL-NAME>
  <BIT-POSITION>12</BIT-POSITION>
  <NUMBER-OF-BITS>4</NUMBER-OF-BITS>
</MEMORY-ELEMENT>
```

IF_DATA

RTA-A2L supports the *ETK* and *XCP IfData* blocks creation, using the *xpand* template files.

Currently the *IfData name* and the Data record contents are supported.

If the *IfData name* is not set in the templates, then RTA-A2L sets the default name as below.

For *CalObjects*, the name is set as "defaultMeasureIfdata_[index]", where *index* = 0,1,2 etc.

Please refer to the *RTA-A2L Error Description* document for the list of log entries supported for *MEASUREMENTs*.

5.7.12 SYMBOL_LINK Details

5.7.12.1 Creation of SYMBOL_LINK

The *SYMBOL_LINK* is generated inside *CHARACTERISTICs*, *MEASUREMENTs* and *AXIS_PTs* for A2L files of version ASAP 1.60 or greater.

It can be enabled in the RTA-A2L Options file by using `--enableSymbolLink y`. (Please refer to RTA-A2L Options (Section 3.2) for more details.)

The structure of the A2L file is as follows:

```
SYMBOL_LINK string SymbolName Long Offset
```

e.g.

```
SYMBOL_LINK "_VehicleSpeed" 0
```

The *SYMBOL_LINK* consists of the *SymbolName* and *Offset value* calculated from the root element. For *CHARACTERISTICs* of type *CURVE* and *MAP*, *SYMBOL_LINK* is not generated.

SymbolName:

- The customized *SymbolName* in the <MC-DATA-INSTANCE> from the input ARXML file.

Offset:

- The *Offset* value is calculated from the root element. It is 0 for all *MEASUREMENTs* and *CHARACTERISTICs*, except those of type simple *ARRAY*. In this case, the *Offset* value is calculated by using the maximum array dimensions and the current indices of the element and its *DataSize*.



NOTE

For *CHARACTERISTICs*, the *DataSize* is fetched from the *FNC_VALUES* of the *RecordLayout*.

Customized SymbolName generation:

- If the *SymbolName* is a simple *ARRAY*, then the *SymbolName* in the *SYMBOL_LINK* is generated without array indexes. e.g.:

```
char [2][3] -> char
```

More examples:

char[3].element[2]	SYMBOL_LINK "char[3].element" 0
char[3].element[2][3]	SYMBOL_LINK "char[3].element" 0
x[1].Curve	No <i>SYMBOL_LINK</i> generated.
x[1].Map	No <i>SYMBOL_LINK</i> generated.
char[3] SYMBOL_LINK "char" 6	SYMBOL_LINK "char" 6
char[1][3]	SYMBOL_LINK "char" 12
EpmNEngArray_Tina[0][1].EngN	SYMBOL_LINK "EpmNEngArray_Tina[0][1].EngN" 0
CoEngStArray[0].EngN	SYMBOL_LINK "CoEngStArray[0].EngN" 0
EngN_Record_t.EngN_Array_t	SYMBOL_LINK "EngN_Record_t.EngN_Array_t" 0
meas[3]	SYMBOL_LINK "meas" 6
meas[1][3]	SYMBOL_LINK "meas" 12

5.7.13 AXIS_DESCR Block Details

The *AXIS_DESCR* blocks will be appearing for the *CURVE* and *MAP* types of *CHARACTERISTICS*. The <SW-CALPRM-AXIS> of the <MC-DATA-INSTANCE> will be the input for the *AXIS_DESCR* block creation. Currently, RTA-A2L supports the *STD_Axis* and the *COM_AXIS* types of *AXIS_DESCR* block.

5.7.13.1 STD_Axis AXIS_DESCR Block

The input for the *AXIS_DESCR* creation looks as below. The <SW-CALPRM-AXIS-SET> represents many <SW-CALPRM-AXIS> blocks. These <SW-CALPRM-AXIS>s will be considered for the creation of *AXIS_DESCR* blocks for the X-Axis, Y-Axis...

If the value of <SW-AXIS-INDEX> is 1, then the *CURVE* type of *CHARACTERISTICS* supports the X-Axis and *AXIS_DESCR* block.

If the <SW-AXIS-INDEX> is missing/invalid, then by default the first Axis in the <SW-CALPRM-AXIS-SET> will be considered as the X-Axis input for the *AXIS_DESCR* block creation.

The *MAP* type of the *CHARACTERISTIC* supports the X-Axis and the Y-Axis *AXIS_DESCR* blocks. If the value of <SW-AXIS-INDEX> is 1, then the *AXIS_DESCR* for X-Axis will be created. If the value of <SW-AXIS-INDEX> is 2, then the *AXIS_DESCR* for the Y-Axis will be created. If the <SW-AXIS-INDEX> is missing /invalid, then by default the first Axis in the <SW-CALPRM-AXIS-SET> will be considered as the X-Axis *AXIS_DESCR* input and the second Axis as the input for the Y-Axis *Axis_DESCR* block creation.

```

<MC-DATA-INSTANCE>
  <SHORT-NAME>characteristicParameter1</SHORT-NAME>
  <LONG-NAME>
    <L-4 L="EN">LongName of characteristicParameter1</L-4>
  </LONG-NAME>
  <CATEGORY>CURVE</CATEGORY>
  <FLAT-MAP-ENTRY-REF DEST="FLAT-INSTANCE-DESCRIPTOR">/EcuFlatMap/EcuFlatMap/LindaPerInstParaCurve_HelloImFromBSW8</FLAT-MAP-ENTRY-REF>
  <RESULTING-PROPERTIES>
    <SW-DATA-DEF-PROPS-VARIANTS>
      <SW-DATA-DEF-PROPS-CONDITIONAL>
        <SW-CALIBRATION-ACCESS>READ-WRITE</SW-CALIBRATION-ACCESS>
        <SW-CALPRM-AXIS-SET>
          <SW-CALPRM-AXIS>
            <SW-AXIS-INDEX>1</SW-AXIS-INDEX>
            <SW-AXIS-INDIVIDUAL>
              <SW-VARIABLE-REFS>
                <MC-DATA-INSTANCE-VAR-REF DEST="MC-DATA-INSTANCE">/RB/RTE/BswImplementation/Variable1</MC-DATA-INSTANCE-VAR-REF>
              </SW-VARIABLE-REFS>
              <COMPU-METHOD-REF BASE="Rte_MCSD_CompuMethods"
                DEST="COMPU-METHOD">/RB/DataTypes/CompuMethods/CompumethodsUnits_CM_LindaPerInstPara2</COMPU-METHOD-REF>
              <SW-MAX-AXIS-POINTS>7</SW-MAX-AXIS-POINTS>
              <DATA-CONSTR-REF BASE="Rte_MCSD_DataConstrs"
                DEST="DATA-CONSTR">/RB/DataTypes/DataConstraints/DataConstraints_DC_Linda3</DATA-CONSTR-REF>
            </SW-AXIS-INDIVIDUAL>
            <BASE-TYPE-REF BASE="Rte_MCSD_SwBaseTypes"
              DEST="SW-BASE-TYPE">/RB/DataTypes/SwBaseTypes/uint16</BASE-TYPE-REF>
            <DISPLAY-FORMAT>%0.5f</DISPLAY-FORMAT>
          </SW-CALPRM-AXIS>
        </SW-CALPRM-AXIS-SET>
        <COMPU-METHOD-REF BASE="Rte_MCSD_CompuMethods"
          DEST="COMPU-METHOD">/RB/DataTypes/CompuMethods/CompumethodsUnits_CM_LindaPerInstPara1</COMPU-METHOD-REF>
        <DATA-CONSTR-REF BASE="Rte_MCSD_DataConstrs"
          DEST="DATA-CONSTR">/RB/DataTypes/DataConstraints/DataConstraints_DC_Linda3</DATA-CONSTR-REF>
        <DISPLAY-FORMAT>%62f</DISPLAY-FORMAT>
        <SW-RECORD-LAYOUT-REF BASE="Rte_MCSD_SwRecordLayouts"
          DEST="SW-RECORD-LAYOUT">/RB/DataTypes/SwRecordLayouts/RecordLayoutCurve</SW-RECORD-LAYOUT-REF>
        <UNIT-REF BASE="Rte_MCSD_Units"
          DEST="UNIT">CompumethodsUnits_U_Linda1</UNIT-REF>
        <BASE-TYPE-REF BASE="Rte_MCSD_SwBaseTypes"
          DEST="SW-BASE-TYPE">/RB/DataTypes/SwBaseTypes/uint16</BASE-TYPE-REF>
      </SW-DATA-DEF-PROPS-CONDITIONAL>
    </SW-DATA-DEF-PROPS-VARIANTS>
  </RESULTING-PROPERTIES>
  <SYMBOL>LindaPerInstPara</SYMBOL>
</MC-DATA-INSTANCE>

```

Figure 5.30. AUTOSAR Input for the *AXIS_DESCR* block creation

5.7.13.2 COM_AXIS AXIS_DESCR Block

For a grouped *CURVE* type, the *COM_AXIS* type of *AXIS_DESCR* blocks will be created using the *COM_AXIS* type of <MC-DATA-INSTANCE>, which has been referenced from the grouped *CURVE* type of <MC-DATA-INSTANCE>.

In creating the *COM_AXIS* type of *AXIS_DESCR* blocks, the *AXIS_PTS* blocks will first be created. By referencing it, the *AXIS_DESCR* block will then be created.

The following shows the <SW-CALPRM-AXIS> for the *AXIS_PTS* block creation:

```

<MC-DATA-INSTANCE>
  <SHORT-NAME>Com_iChrgLimBatVDCHYB_CUR</SHORT-NAME>
  <CATEGORY>CURVE</CATEGORY>
  <RESULTING-PROPERTIES>
    <SW-DATA-DEF-PROPS-VARIANTS>
      <SW-DATA-DEF-PROPS-CONDITIONAL>
        <SW-CALIBRATION-ACCESS>READ-WRITE</SW-CALIBRATION-ACCESS>
        <SW-CALPRM-AXIS-SET>
          <SW-CALPRM-AXIS>
            <SW-AXIS-INDEX>1</SW-AXIS-INDEX>
            <SW-AXIS-GROUPED>
              <MC-DATA-INSTANCE-REF BASE="Rte_MCSd_McDataInstance"
                                DEST="MC-DATA-INSTANCE">Rte/ComAxisExample</MC-DATA-INSTANCE-REF>
            </SW-AXIS-GROUPED>
          </SW-CALPRM-AXIS>
        </SW-CALPRM-AXIS-SET>
        <COMPU-METHOD-REF BASE="Rte_MCSd_CompuMethods"
                          DEST="COMPU-METHOD">RB_PT_PCT_ComRx_HBatLim_CompuMethods_OneToOne_DegCgrd</COMPU-METHOD-REF>
        <DATA-CONSTR-REF BASE="Rte_MCSd_DataConstrs"
                        DEST="DATA-CONSTR">RB_PT_PCT_ComRx_HBatLim_DataConstrs_DataConstrC_m400p0_390p0</DATA-CONSTR-REF>
        <BASE-TYPE-REF BASE="Rte_MCSd_SwBaseTypes"
                      DEST="SW-BASE-TYPE">float32</BASE-TYPE-REF>
        <SW-RECORD-LAYOUT-REF BASE="Rte_MCSd_SwRecordLayouts"
                              DEST="SW-RECORD-LAYOUT">RB_PT_Common_CentralElements_SwRecordLayouts_RB_CurveGroup_R32</SW-RECORD-LAYOUT-REF>
        <UNIT-REF BASE="Rte_MCSd_Units"
                 DEST="UNIT">RB_PT_PCT_ComRx_HBatLim_Units_Ampr</UNIT-REF>
      </SW-DATA-DEF-PROPS-CONDITIONAL>
    </SW-DATA-DEF-PROPS-VARIANTS>
  </RESULTING-PROPERTIES>
  <SYMBOL>Com_iChrgLimBatVDCHYB_CUR</SYMBOL>
</MC-DATA-INSTANCE>

```

Figure 5.31. Grouped Axis *CURVE*

In the figure above, the <MC-DATA-INSTANCE-REF> refers to the *COM_AXIS* type of <MC-DATA-INSTANCE>, which will be used for the *AXIS_PTS* and *AXIS_DESCR* blocks creations:

```

<MC-DATA-INSTANCE>
  <SHORT-NAME>ComAxisExample</SHORT-NAME>
  <CATEGORY>COM_AXIS</CATEGORY>
  <RESULTING-PROPERTIES>
    <SW-DATA-DEF-PROPS-VARIANTS>
      <SW-DATA-DEF-PROPS-CONDITIONAL>
        <SW-CALIBRATION-ACCESS>READ-WRITE</SW-CALIBRATION-ACCESS>
        <SW-CALPRM-AXIS-SET>
          <SW-CALPRM-AXIS>
            <SW-AXIS-INDEX>1</SW-AXIS-INDEX>
            <SW-AXIS-INDIVIDUAL>
              <SW-VARIABLE-REFS>
                <MC-DATA-INSTANCE-VAR-REF BASE="Rte_MCSd_McDataInstance"
                                          DEST="MC-DATA-INSTANCE">Rte/LindaStatMemVarX</MC-DATA-INSTANCE-VAR-REF>
              </SW-VARIABLE-REFS>
              <COMPU-METHOD-REF BASE="Rte_MCSd_CompuMethods"
                                DEST="COMPU-METHOD">RB_PT_PCT_ComRx_HBatLim_CompuMethods_OneToOne_DegCgrd</COMPU-METHOD-REF>
              <UNIT-REF BASE="Rte_MCSd_Units"
                        DEST="UNIT">RB_PT_PCT_ComRx_HBatLim_Units_DegCgrd</UNIT-REF>
              <SW-MAX-AXIS-POINTS>10</SW-MAX-AXIS-POINTS>
              <DATA-CONSTR-REF BASE="Rte_MCSd_DataConstrs"
                              DEST="DATA-CONSTR">RB_PT_PCT_ComRx_HBatLim_DataConstrs_DataConstrC_m400p0_390p0</DATA-CONSTR-REF>
            </SW-AXIS-INDIVIDUAL>
            <DISPLAY-FORMAT>%0.5f</DISPLAY-FORMAT>
            <BASE-TYPE-REF BASE="Rte_MCSd_SwBaseTypes"
                          DEST="SW-BASE-TYPE">float32</BASE-TYPE-REF>
          </SW-CALPRM-AXIS>
        </SW-CALPRM-AXIS-SET>
        <BASE-TYPE-REF BASE="Rte_MCSd_SwBaseTypes"
                      DEST="SW-BASE-TYPE">float32</BASE-TYPE-REF>
        <MC-FUNCTION>ExampleFuntion</MC-FUNCTION>
        <SW-RECORD-LAYOUT-REF BASE="Rte_MCSd_SwRecordLayouts"
                              DEST="SW-RECORD-LAYOUT">RB_PT_Common_CentralElements_SwRecordLayouts_RB_Axis_R32</SW-RECORD-LAYOUT-REF>
      </SW-DATA-DEF-PROPS-CONDITIONAL>
    </SW-DATA-DEF-PROPS-VARIANTS>
  </RESULTING-PROPERTIES>
  <SYMBOL>X_TBat_ILim_HYB</SYMBOL>
</MC-DATA-INSTANCE>

```

Figure 5.32. *COM_AXIS* *McDataInstance* example

Currently, RTA-A2L supports the following attributes for the *AXIS_DESCR*:


```

/begin AXIS_DESCR
  Attribute
  InputQuantity
  Conversion
  MaxAxisPoints
  LowerLimit
  UpperLimit
  [-> FORMAT]
  [-> PHYS_UNIT]
  [-> READ-ONLY]
  [-> DEPOSIT]
  [-> AXIS_PTS_REF]
  [-> STEP_SIZE]
  [-> BYTE_ORDER]
/end AXIS_DESCR

```

Attribute

This will be set to the *STD_Axis* for the Simple *CURVE* and Simple *MAP* types. For the Grouped *CURVE* type, this will be set as *COM_AXIS*.

InputQuantity

The `<SW-VARIABLE-REF>` will be the input for this attribute creation if provided (there might be many `<SW-VARIABLE-REF>` occurrences). Since the ASAP standard supports the setting of only one *InputQuantity*, the first `<SW-VARIABLE-REF>` will be considered.

From the `<SW-VARIABLE-REF>`, the `<MC-DATA-INSTANCE>` reference will be considered, and the corresponding A2L *MEASUREMENT* block will be fetched from the A2LModule. This *MEASUREMENT* object will be set as the *InputQuantity*.

If no `<SW-VARIABLE-REF>`s exist, or if the referenced variable is of type *NOT-ACCESSIBLE*, or `<SW-CALIBRATION-ACCESS>` is unavailable, or a proxy `<SW-VARIABLE-REF>` exists, or no *MEASUREMENT* block occurs in the A2LModule for the corresponding `<MC-DATA-INSTANCE>`, then the *NO_INPUT_QUANTITY* keyword will be set.

Conversion

To set this attribute, RTA-A2L refers to both the `<COMPU-METHOD-REF>` and `<SW-VARIABLE-REF>` tags:

If both the `<COMPU-METHOD-REF>` of the Axis and the `<SW-VARIABLE-REF>` exist in the `<SW-CALPRM-AXIS>` (*AXIS_DESCR* is set with the *InputQuantity*), then the definition of the *CompuMethod* of the `<SW-CALPRM-AXIS>`, as well as the `<COMPU-METHOD>` provided in the `<SW-VARIABLE-REF>`, should be the same (except the *ShortName* and *LongIdentifier/Desc*).

In this case, the `<COMPU-METHOD-REF>` from the Axis will be considered and the corresponding A2L *Compu* will be set as the *Conversion*.

If the definitions are not same, then a warning message will be provided, and the *CompuMethod* of the Axis will be set. Or else, if only the `<COMPU-METHOD-REF>` exists in the Axis, then the corresponding A2L *Compu* object will be set as the *Conversion*.

If only the `<SW-VARIABLE-REF>` exists in the Axis, then the `<COMPU-METHOD-REF>` of the corresponding `<MC-DATA-INSTANCE>` will be fetched, and the corresponding A2L *Compu* object will be set as the *Conversion*.

If no `<COMPU-METHOD-REF>` and `<SW-VARIABLE-REF>` exist, then the

NO_COMPU_METHOD keyword will be set.

If the proxy <COMPU-METHOD-REF> exists, the *Conversion* will be set from the <SW-VARIABLE-REF> *Compu* object if it exists. Or else, the *NO_COMPU_METHOD* keyword will be set.

When checking the *CompuMethods* definitions, only the attributes currently supported by the RTA-A2L will be considered i.e. *ConversionType*, *Format*, *Unit*, *COEFS*, *COMPU_TAB_REF*, *REF_UNIT*. However, checking should also be done for any new attributes that are supported.

MaxAxisPoints

The <SW-MAX-AXIS-POINTS> tag is the input for the *MaxAxisPoints* creation.

If <SW-MAX-AXIS-POINTS> does not exist or is empty, then no *AXIS_DESCR* blocks will be created.

If the variation point occurs with the value >=PRE_COMPILE_TIME, along with the system condition and expressions in <SW-MAX-AXIS-POINTS>, then no *AXIS_DESCR* blocks will be created.

LowerLimit

UpperLimit

The <DATA-CONSTR-REF> and the <BASE-TYPE-REF> of the <SW-CALPRM-AXIS> will be the input for the *LowerLimit* and the *UpperLimit* creation.

If the *DataConstrRef* doesn't exist or the referenced *DataConstr* Object is invalid, then, using the *baseType* object, the default limits will be set.

If the *baseType* at the Axis Level does not exist or is invalid, then the *baseType* will be fetched from the provided *VariableRef* object. If no *VariableRef* exists, then the *baseType* will be fetched from the referenced *RecordLayout* X-Axis *baseType* for the X-Axis *AXIS_DESCR* block, and from the *RecordLayout* Y-Axis *baseType* for the Y-Axis *Axis_DESC* block.

If the *baseType* at the Axis Level does not exist or is invalid, then the *baseType* will be fetched from the provided *VariableRef* object. If no *VariableRef* exists, then the *baseType* will be fetched from the referenced *RecordLayout* X-axis *baseType* for the X-axis *AXIS_DESC* block and from the *RecordLayout* Y-axis *baseType* for the Y-axis *AXIS_DESC* block. Also, the *baseType* from the *recordLayout* should be valid

Also, the *baseType* from the *RecordLayout* should be valid and not *FLOAT*. For the *FLOAT* type of *baseType*, the <DATA-CONSTR-REF> is mandatory.

Format

The <DISPLAY-FORMAT> tag of the <SW-CALPRM-AXIS> will be referenced to set this value.

PHYS_UNIT

If the <SW-AXIS-INDIVIDUAL> tag inside <MC-DATA-INSTANCE> has a valid <UNIT-REF> tag, then *PHYS_UNIT* will be generated for the *COM_AXIS* *AXIS_DESCR* block in the A2L file.

The *PHYS_UNIT* will be generated in the A2L file as *PHYS_UNIT* <UNIT>

displayName.

e.g. *PHYS_UNIT* m/s

If the *<UNIT>* referenced by the *<SW-AXIS-INDIVIDUAL>* tag inside *<MC-DATA-INSTANCE>* does not have *<DISPLAY-NAME>* or an empty *<DISPLAY-NAME>*, then *PHYS_UNIT* will not be generated in the A2L file for the *COM_AXIS* *AXIS_DESCR* and a warning message will be logged with message ID: *A2L_MCS_013* in the *tool.log* file.

e.g.

ICE Message ID: A2L_MCS_013, WARN: *<DISPLAY-NAME>* in *<UNIT>* 'RB_PT_PCT_ComRx_HBatLim_Units_Ampr' is Missing/Empty, Hence Physical Unit will not be generated.

If the *<UNIT>* referenced in the *<COMPU-METHOD>* of the *<SW-AXIS-INDIVIDUAL>* tag inside *<MC-DATA-INSTANCE>* is different from the *<UNIT>* referenced by the *<SW-AXIS-INDIVIDUAL>* tag inside *<MC-DATA-INSTANCE>*, then *PHYS_UNIT* will be generated and a warning message will be logged with message ID: *A2L_MCS_WARN_038* in the *tool.log* file.

e.g.

ICE Message ID: A2L_MCS_WARN_038, WARN: *<UNIT-REF>* RB_PT_PCT_ComRx_HBatLim_Units_Ampr and compuMethod *<UNIT-REF>* RB_PT_PCT_ComRx_HBatLim_Units_NoUnit which are referred in *<MC-DATA-INSTANCE>* Com_iLimBatTempMin_C are not compatible.

READ-ONLY

If the *<SW-CALIBRATION-ACCESS>* is set as *READ-ONLY*, then this attribute will be created for the *AXIS_DESCR* blocks.

If the *<SW-CALIBRATION-ACCESS>* is of type *NOT-ACCESSIBLE* or *<SW-CALIBRATION-ACCESS>* is unavailable, then this tag will be ignored and the *READ-ONLY* attribute will NOT be created in the *AXIS_DESCR* blocks, and an error message will be logged.

If the *<SW-CALIBRATION-ACCESS>* tag is missing at the *<SW-CALPRM-AXIS>* level, and the *Parent <MC-DATA-INSTANCE>* contains the *READ-ONLY* type of *<SW-CALIBRATION-ACCESS>*, then the *READ-ONLY* keyword will be set for the *AXIS_DESCR* blocks.

In the case of a Grouped *CURVE/MAP*, this attribute will be set for the *AXIS_DESCR* blocks, if the referenced *COM_AXIS* contains the *READ-ONLY* attribute.

DEPOSIT

This attribute will be set based on the *<SW-RECORD-LAYOUT-REF>* of the *<MC-DATA-INSTANCE>*.

If the referenced *<SW-RECORD-LAYOUT>* contains the *<SW-RECORD-LAYOUT-V-PROP>* as *LEFTDIFF* or *RIGHTDIFF*, then the *DEPOSIT* will be set as *DIFFERENCE*. Or else, the *DEPOSIT* will be set as *ABSOLUTE*.

The *A2LRecordLayout* will be created and set to the *CHARACTERISTIC* block.

AXIS_PTS_REF

This attribute will only be set for the Grouped Type of *Calprm*.

It will be set based on the `<MC-DATA-INSTANCE-REF>` of the `<SW-Axis--GROUPED>` *CalprmAxis*.

Before setting this attribute, all the *AXIS_PTS* blocks should be populated to the A2LModule. The *ShortName* of the referenced *AXIS_PTS* will be set as the *AXIS_PTS_REF* value.

If the referenced *AXIS_PTS* is of type *NOT-ACCESSIBLE* or `<SW-CALIBRATION-ACCESS>` is unavailable, then this attribute will not be generated in the A2L file.

Please refer to the *RTA-A2L Error Description* document for the list of log entries supported for the creation of the *AXIS_DESCR* block.

STEP_SIZE

If the `<MC-DATA-INSTANCE>` of type *MC-PARAMETER-INSTANCE* consists of a `<STEP-SIZE>` tag, then this attribute will be created for the *AXIS_DESCR* blocks.

e.g. STEP_SIZE 0.025

BYTE_ORDER

If the `<BYTE-ORDER>` value of the `<SW-BASE-TYPE>` is specified, the `<MC-DATA-INSTANCE>` referencing this `<SW-BASE-TYPE>` will include a *BYTE_ORDER* keyword with the corresponding value in the A2L file.

Only *MOST-SIGNIFICANT-BYTE-FIRST* and *MOST-SIGNIFICANT-BYTE-LAST* values are supported for the *BYTE-ORDER* in A2L file generation. Consequently, *MSB-FIRST* and *MSB-LAST* will be produced in the A2L file.

The *OPAQUE* value for `<BYTE-ORDER>` is not supported for A2L generation.

5.7.14 A2L FUNCTION Block Details

The A2L *FUNCTION* blocks can be created using the `<MC-FUNCTION>` tags, which appear in the `<MC-DATA-INSTANCE>`s, or by using the `<MC-FUNCTION>` blocks.

5.7.14.1 A2L FUNCTION Block Creation Using the `<MC-FUNCTION>` Tags

The `<MC-DATA-INSTANCE>` tag corresponding to the *MEASUREMENT* or *CHARACTERISTIC* blocks in the *ARXML* files may consist of the `<MC-FUNCTION>` attribute defined in it, for which the A2L *FUNCTION* block will be generated in the A2L file.

If the `<SW-CALIBRATION-ACCESS>` tag of the `<MC-DATA-INSTANCE>` is of type *NOT-ACCESSIBLE*, or `<SW-CALIBRATION-ACCESS>` is unavailable, then that entry will not be added to the *FUNCTION* block.

In the case of complex `<MC-DATA-INSTANCE>`s, if the `<MC-FUNCTION>` is missing, then it will be referenced from the root *Parent* `<MC-DATA-INSTANCE>`, if it exists.

Currently, the *FUNCTION* block generated consists of *DEF_CHARACTERISTIC* and *LOC_MEASUREMENT* objects defined in it as per the *STRUCTURE* below:

/begin FUNCTION

```

    Name
    LongIdentifier
    [-> DEF_CHARACTERISTIC]
    [-> LOC_MEASUREMENT]
/end FUNCTION

```

Name

The Name of *FUNCTION* block is the *Name* of the <MC-FUNCTION> attribute present in *McDataInstance*.

LongIdentifier

Currently, the *LongIdentifier* is considered as an empty string "".

DEF_CHARACTERISTIC

This will be the list of the <SHORT-NAME>s of the <MC-DATA-INSTANCE> of type <MC-PARAMETER-INSTANCE>, which defines the <MC-FUNCTION> attribute.

All the *CHARACTERISTIC* blocks, which will have the same <MC-FUNCTION> attribute, will be listed to the *DEF-CHARACTERISTIC*.

LOC-MEASUREMENT

This will be the list of <SHORT-NAME>s of the <MC-DATA-INSTANCE> of type <MC-VARIABLE-INSTANCE>, which defines the <MC-FUNCTION> attribute.

All the *MEASUREMENT* blocks, which will have the same <MC-FUNCTION> attribute, will be listed to the *LOC-MEASUREMENT*.

The example below shows a variable (*MEASUREMENT*) block defining a <MC--FUNCTION> attribute:

```

<MC-DATA-INSTANCE>
  <SHORT-NAME>Com_swtATempRx_CanAP_PduCtrl_C</SHORT-NAME>
  <LONG-NAME>
    <L-4 L="EN">Calibration parameter to control Ipdu A_TEMP_Rx</L-4>
  </LONG-NAME>
  <CATEGORY>VALUE</CATEGORY>
  <RESULTING-PROPERTIES>
    <SW-DATA-DEF-PROPS-VARIANTS>
      <SW-DATA-DEF-PROPS-CONDITIONAL>
        <BASE-TYPE-REF DEST="SW-BASE-TYPE">
          BASE="Rte_MCSD_SwBaseTypes">AUTOSAR_Platform_BaseTypes_boolean</BASE-TYPE-REF>
        <SW-CALIBRATION-ACCESS>READ-WRITE</SW-CALIBRATION-ACCESS>
        <COMPU-METHOD-REF DEST="COMPU-METHOD">
          BASE="Rte_MCSD_CompuMethods">AUTOSAR_Com_CompuMethods_Com_IPduStatus_enum_type</COMPU-METHOD-REF>
        <DATA-CONSTR-REF DEST="DATA-CONSTR">
          BASE="Rte_MCSD_DataConstrs">AUTOSAR_Com_DataConstraints_Com_DataConstraint_b</DATA-CONSTR-REF>
        <DISPLAY-FORMAT>%5.0f</DISPLAY-FORMAT>
        <MC-FUNCTION>Com</MC-FUNCTION>
        <UNIT-REF DEST="UNIT">
          BASE="Rte_MCSD_Units">AUTOSAR_Com_Units_NoUnit</UNIT-REF>
      </SW-DATA-DEF-PROPS-CONDITIONAL>
    </SW-DATA-DEF-PROPS-VARIANTS>
  </RESULTING-PROPERTIES>
  <SYMBOL>Com_swtATempRx_CanAP_PduCtrl_C</SYMBOL>
</MC-DATA-INSTANCE>

```

Figure 5.33.AUTOSAR McFunction block

The following example shows the *FUNCTION* block generated in the A2L file for the variable instance presented in the above sample:

```

/begin FUNCTION
    Com
    ""
    /begin DEF_CHARACTERISTIC
        Com_swtATempRx_CanAP_PduCtrl_C
    /end DEF_CHARACTERISTIC
/end FUNCTION

```

Figure 5.34. A2L *FUNCTION* block Details

Please refer to the *RTA-A2L Error Description* document for the log entry details for the *FUNCTION* block creation.

5.7.14.2 A2L FUNCTION Block Creation Using the <MC-FUNCTION> Blocks

AUTOSAR files >= v4.1.1 will support <MC-FUNCTION> blocks as input to create the *FUNCTION* blocks. The *McFunction* blocks will contain all the information required to create the A2L *FUNCTION* block, except the *FUNCTION_VERSION* attribute.

If the input *McSupport* file contains both the <MC-FUNCTION> tag and the <MC-FUNCTION> blocks, preference will be given to the <MC-FUNCTION> blocks.

In case of duplicate *McFunction* blocks, the first occurrence of the *McFunction* block is considered and generated in the A2L file.

The following attributes are supported for the *FUNCTION* blocks created from the *McFunction* input blocks:

```

/begin FUNCTION ident Name string
LongIdentifier [>ANNOTATION] *
    [-> DEF_CHARACTERISTIC]
    [-> REF_CHARACTERISTIC]
    [-> FUNCTION_VERSION]
    [-> IN_MEASUREMENT]
    [-> LOC_MEASUREMENT]
    [-> OUT_MEASUREMENT]
    [-> SUB_FUNCTION]
/end FUNCTION

```

Name

The <SHORT-NAME> of the *McFunction* block input.

LongIdentifier

This will be created using the <LONG-NAME> tag of the <MC-FUNCTION> (please refer to Section 5.7.15 for more details).

ANNOTATION

This will be created only if the <ANNOTATIONS> tag exists. RTA-A2L supports the creation of the *ANNOTATION_LABEL* and the *ANNOTATION_TEXT*. (Please refer to Section 5.7.19 for more details.)

DEF_CHARACTERISTIC

The list of *CHARACTERISTIC* blocks corresponding to `<DEF-Calprm-SET>/<MC--DATA-INSTANCE-REF>` will be set as the *DEF_CHARACTERISTICS*.

FUNCTION_VERSION

This information will be the `<REVISION-LABEL>` information of the `<MC-FUNCTION>`.

IN_MEASUREMENT

The list of *MEASUREMENT* blocks corresponding to `<IN-MEASUREMENT>/<MC--DATA-INSTANCE-REFS>`s will be set as the *IN_MEASUREMENT*.

OUT_MEASUREMENT

The list of *MEASUREMENT* blocks corresponding to `<OUT-MEASUREMENT>/<MC-DATA-INSTANCE-REFS>`s will be set as the *OUT_MEASUREMENT*.

LOC_MEASUREMENT

The list of *MEASUREMENT* blocks corresponding to `<LOC-MEASUREMENT>/<MC-DATA-INSTANCE-REFS>`s will be set as the *LOC_MEASUREMENT*.

REF_CHARACTERISTIC

The list of *CHARACTERISTIC* blocks corresponding to `<REF-Calprm-SET>/<MC--DATA-INSTANCE-REFS>`s will be set as the *REF_CHARACTERISTIC*.

SUB_FUNCTION

The list of `<SUB-FUNCTION>`s corresponding to the A2L FUNCTION will be set as the *SUB_FUNCTION*.

Please refer to the *RTA-A2L Error Description* document for the log messages supported for the *FUNCTION* block creation.

5.7.15 Creation of A2L LongIdentifier

The *LongIdentifier* for any of the A2L blocks will be the string value of `<LONG-NAME>`.

Currently, RTA-A2L refers to the language type set by the options file (`--language`) to select the `<LONG-NAME>`.

If *longName* does not contain a build-specified language but contains *FOR-ALL*, then that will be given preference.

e.g. if LanguageType - DE

```
<LONG-NAME>
  <L-4 L="FOR-ALL">FOR-ALL  longName for Unit2</L-4>
  <L-4 L="EN">EN  longName for Unit2</L-4>
</LONG-NAME>
```

Then, *LongIdentifier* = FOR-ALL *longName* for Unit2.

If there is no `--language` option in the options file, then RTA-A2L considers *EN* as the default type.

If both the options file provided language type and the *FOR-ALL* type do not appear in the `<LONG-NAME>`, then preference will be given to *EN*.

e.g. if LanguageType - DE

```
<LONG-NAME>
  <L-4 L="EN">EN longName for Unit2</L-4>
  <L-4 L="IT">IT longName for Unit2</L-4>
</LONG-NAME>
```

Then, *LongIdentifier* = *EN longName for Unit2*

If NONE of the options file specifies a language type i.e. the *FOR-ALL* and *EN* types do not appear in the input file, then an empty string "" will be generated.

The following condition sets the *LongIdentifier*:

```
If the <LONG-NAME> exists, then
LongIdentifier = <LONG-NAME>
```

If the <LONG-NAME> tag is missing, then a warning message will be thrown.

5.7.15.1 Creation of LongIdentifier for the STRUCTURE elements

For *STRUCTURE* elements (*CHARACTERISTIC*, *MEASUREMENT*), the *LongIdentifier* will be the combination of the *Parents' longName* and the *subElements' longName* separated with "/". (This change will be from the ICE 2.0.0 release. Earlier versions of the RTA-A2L did not consider the *Parent* element *longName* for the *Structure subElements*.)


```

<MC-DATA-INSTANCE>
  <SHORT-NAME>EpmNEngRecAr_MaxComplicated</SHORT-NAME>
  <LONG-NAME>
    <L-4 L="FOR-ALL">Longname for EpmNEngRecAr_MaxComplicated</L-4>
  </LONG-NAME>
  <CATEGORY>STRUCTURE</CATEGORY>
  ...
  <SUB-ELEMENTS>
    <MC-DATA-INSTANCE>
      <SHORT-NAME>EngNControl1</SHORT-NAME>
      <LONG-NAME>
        <L-4 L="FOR-ALL">Longname for EngNControl1</L-4>
      </LONG-NAME>
      <CATEGORY>STRUCTURE</CATEGORY>
      ...
      <SUB-ELEMENTS>
        <MC-DATA-INSTANCE>
          <SHORT-NAME>EngNControl2</SHORT-NAME>
          <LONG-NAME>
            <L-4 L="FOR-ALL">Longname for EngNControl2</L-4>
          </LONG-NAME>
          <CATEGORY>ARRAY</CATEGORY>
          <ARRAY-Size>2</ARRAY-Size>
          ...
          <SUB-ELEMENTS>
            <MC-DATA-INSTANCE>
              <SHORT-NAME>EngNControl123</SHORT-NAME>
              <LONG-NAME>
                <L-4 L="FOR-ALL">Longname for EngNControl123</L-4>
              </LONG-NAME>
              <CATEGORY>VALUE</CATEGORY>
              ...
            </MC-DATA-INSTANCE>
          </SUB-ELEMENTS>
        </MC-DATA-INSTANCE>
      </SUB-ELEMENTS>
    </MC-DATA-INSTANCE>
  </SUB-ELEMENTS>
</MC-DATA-INSTANCE>

```

Figure 5.35. *STRUCTURE-STRUCTURE-STRUCTURE-VALUE* combinations

In the above example, the *LongIdentifier* for the *VALUE McDataInstance* will be:

```

ShortName = EpmNEngRecAr_MaxComplicated.EngNControl1.EngNControl2.EngNControl23
LongIdentifier = "Longname for EpmNEngRecAr_MaxComplicated / Longname for EngNControl1 / Longname for EngNControl2 / Longname for EngNControl23".

```

Consider a *STRUCTURE-ARRAY-VALUE* example:

```

<MC-DATA-INSTANCE>
  <SHORT-NAME>CoEngStArray</SHORT-NAME>
  <LONG-NAME>
    <L-4 L="FOR-ALL">Longname for CoEngStArray</L-4>
  </LONG-NAME>
  <CATEGORY>ARRAY</CATEGORY>
  <ARRAY-SIZE>3</ARRAY-SIZE>
  ...
  <SUB-ELEMENTS>
    <MC-DATA-INSTANCE>
      <SHORT-NAME>element_0</SHORT-NAME>
      <LONG-NAME>
        <L-4 L="FOR-ALL">Longname for element_0</L-4>
      </LONG-NAME>
      <CATEGORY>ARRAY</CATEGORY>
      <ARRAY-SIZE>2</ARRAY-SIZE>
      ...
      <SUB-ELEMENTS>
        <MC-DATA-INSTANCE>
          <SHORT-NAME>element_0_value</SHORT-NAME>
          <CATEGORY>VALUE</CATEGORY>
          <LONG-NAME>
            <L-4 L="FOR-ALL">Longname for element_0_value</L-4>
          </LONG-NAME>
          ...
        </SUB-ELEMENTS>
      </SUB-ELEMENTS>
    </MC-DATA-INSTANCE>
  </SUB-ELEMENTS>
</MC-DATA-INSTANCE>

```

Figure 5.36. *STRUCTURE-ARRAY-VALUE*

In the above example, the *LongIdentifier* for the *VALUE McDataInstance* will be:

ShortName = EpmNEngRecAr_MaxComplicated.EngNControl1.[0] LongIdentifier = "Longname for EpmNEngRecAr_MaxComplicated / LongName for EngNControl1[0]" ShortName = EpmNEngRecAr_MaxComplicated.EngNControl1.[1] LongIdentifier = "Longame for EpmNEngRecAr_MaxComplicated / LongName for EngNControl1[1]"

5.7.15.2 Creation of LongIdentifier for the ARRAY elements

In the case of *ARRAY* elements, the *LongIdentifier* for the *subElements* will be the *LongIdentifier* value of the *Parent ARRAY* element, along with the indexes if they exist, or else an empty string "":

```

<MC-DATA-INSTANCE>
  <SHORT-NAME>EpmNEngRecAr_MaxComplicated</SHORT-NAME>
  <LONG-NAME>
    <L-4 L="FOR-ALL">Longname for EpmNEngRecAr_MaxComplicated</L-4>
  </LONG-NAME>
  <CATEGORY>STRUCTURE</CATEGORY>
  ...
  <SUB-ELEMENTS>
    <MC-DATA-INSTANCE>
      <SHORT-NAME>EngNControl1</SHORT-NAME>
      <LONG-NAME>
        <L-4 L="FOR-ALL">Longname for EngNControl1</L-4>
      </LONG-NAME>
      <CATEGORY>STRUCTURE</CATEGORY>
      ...
      <SUB-ELEMENTS>
        <MC-DATA-INSTANCE>
          <SHORT-NAME>EngNControl2</SHORT-NAME>
          <LONG-NAME>
            <L-4 L="FOR-ALL">Longname for EngNControl2</L-4>
          </LONG-NAME>
          <CATEGORY>STRUCTURE</CATEGORY>
          ...
          <SUB-ELEMENTS>
            <MC-DATA-INSTANCE>
              <SHORT-NAME>EngNControl123</SHORT-NAME>
              <LONG-NAME>
                <L-4 L="FOR-ALL">Longname for EngNControl123</L-4>
              </LONG-NAME>
              <CATEGORY>VALUE</CATEGORY>
              ...
            </MC-DATA-INSTANCE>
          </SUB-ELEMENTS>
        </MC-DATA-INSTANCE>
      </SUB-ELEMENTS>
    </MC-DATA-INSTANCE>
  </SUB-ELEMENTS>
</MC-DATA-INSTANCE>

```

Figure 5.37. *longName* consideration for the *ARRAY* elements

In the above example, the *longName* for the generated A2L blocks will be:

```

ShortName : CoEngStArray[0][0]
LongIdentifier : " Longname for CoEngStArray[0][0]"
ShortName : CoEngStArray[0][1]
LongIdentifier : " Longname for CoEngStArray[0][1]"
ShortName : CoEngStArray[1][0]
LongIdentifier : " Longname for CoEngStArray[1][0]"
ShortName : CoEngStArray[1][1]
LongIdentifier : " Longname for CoEngStArray[1][1]"
ShortName : CoEngStArray[2][0]
LongIdentifier : " Longname for CoEngStArray[2][0]"
ShortName : CoEngStArray[2][1]
LongIdentifier : " Longname for CoEngStArray[2][1]"

```

ARRAY-STRUCTURE-VALUE example:

```

<MC-DATA-INSTANCE>
  <SHORT-NAME>EngNControl1</SHORT-NAME>
  <LONG-NAME>
    <L-4 L="FOR-ALL">Longname for EngNControl1</L-4>
  </LONG-NAME>
  <CATEGORY>ARRAY</CATEGORY>
  <ARRAY-SIZE>2</ARRAY-SIZE>
  ...
  <SUB-ELEMENTS>
    <MC-DATA-INSTANCE>
      <SHORT-NAME>EngNControl2</SHORT-NAME>
      <LONG-NAME>
        <L-4 L="FOR-ALL">Longname for EngNControl2</L-4>
      </LONG-NAME>
      <CATEGORY>STRUCTURE</CATEGORY>
      ...
      <SUB-ELEMENTS>
        <MC-DATA-INSTANCE>
          <SHORT-NAME>EngNControl23</SHORT-NAME>
          <LONG-NAME>
            <L-4 L="FOR-ALL">Longname for EngNControl23</L-4>
          </LONG-NAME>
          <CATEGORY>VALUE</CATEGORY>
          ...
        </SUB-ELEMENTS>
      ...
    </SUB-ELEMENTS>
  </MC-DATA-INSTANCE>

```

Figure 5.38. ARRAY-STRUCTURE-VALUE LongIdentifier example

The *LongIdentifier* is as below:

```

Name : EngNControl1[0].EngNControl23
LongIdentifier : "Longname for EngNControl1[0]. Longname for EngNControl23"
Name : EngNControl1[1].EngNControl23
LongIdentifier : "Longname for EngNControl1[1]. Longname for EngNControl23"

```

5.7.15.3 Restrict the Length of Generated LongIdentifier to 255 Chars

If *longName* is more than 255 characters, then the first 80 characters are appended by '...' followed by the last 172 characters. This applies to the *LongIdentifiers* of *FUNCTION*, *CHARACTERISTIC*, *MEASUREMENT*, *Unit*, *CompuMethod*, *AXIS_PTS* and *CompuTable*.

This feature is disabled by default. To enable it, set the attribute `--cut_a2l_longname_255` to *Y* in the A2L options file as shown below:

```
--cut_a2l_longname_255 Y
```

5.7.16 DataTypes for the A2L Blocks

The *DataTypes* are created using the *SwBaseTypes* from AUTOSAR. The following

snapshot represents the *SwBaseType* *STRUCTURE*:

```
<AR-PACKAGE>
  <SHORT-NAME>SwBaseTypes</SHORT-NAME>
  <ELEMENTS>
    <SW-BASE-TYPE>
      <SHORT-NAME>real64</SHORT-NAME>
      <CATEGORY>FIXED_LENGTH</CATEGORY>
      <BASE-TYPE-SIZE>64</BASE-TYPE-SIZE>
      <BASE-TYPE-ENCODING>IEEE754</BASE-TYPE-ENCODING>
    </SW-BASE-TYPE>
  </ELEMENTS>
</AR-PACKAGE>
```

Figure 5.39. AUTOSAR *SwBaseType* Input block

These *baseTypes* will be referenced based on the <SHORT-NAME> of the <SW-BASE-TYPE> from the <MC-VARIABLE-INSTANCES> and from the <SW-RECORD-LAYOUT>s.

Based on the <BASE-TYPE-SIZE> and <BASE-TYPE-ENCODING>, the RTA-A2L *DataTypes* are created.

The following *DataTypes* are supported: *UBYTE*, *SBYTE*, *UWORD*, *SWORD*, *ULONG*, *SLONG*, *FLOAT32_IEEE*, *FLOAT64_IEEE*.

The following table shows mappings from AUTOSAR to RTA-A2L:

BASE-TYPE-SIZE(AR)	BASE-TYPE-ENCODING(AR)	DataType(A2L)
8	BOOLEAN	UBYTE
8	NONE	UBYTE
16	NONE	UWORD
32	NONE	ULONG
8	2C	SBYTE
16	2C	SWORD
32	2C	SLONG
32	IEEE754	FLOAT32_IEEE
64	IEEE754	FLOAT64_IEEE
64	NONE	A_UINT64
64	2C	A_INT64

Please refer to the *RTA-A2L Error Description* document for log entry details for the *DataType* Creation.

5.7.17 DataSizes for the A2L Blocks

The *DataSizes* are created using AUTOSAR's *SwBaseTypes*. The following snapshot represents the *SwBaseType* *STRUCTURE*:

```
<AR-PACKAGE>
  <SHORT-NAME>SwBaseTypes</SHORT-NAME>
  <ELEMENTS>
    <SW-BASE-TYPE>
      <SHORT-NAME>real64</SHORT-NAME>
      <CATEGORY>FIXED_LENGTH</CATEGORY>
      <BASE-TYPE-SIZE>64</BASE-TYPE-SIZE>
      <BASE-TYPE-ENCODING>IEEE754</BASE-TYPE-ENCODING>
    </SW-BASE-TYPE>
  </ELEMENTS>
</AR-PACKAGE>
```

Figure 5.40.STRUCTURE of the AUTOSAR Swbasetype

These *baseTypes* will be referenced based on the <SHORT-NAME> of the <SW-BASE-TYPE>, from the <MC-VARIABLE-INSTANCES> and also from the <SW-RECORD-LAYOUT>s.

Based on the <BASE-TYPE-SIZE> and <BASE-TYPE-ENCODING>, the RTA-A2L *DataSizes* are created.

The following *DataSizes* are supported: *BYTE, WORD, LONG*.

The following table shows mappings from AUTOSAR to RTA-A2L:

BASE-TYPE-SIZE(AR)	BASE-TYPE-ENCODING(AR)	DataType(A2L)
8	BOOLEAN	BYTE
8	NONE	BYTE
16	NONE	WORD
32	NONE	LONG
8	2C	BYTE
16	2C	WORD
32	2C	LONG
32	IEEE754	LONG
64	IEEE754	LONG

5.7.18 Conversion of String Literals

When string literals (e.g. *longName* of elements and *DisplayName* of *Unit* or *CompuMethod*) contain the characters " or \, then they are generated according to the ASAM specifications as \" and \\ respectively.

e.g.

Input ARXML file sample "long name" Output A2L file sample \"long name\"

5.7.19 ANNOTATION Block Details

RTA-A2L currently supports the *ANNOTATION* block with the *ANNOTATION_LABEL* and the *ANNOTATION_TEXT* details.

ANNOTATION blocks will be created using the input `<ANNOTATION>` and `<DESC>` blocks. The following figure shows the *STRUCTURE* of the Annotation in the *McDataInstance*.

Creation of *ANNOTATION* blocks using the input `<ANNOTATIONS>` blocks:

```
<ANNOTATIONS>
  <ANNOTATION>
    <LABEL>
      <L-4 L="FOR-ALL">This is the annotation for FrTrcv_DeviceSelection_C</L-4>
    </LABEL>
    <ANNOTATION-TEXT>
      <P>
        <L-1 L="AA">This is the annotation for ValueExample</L-1>
      </P>
    </ANNOTATION-TEXT>
  </ANNOTATION>
</ANNOTATIONS>
```

Figure 5.41. *STRUCTURE* of the *ANNOTATION* block in the *McSupport* File

In the list of Annotations supported in the `<MC-DATA-INSTANCE>`, all the `<ANNOTATION>` blocks will be converted into A2L *ANNOTATION* blocks.

The language type will not be considered in creating the *ANNOTATION* blocks using input `<ANNOTATION>` blocks.

The `<LABEL>` contents will be used to create the *ANNOTATION_LABEL* in the A2L file. Both the *MultiLanguage longName* and the list of *longNames* will be taken from the first Annotation, with the first *longName* used for the label creation.

The `<ANNOTATION-TEXT>` will be used to create the *ANNOTATION_TEXT* in the A2L file. In the list of *MultiLanguageParagraph (<P>)* tags, the first *MultiLanguageParagraph* will be used. In the list of *LParagraphs* fetched from the *MultiLanguageParagraph*, the first *LParagraph* will be used for the *ANNOTATION_TEXT* creation in the A2L file. The A2L *ANNOTATION* block will appear as below:

```
/begin ANNOTATION
  ANNOTATION_LABEL "This is the annotation for FrTrcv_DeviceSelection_C"
  /begin ANNOTATION_TEXT
    "This is the annotation for ValueExample"
  /end ANNOTATION_TEXT
/end ANNOTATION
```

Figure 5.42. *STRUCTURE* of the *ANNOTATION* block in the A2L file in the *MEASUREMENT / CHARACTERISTIC* block

Creation of the *ANNOTATION* blocks using the input `<DESC>` block

For the *CHARACTERISTIC*, *MEASUREMENT*, *AXIS_PTS* and *FUNCTION* blocks, RTA-A2L uses the input <DESC> block to create the *ANNOTATION* blocks in the A2L file.

For other A2L blocks (e.g. *COMPU_METHOD*, *Unit*, *RECORD_LAYOUT*) the input <DESC> tags will be ignored, since A2L *ANNOTATION* blocks do not appear for these blocks.

The input <DESC> block contents will be fetched based on the input language type and will be used as the new *ANNOTATION_TEXT* contents. Each line will be appended with the "rn" characters.

If the specified language <DESC> content is not present, but the *FOR-ALL* type is present, then the *FOR-ALL* type will be provided as preference. If both the specified language type and the *FOR-ALL* type are not present, but the *EN* type is present, then *EN* type will be given preference.

By default, the *ANNOTATION_LABEL* will be set as "Description (coming from AUTOSAR DESC field)" e.g. if the <DESC> block provided in the input is as below, and the language type is *DE* then:

Input:

```
<DESC>
<L-2 L="EN">Consigne Soulagement Puissance Compresseur.
1 = normal 2 = deb ou soulagement maximal pour les clim non dd.
= Figeage
= cylindre mini 5% </L-2>
</DESC>
```

Output:

```
/begin ANNOTATION
  ANNOTATION_LABEL "Description (coming from AUTOSAR DESC field)"
  /begin ANNOTATION_TEXT
    "Consigne Soulagement Puissance\r\n"
    "Compresseur.\r\n"
    "1 = normal\r\n"
    "2 = deb ou soulagement maximal pour les clim non dd.\r\n"
    "3 = Figeage\r\n"
    "4 = cylindre mini 5%\r\n"
  /end ANNOTATION_TEXT
/end ANNOTATION
```

Creation of ANNOTATION blocks for the ARRAY elements (Special Behavior)

In the case of *ARRAY* elements, if the *subElement* doesn't contain either the <DESC> or <ANNOTATION> contents, then the contents will be fetched from its immediate *ARRAY Parent* element, if it exists. Or else, in the case of multi-dimensional *ARRAYs*, the iteration will traverse back to the next level *ARRAY* element.

If the *Parent* element is *STRUCTURE*, then the iteration will be stopped, and no values considered.


```

<MC-DATA-INSTANCE>
  <SHORT-NAME>CoEngStArray</SHORT-NAME>
  <LONG-NAME>
    <L-4 L="FOR-ALL">Longname for CoEngStArray</L-4>
  </LONG-NAME>
  <CATEGORY>ARRAY</CATEGORY>
  <ARRAY-SIZE>3</ARRAY-SIZE>
  ...
  <SUB-ELEMENTS>
    <MC-DATA-INSTANCE>
      <SHORT-NAME>element_0</SHORT-NAME>
      <LONG-NAME>
        <L-4 L="FOR-ALL">Longname for element_0</L-4>
      </LONG-NAME>
      <CATEGORY>ARRAY</CATEGORY>
      <ARRAY-SIZE>2</ARRAY-SIZE>
      ...
      <SUB-ELEMENTS>
        <MC-DATA-INSTANCE>
          <SHORT-NAME>element_0_value</SHORT-NAME>
          <CATEGORY>VALUE</CATEGORY>
          ...
        </SUB-ELEMENTS>
      ...
    </SUB-ELEMENTS>
  </MC-DATA-INSTANCE>

```

Figure 5.43. *DESC* and *ANNOTATION* blocks for the *ARRAY* elements

In the above case, the <DESC> for the *element_0_value* will be from the CoEngStArray block. i.e. DESC = "Desc of CoEngStArray".

The <ANNOTATION>s are from the *element_0* block:

ANNOTATIONS="This is the annotation for element_0_value"
--

In the case of *ARRAY-STRUCTURE-VALUE*, if the *VALUE* element does not contain the <DESC> or the <ANNOTATION> tags, then the immediate *Parent* element will be checked. Since the immediate *Parent* of *VALUE* is type *STRUCTURE*, then the backwards iteration will be stopped and the *STRUCTURE* contents will not be considered:

```

<MC-DATA-INSTANCE>
  <SHORT-NAME>CoEngStArray</SHORT-NAME>
  <LONG-NAME>
    <L-4 L="FOR-ALL">Longname for CoEngStArray</L-4>
  </LONG-NAME>
  <CATEGORY>ARRAY</CATEGORY>
  <ARRAY-SIZE>3</ARRAY-SIZE>
  ...
  <SUB-ELEMENTS>
    <MC-DATA-INSTANCE>
      <SHORT-NAME>element_0</SHORT-NAME>
      <LONG-NAME>
        <L-4 L="FOR-ALL">Longname for element_0</L-4>
      </LONG-NAME>
      <CATEGORY>STRUCTURE</CATEGORY>
      ...
      <SUB-ELEMENTS>
        <MC-DATA-INSTANCE>
          <SHORT-NAME>EngN</SHORT-NAME>
          <CATEGORY>VALUE</CATEGORY>
          ...
        </SUB-ELEMENTS>
      ...
    </SUB-ELEMENTS>
  </MC-DATA-INSTANCE>

```

Figure 5.44. ANNOTATION generations for ARRAY-STRUCTURE-VALUE example

In the above example, the *VALUE* block *EngN* does not have `<DESC>` or the *ANNOTATION* blocks, hence its immediate *Parent element_0* will be considered. Since it is of type *STRUCTURE*, no values will be considered from *element_0* and the iteration will be stopped. For *EngN*, no *ANNOTATION* blocks will be generated.

5.7.20 MAX_REFRESH Block Details

The *MAX_REFRESH* block will appear in the A2L file as *MAX_REFRESH ScalingUnit Rate*.

To create *MAX_REFRESH* for the *CHARACTERISTIC* or *MEASUREMENT* block, we reference the content of `<SW-REFRESH-TIMING>` in the `<MC-DATA-INSTANCE>`. *MAX_REFRESH* will appear for the *CHARACTERISTIC* or *MEASUREMENT* block, only if `<SW-REFRESH-TIMING>` exists.

From `<SW-REFRESH-TIMING>`, we take the value of `<CSE-CODE>` and map it to the *ScalingUnit*.

The value of `<CSE-CODE-FACTOR>` will be mapped to *Rate* of the *MAX_REFRESH*.

5.7.21 Format Details

The `<DISPLAY-FORMAT>` is the input for the *Format* value creation in the input File.

The *displayFormats* *%dddd.ddf*, *%dddd.ddF* and *%dddd.dd* are supported by

RTA-A2L.

If the *displayFormat* ends with *f/F*, then the *lastIndex f/F* will be truncated and the remaining *displayFormat* will be set as the format value.

Or else, if no *Format* specifier has been specified, then the *displayFormat* will also be set to the provided value.

If the *Format* ends other than *f*, then:

- For the *VALUE* type of *MEASUREMENT*, no *Format* will be set and the *INFO* message will be displayed.
- For the *VALUE*, *CURVE* and *MAP* types of *CHARACTERISTICS*, the default value *%10.5* will be set.
- For the *AXIS_DESCR* block, the default value *%10.5* will be set.
- For the *COMPU_METHOD*, the default value *%10.5* will be set.

If the format has only digits, and doesn't end with any specifiers, then:

- If the <DISPLAY-FORMAT> has only digits and no *FORMAT* specifier, it is of type *%dddd.dd %.dd %dddd*.
- For the *VALUE*, *VAL_BLK*, *CURVE* and *MAP* types of *CHARACTERISTIC*, the value will be set as given in the input.
- For the *AXIS_DESCR* block and the *COMPU_METHOD*, the value will be set as given in the input.
- For the *AXIS_PTS*, the value will be set as given in the input.
- For the *VALUE* type of *MEASUREMENT*, the value will be set as given in the input.

If the format ends with specifiers other than those that are AUTOSAR valid, then:

- The *displayFormat* ends with other than AUTOSAR *FORMAT* specifiers or *f/F*.
- For the *VALUE*, *VAL_BLK*, *CURVE* and the *MAP* types of *CHARACTERISTIC*, the default value *%10.5* will be set.
- For the *AXIS_DESCR* block, the default value *%10.5* will be set for the *AXIS_PTS*.
- For the *VALUE* type of *MEASUREMENT*, the default value *%10.5* will be set.
- For the *COMPU_METHOD*, the default value *%10.5* will be set.

If no <DISPLAY-FORMAT> occurs in the input file:

- For the *VALUE* type of *MEASUREMENT*, no *FORMAT* will be created.
- For the *VALUE*, *CURVE* and the *MAP* types of *CHARACTERISTICS*, the default value *%10.5* will be set.
- For the *AXIS_DESCR* block, the default value *%10.5* will be set.
- For the *COMPU_METHOD*, the default value *%10.5* will be set.

5.7.22 Limits Details

The <DATA-CONSTR> block referenced from <MC-DATA-INSTANCE>/<SW-CALPRM-AXIS> will be used as the input for the creation of the *LowerLimit* and *UpperLimit* for the *CHARACTERISTIC/MEASUREMENT/AXIS_DESCR* block.

If no <DATA-CONSTR-REF> is in the <MC-DATA-INSTANCE>/<SW-CALPRM-AXIS>, and the <BASE-TYPE-REF> of the <SW-CALPRM-AXIS> is other than *FLOAT*, then the *Limits* will be assigned with the default *Limit* value, based on the *baseType* (i.e. the *DataType* of the *baseType*). Please refer to Section 5.7.17 for more details.

When the <DATA-CONSTR> object exists:

- The <DATA-CONSTR> might contain many <DATA-CONSTR-RULES>, but currently RTA-A2L references the first <DATA-CONSTR-RULE>.
- The <DATA-CONSTR-RULE> might contain any of <PHYS-CONSTRS>/<INTERNAL-CONSTRS>, or both, but only the <PHYS-CONSTRS> will be considered by RTA-A2L.
- If no <PHYS-CONSTRS> exists, then that <DATA-CONSTR> block will not be considered for the *Limits* generation.

The figure below shows the *STRUCTURE* of the <DATA-CONSTR> in the *McDataInstance*:

```
<DATA-CONSTR>
  <SHORT-NAME>FlxrXcvr_DeviceSelection_C_CNS</SHORT-NAME>
  <DATA-CONSTR-RULES>
    <DATA-CONSTR-RULE>
      <PHYS-CONSTRS>
        <LOWER-LIMIT INTERVAL-TYPE="CLOSED">0xAB</LOWER-LIMIT>
        <UPPER-LIMIT INTERVAL-TYPE="CLOSED">0xFF</UPPER-LIMIT>
      </PHYS-CONSTRS>
    </DATA-CONSTR-RULE>
    <DATA-CONSTR-RULE>
      <PHYS-CONSTRS>
        <LOWER-LIMIT INTERVAL-TYPE="CLOSED">0</LOWER-LIMIT>
        <UPPER-LIMIT INTERVAL-TYPE="CLOSED">10</UPPER-LIMIT>
      </PHYS-CONSTRS>
    </DATA-CONSTR-RULE>
    <DATA-CONSTR-RULE>
      <PHYS-CONSTRS>
        <LOWER-LIMIT INTERVAL-TYPE="CLOSED">0</LOWER-LIMIT>
        <UPPER-LIMIT INTERVAL-TYPE="CLOSED">0</UPPER-LIMIT>
      </PHYS-CONSTRS>
    </DATA-CONSTR-RULE>
    <DATA-CONSTR-RULE>
      <PHYS-CONSTRS>
        <LOWER-LIMIT INTERVAL-TYPE="CLOSED">0xAB</LOWER-LIMIT>
        <UPPER-LIMIT INTERVAL-TYPE="CLOSED">0xFF</UPPER-LIMIT>
      </PHYS-CONSTRS>
    </DATA-CONSTR-RULE>
  </DATA-CONSTR-RULES>
</DATA-CONSTR>
```

Figure 5.45. *STRUCTURE* of the *DataConstr* block in the *McSupport* File

- From the <PHYS-CONSTRS>, the <LOWER-LIMIT> value will be fetched and mapped to the *CHARACTERISTIC* or *MEASUREMENT LowerLimit*. If the <LOWER-LIMIT> value is Hexadecimal, then it will be converted to Decimal and assigned.
- From the <PHYS-CONSTRS>, the <UPPER-LIMIT> value will be fetched and mapped to the *CHARACTERISTIC* or *MEASUREMENT UpperLimit*. If the <UPPER-LIMIT> value is Hexadecimal, then it will be converted to Decimal and assigned.

When the <DATA-CONSTR> object does not exist:

- Get the *DataType* of the corresponding *baseType* (please refer to Section 5.7.16 for more details). Based on the *DataType* the following default *LowerLimit* and *UpperLimit* values will be set:

DataType(A2L)	Default LowerLimit	Default UpperLimit
UBYTE	0	255
UWORD	0	65535
ULONG	0	4294967295L
SBYTE	0	127
SWORD	-32768	32767
SLONG	-214783647L - 1	214783647L
FLOAT32_IEEE	-3.4028235E38	3.4028235E38
FLOAT64_IEEE	-1.7976931E308	1.7976931E308

Generation of A2L Limits based on the input display format:

- The *UpperLimit* and *LowerLimit* are generated in the A2L file based on the *displayFormat* (e.g. if *displayFormat* = 10.5, 5 is given priority).
- The decimal part of the *displayFormat* is given priority for the generation.
- The number of digits after the decimal in the A2L Limit should be equal to the value of the decimal part of the display format.
- If the number of digits in the A2L Limit after the decimal are less than the required format, 0s are added to the decimal part. If the number of digits is greater than the required format, the number is rounded-up to match the format.
- If the A2L Limit is an exponential number, the format is applied to the non--exponential part.

e.g.

Input limit = 3.2 Display format = 8.5 Generated limit = 3.20000 Input limit = 12345.5 Display format = 10.0 Generated limit = 12346.

Exponential numbers: Input limit = 1.4E20 Display format = 8.3 Generated limit = 1.400E20 Input limit = 1.526E12 Display format = 10.2 Generated limit = 1.53E12

Assumptions: The decimal part of the display format is a single-digit number.

Please refer to the *RTA-A2L Error Description* document for the log entry details for the *DataConstr* validations.

Support of infinite range for FLOAT data types:

- If infinite range is found in the input *ARXML* file, then the default limits are generated for *FLOAT32* and *FLOAT64*.
- For *FLOAT32*, $-3.4028235E38$ and $3.4028235E38$ will be generated as the *LowerLimit* and *UpperLimit*.
- For *FLOAT64*, $-1.7976931E308$ and $1.7976931E308$ will be generated as the *LowerLimit* and *UpperLimit*.
- *CHARACTERISTICS* or *MEASUREMENTS*, which are referring to infinite range, will be generated with the above values.

e.g.

```
<PHYS-CONSTRS>
  <LOWER-LIMIT INTERVAL-TYPE="OPEN">0</LOWER-LIMIT>
  <UPPER-LIMIT INTERVAL-TYPE="INFINITE">INF</UPPER-LIMIT>
</PHYS-CONSTRS>
```

In the output A2L file, the *LowerLimit* will be generated as *0* and *UpperLimit* as $3.4028235E38$, since the *baseType* is *FLOAT32*.

5.7.23 System Condition/Expression Error Message

System conditions and binding expressions will be resolved by RTE Generation when the binding time is set to *CodeGeneration*. In this case, they will not appear in *McSupport*. But system conditions and binding expressions with *BindingTime PreCompile* (or later) may occur, and it will be a later task of RTA-A2L to resolve them.

But the current behavior of RTA-A2L cannot resolve the system conditions and expressions; an error message is thrown instead and the object affected - together with those that reference it - will not be written to the A2L file. System conditions and expressions can occur in different kinds of variation points e.g. Limit variation points, Numerical value variation points.

Please refer to the *RTA-A2L Error Description* document for the list of error messages thrown for system conditions and expressions.

5.7.24 ETK, XCP ifData Blocks Creation

The *<SW-REFRESH-TIMING>*, *CSECODE* and the *CSECODEFACTOR* values are used for the *IfData* blocks creation for the *CHARACTERISTICS*, *MEASUREMENTS* and *AXIS_PTS* blocks.

Currently, *FTL*, which is a sample template, needs to be used for the *IF_DATA* blocks creation.

FTL Templates are used to generate the *IF_DATA* blocks for *CHARACTERISTICS* and *MEASUREMENTS*.

5.7.25 Virtual MEASUREMENT Block Details

RTA-A2L generates virtual *MEASUREMENTs* only when `<SW-IS-VIRTUAL>1</SW-IS-VIRTUAL>` and `<SW-DATA-DEPENDENCY>` are present in the `<MC--DATA-INSTANCE>`.

For each virtual *MEASUREMENT* block, there would be a *FORM CompuMethod* generated. (Please refer to Section 5.7.6.2 for more details.)

If `<SW-IS-VIRTUAL>1</SW-IS-VIRTUAL>` and `<SW-DATA-DEPENDENCY>` are not present, then RTA-A2L will throw a warning message and the *McDataInstance* block would not be considered for virtual *MEASUREMENT* generation.

If `<SW-DATA-DEPENDENCY-FORMULA>` variables are not matching with `<SW--DATA-DEPENDENCY-ARGS>`, RTA-A2L would not do any validation and proceed with the generation of A2L.

If `<SW-DATA-DEPENDENCY-FORMULA>` is null/empty and *McDataInstance-VarRef* is present, then the *McDataInstance* will not be considered for virtual *MEASUREMENTs*.

If the `<SW-DATA-DEPENDENCY-FORMULA>` mentioned is correct, but one of the *McDataInstanceVarRefs* isn't present in the input ARXML file, then the *McDataInstance* will not be considered for virtual *MEASUREMENTs*.

The `<COMPU-METHOD-REF>` that is present in the *McDataInstance* is completely ignored.

The *McDataInstance* that is referenced to inside the *McDataInstanceVarRef* should be of type *VALUE* and its `<SW-CALIBRATION-ACCESS>` value should not be *NOT-ACCESSIBLE*. If the `<SW-CALIBRATION-ACCESS>` is *NOT-ACCESSIBLE*, then the *McDataInstance* will not be considered for virtual measurement generation.



NOTE

The ADX entry will not be generated in the *mapping.adx* file for virtual *MEASUREMENTs*. The *ECU_ADDRESS* tag will not be generated for virtual *MEASUREMENTs*.

5.8 Generation of a Mapping ADX File

Along with the A2L File generation, RTA-A2L generates a mapping ADX file if its path is supplied as a value to the argument `--pathToMapAdxFile` in the options file.

The input for the ADX entry will be the *McDataInstance*. Based on its category, the name will be updated and the number of ADX entries will differ.

If the `<SW-CALIBRATION-ACCESS>` is of type *NOT-ACCESSIBLE* or `<SW-CALIBRATION-ACCESS>` is unavailable for the `<MC-DATA-INSTANCE>`, then no ADX entries will be generated for that entry.

If the `<SW-CALIBRATION-ACCESS>` of the *Parent* element of the *STRUCTURE* or *ARRAY* element is of type *NOT-ACCESSIBLE*, or `<SW-CALIBRATION-ACCESS>` is

unavailable, then no ADX entries will be generated, either for the *Parent* or the child elements.

If only the child element is of type *NOT-ACCESSIBLE* or *<SW-CALIBRATION-ACCESS>* is unavailable then, only for that element, no ADX entries will be generated.

If the *<SHORT-NAME>* and *<SYMBOL>* information of the *McDataInstances* are the same (1:1), then no ADX entry will be generated by RTA-A2L. (In AUTOSAR, it is not possible to define C-file static *MEASUREMENTS*. Therefore, RTA-A2L is not able to generate prefix information for *LABEL-NAME*).

5.8.1 ADX Entries for VALUE McDataInstance

If the category of the *McDataInstance* is *VALUE/VAL_BLK/CURVE/MAP/STRING*, then a single *<MEMORY-ELEMENT>* entry will be created in the ADX file.

The *<MEMORY-ELEMENT>* entry for the *McDataInstance* will be created as below:

```
<MAP-LABEL-NAME>: Value of the <SHORT-NAME> of the <MC-DATA-INSTANCE>
<LABEL-NAME>: Value of the <SYMBOL>.
```

If the *McDataInstance* does not contain any *<SHORT-NAME>* or *<SYMBOL>* information, then no ADX entry will be created, and messages will be logged in the log file.

Please refer to the *RTA-A2L Error Description* document for the list of log entries supported for *VALUE McDataInstance*.

5.8.2 ADX Entries for STRUCTURAL McDataInstance

If the category of the *Variable McDataInstance/Parameter McDataInstance* is *STRUCTURE*, then the number of *<MEMORY-ELEMENT>* entries in the ADX file is equal to the one entry for all the *Parent* elements plus the number of entries of the *subElement*.

The *<MEMORY-ELEMENT>* entry for the *Parent McDataInstance* will be created as below:

```
<MAP-LABEL-NAME>: Value of the <SHORT-NAME> of the Parent element
<LABEL-NAME>: Value of the <SYMBOL>
```

Based on the category of each *subElement*, the number of *<MAP-LABEL-NAME>*s is created.

If the category is of type VALUE/VAL_BLK, the following conditions are used to set the key-value pairs:

- *<MAP-LABEL-NAME>*: The value of the *<SHORT-NAME>* of the *Parent McDataInstance*.
- *<LABEL-NAME>*: The value of the *<SYMBOL>* of *STRUCTURE*. The value of the *<SHORT-NAME>* of the *subElement*.

If the category is of type STRUCTURE:

The *<SHORT-NAME>*s will be looped until we get the *VALUE* category of the *McDataInstance* as the child element.

- <MAP-LABEL-NAME>: The value of the <SHORT-NAME> of the *Parent McDataInstance*. The value of the <SHORT-NAME> of the second *Parent McDataInstance*. The <SHORT-NAME> of the *subElement*.
- <LABEL-NAME>: The value of the <SYMBOL> of *STRUCTURE*. The value of the <SHORT-NAME> of the second *Parent McDataInstance*. The value of the <SHORT-NAME> of the *subElement*.

If the category is of type ARRAY:

In this case, based on the value of the <ARRAY_SIZE> of the *McDataInstance*, the <MEMORY-ELEMENT> blocks will be created.

- The <MEMORY-ELEMENT> entry will be created for the first child element of the *ARRAY* element.
- <MAP-LABEL-NAME>: The value of the <SHORT-NAME> of the *Parent McDataInstance*. The value of the <SHORT-NAME> of the *ARRAY Parent McDataInstance[i]*, where $i = 0$ to $(ARRAY_SIZE-1)$.
- <LABEL-NAME>: The value of the <SYMBOL> of *STRUCTURE*. The value of the <SHORT-NAME> of the *ARRAY Parent McDataInstance[i]*, where $i = 0$ to $(ARRAY_SIZE-1)$.
- If the *McDataInstance* does not contain any <SHORT-NAME> or <SYMBOL> information, then no ADX entry will be created and messages will be logged in the log file.
- If the *subElement McDataInstance* of the structural element does not contain the <SHORT-NAME>, then no ADX entry will be created for that particular *subElement*.

The log entry details created for the ADX file:

The validation checks are done for *ShortName* and *CSymbol*, independent of the *configSwitch*.

Please refer to the *RTA-A2L Error Description* document for the list of log entries supported for *Structure McDataInstance*.

5.8.3 ADX Entries for ARRAY McDataInstance

If the category of the *Variable McDataInstance/Parameter McDataInstance* is *ARRAY*, then the number of <MEMORY-ELEMENT> entries in the ADX file is equal to the one entry for all the *Parent* elements plus the number of entries of the *VALUE/VAL_BLK/CURVE/MAP subElement*.

The <MEMORY-ELEMENT> entry for the *Parent McDataInstance* will be created as follows:

```
<MAP-LABEL-NAME>: value of the <SHORT-NAME> of the Parent element
<LABEL-NAME>: value of the <SYMBOL>
```

Based on the category of each *subElement*, the number of <MAP-LABEL-NAME>s is created.

If the category is VALUE/VAL_BLK/CURVE/MAP, the following conditions are

used to set the key-value pairs:

- **<MAP-LABEL-NAME>**: The value of the **<SHORT-NAME>** of the *Parent McDataInstance[i]*, where $i = 0$ to $(ARRAY_SIZE - 1)$.
- **<LABEL-NAME>**: The value of the **<SYMBOL>** of *STRUCTURE[i]*, where $i = 0$ to $(ARRAY_SIZE - 1)$.

If the category is STRUCTURE:

In this case, the **<SHORT-NAME>**s will be looped, until we get the *VALUE/CURVE /MAP* type of *McDataInstance* as the child element.

- **<MAP-LABEL-NAME>**: The value of the **<SHORT-NAME>** of the *Parent McDataInstance[i]*. The **<SHORT-NAME>** of the value type *subElement*, where $i = 0$ to $(ARRAY_SIZE - 1)$.
- **<LABEL-NAME>**: The value of the **<SYMBOL>** of *STRUCTURE[i]*. The value of the **<SHORT-NAME>** of the *VALUE* type *subElement*, where $i = 0$ to $(ARRAY_SIZE - 1)$.

If the category is of type ARRAY:

In this case, based on the value of the *ARRAY_SIZE* of the *McDataInstance*, the **<MEMORY-ELEMENT>** blocks will be created.

- The **<MEMORY-ELEMENT>** entry will be created for the first child element of the *ARRAY* element.
- **<MAP-LABEL-NAME>**: The value of the **<SHORT-NAME>** of the *Parent McDataInstance[i][j]*, where $i = 0$ to *1st Parent* $(ARRAY_SIZE - 1)$, $j = 0$ to *2nd Parent* $(ARRAY_SIZE - 1)$.
- **<LABEL-NAME>**: The value of the **<SYMBOL>** of the *McDataInstance[i][j]*, where $i = 0$ to *1st Parent* $(ARRAY_SIZE - 1)$, $j = 0$ to *2nd Parent* $(ARRAY_SIZE - 1)$.
- If the *McDataInstance* does not contain any **<SHORT-NAME>** or **<SYMBOL>** information, then no ADX entry will be created, and messages will be logged in the log file.
- If the *SubMcDataInstance* of the *Structure* element does not contain the **<SHORT-NAME>**, then no ADX entry will be created for that particular *subElement*.

(Please refer to Section 5.6.3 for more details.)

The log entry details for the ADX file:

The validation checks are done for the *ShortName* and the *CSymbol*, independent of the *configSwitch*.

Please refer to the *RTA-A2L Error Description* document for the list of log entries supported for *Structure McDataInstance*.

5.8.4 ADX Entry for CHARACTERISTICS/MEASUREMENTS with BIT_MASK

The ADX Entry *STRUCTURE* is represented as follows:

```

<MAP-LABEL-NAME>: The value of the <SHORT-NAME> of the <MC-DATA-INSTANCE>.
<LABEL-NAME>: If the HOST-VARIABLE is present, then the value of the <SYMBOL> of <HOST-VARIABLE>. If
the HOST-VARIABLE is not present, then the value of the <SYMBOL> of current <MC-DATA-INSTANCE>.
<BIT-POSITION>: The value of the tag <BIT-POSITION> of the <MC-DATA-INSTANCE>.
<NUMBER-OF-BITS>: The value of the tag <NUMBER-OF-BITS> of the <MC-DATA-INSTANCE>.

```

5.8.5 ADX Entry for Static AUTOSAR Elements

- If the <ADMIN-DATA> block of <MC-DATA-INSTANCE> contains the following <SDG> tag, then the <MC-DATA-INSTANCE> or the element is *Static*:

```

<SDG GID="STATIC-MEMORY">
  <SD GID="DATA-SOURCE">filename</SD>
</SDG>

```

- The ADX Entry for *Static* element will be generated as follows:

```

<MAP-LABEL-NAME>: shortName of <MC-DATA-INSTANCE>
<LABEL-NAME> : adminData.dataSource_c%symbol of <MC-DATA-INSTANCE>

```

- For the complex datatypes *Structures* and *Arrays*, the ADX entries will be generated as per Section 5.8.3 and Section 5.8.2. From the above example, the *DATA-SOURCE* value mentioned in the <SD> tag of <ADMIN-DATA> *dataSource_c%* will be added as a prefix for <LABEL-NAME>.

e.g.

If the Static Element is of Type Primitive Data Type:

```

<MC-DATA-INSTANCE>
  <SHORT-NAME>element_0</SHORT-NAME>
  <CATEGORY>VALUE</CATEGORY>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="STATIC-MEMORY">
        <SD GID="DATA-SOURCE">filename</SD>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
  <SYMBOL>element_0_leafElement</SYMBOL>
</MC-DATA-INSTANCE>

```

The ADX Entry will be generated as below:

```

<MAP-LABEL-NAME>: element_0
<LABEL-NAME>: filename_c%element_0_leafElement

```

e.g.

If the Static Element is of Type STRUCTURE:

```

<MC-DATA-INSTANCE>
  <SHORT-NAME>CanIf_ControllerState</SHORT-NAME>
  <CATEGORY>STRUCTURE</CATEGORY>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="STATIC-MEMORY">
        <SD GID="DATA-SOURCE">filename</SD>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
  <SUB-ELEMENTS>
    <MC-DATA-INSTANCE>
      <SHORT-NAME>CanIf_ControllerModeType</SHORT-NAME>

```

```

<CATEGORY>VALUE</CATEGORY>
<SYMBOL>FirstChildElementSymbol</SYMBOL>
</MC-DATA-INSTANCE>
</SUB-ELEMENT>
<SYMBOL>CanIf_ControllerStateSymbol</SYMBOL>
</MC-DATA-INSTANCE>

```

The ADX Entry will be generated as below:

<MAP-LABEL-NAME> of parentMcDataInstance: CanIf_ControllerState
 <LABEL-NAME> of parentMcDataInstance: filename_c%CanIf_Controller-
 StateSymbol <MAP-LABEL-NAME> of childMcDataInstance: CanIf_Con-
 trollerModeType <LABEL-NAME> of childMcDataInstance: filename_c%-
 FirstChildElementSymbol

e.g.

If the Static element is of Type ARRAY:

```

<MC-DATA-INSTANCE>
  <SHORT-NAME>EpmNEngArray_Tina</SHORT-NAME>
  <CATEGORY>ARRAY</CATEGORY>
  <ADMIN-DATA>
    <SDGS><SDG GID="STATIC-MEMORY">
      <SD GID="DATA-SOURCE">filename</SD>
    </SDG></SDGS>
  </ADMIN-DATA>
  <SUB-ELEMENTS>
    <MC-DATA-INSTANCE>
      <SHORT-NAME>element_0</SHORT-NAME>
      <CATEGORY>VALUE</CATEGORY>
      <SYMBOL>element_0_leafElement</SYMBOL>
    </MC-DATA-INSTANCE>
  </SUB-ELEMENTS>
  <SYMBOL>EpmNEngArray_Tina_Symbol</SYMBOL>
</MC-DATA-INSTANCE>

```

The ADX Entry will be generated as below:

```

<MAP-LABEL-NAME>:EpmNEngArray_Tina
<LABEL-NAME>: filename_c%EpmNEngArray_Tina_Symbol

```

5.9 Restrictions

5.9.1 Input MC-Support Files

If there are any well-formedness errors present in the input *McSupport* file, then the RTA-A2L Generator will abort and not generate the A2L or ADX files.

If the *McSupport* file is well-formed but not valid as per the specified AUTOSAR schema (e.g. if some mandatory attributes are missing) then an error message is logged and RTA-A2L continues processing the files.



NOTE

RTA-A2L generates only a single module with name *AR* and an empty long identifier "".

5.9.2 Short Name

It has been agreed by the RTE Generator that the *SHORT-NAME* of the *MC-DATA-INSTANCE* will be unique i.e. no two *MC-DATA-INSTANCES*, either in the

same or different packages, can have the same *SHORT-NAME*.

In case of elements other than *MC-DATA-INSTANCE* (e.g. *SWCOMPU-METHOD*, *BASE-TYPE*, *Unit*, *SW-SYSTEMCONST* etc.) there can exist multiple definitions of the elements with the same *SHORT-NAME*, but in different *AR-PACKAGES*.

5.9.3 RTA-A2L Behavior

If *MC-DATA-INSTANCE* elements are present with the same *SHORT-NAME*, then an error message is logged stating that multiple definitions of the element (with the same *SHORT-NAME*) are present in different packages. The first encountered element will then be considered for further processing.

If elements other than *MC-DATA-INSTANCES* are present in different packages with the same *SHORT-NAME*, then RTA-A2L will consider these duplicate elements; it will generate these blocks in the A2L file with unique names and update the references accordingly.

5.9.4 Multi-Language Support

Multi-language support is applicable for *LONG-NAME* tags in the *MC-Support* file.

RTA-A2L will generate the *LongIdentifier* based on the language specified by the `--language` option.

If no language has been specified, then *EN* will be considered as the default language.

If the `--language`-specified language type is not present, but a *FOR-ALL* type is, then the latter will be given preference.

If the `--language`-type specified in the options file and the *FOR-ALL* type is not present, but the default language type *EN* is, then *EN* will be given preference.

If the options file does not specify a language, i.e. neither the *FOR-ALL* or *EN* types exist, then "" (empty string) will be generated.

e.g.

Case1 If `--language DE`

```
<LONG-NAME>
<L-4 L="FOR-ALL">FOR-ALL Longname for Unit2</L-4>
<L-4 L="EN">EN Longname for Unit2</L-4>
</LONG-NAME>
LongIdentifier = FOR-ALL Longname for Unit2
```

Case2 If `--language IT`

```
<LONG-NAME>
<L-4 L="DE">DE Longname for Unit2</L-4>
<L-4 L="EN">EN Longname for Unit2</L-4>
</LONG-NAME>
LongIdentifier = EN Longname for Unit2 EN
```

5.9.5 Values

RTA-A2L supports decimal values, hex values and exponential values. If hex values are present in the *MC-Support* file, they are converted to decimal in the generated A2L file. In the *MC-Support* file, RTA-A2L does not support a value

having expressions with *SW-SYSTEMCONSTs*.

5.9.6 Handling of Invalid/Unsupported Contents in MC-Support blocks

If invalid/unsupported contents are encountered in the elements of input files, neither these elements nor the ones referencing them will be generated in the A2L file.

For every element in the *MC-Support* file, all tags must be mandatory as per the AUTOSAR specification. If mandatory tags are missing, then RTA-A2L will not be able to write the corresponding block in the A2L file.

5.9.7 Handling of SystemConditions

RTA-A2L is unable to resolve expressions with *SystemConditions*. The RTE Generator is run with a special switch to support the resolution of conditions and generate an *MC-Support* file without any system conditions.

Therefore, if the elements in the *MC-Support* file encounter expressions with *SystemConditions* in which the binding time is $\geq$ to *PRE_COMPILE*, then RTA-A2L will ignore the corresponding *MC-Support* elements, as well as the elements they reference. It will also log an error message.

5.9.8 ASAP2_VERSION

The generated A2L file will be of *ASAP2_VERSION 1.4*. There is a possibility to configure the *ASAP2_VERSION* for the A2L file to be generated by RTA-A2L, but this will affect only the *ASAP2_VERSION* keyword value, and generation of *STATIC_RECORD_LAYOUT* attributes (in case the generation of *StaticRecordLayouts* is enabled).

- If *ASAP2_VERSION* is set as *1.60* or *1.61*, then the generated A2L file will contain an *ASAP2_VERSION* as *1.60* or *1.61* and all the *RecordLayouts* will contain the attribute *STATIC_RECORD_LAYOUT*.
- If the *ASAP2_VERSION* is set as *1.60* or *1.61* but the `--enableStatic*RecordLayouts` option is set as *n*, then no *STATIC_RECORD_LAYOUTs* will be generated in the A2L *RECORD_LAYOUTs*.

5.9.9 SYMBOL

In the *MC-Support* file, the *SYMBOL* tag is present inside *MC-DATA-INSTANCE*. This tag holds the value of *C-Symbol* for the corresponding *MC-DATA-INSTANCE*.

For a root *MC-DATA-INSTANCE*, it is mandatory for the *SYMBOL* tag to have a valid value.

In case of *subElements* of root *MC-DATA-INSTANCE*, RTA-A2L will derive their *SYMBOL* information from the root element. In *MC-Support*, the *STRUCTURE* of *MC-DATA-INSTANCE* and its *subElements* is identical to the C code generated by the RTE Generator.

If the *subElement* contains the *SYMBOL* information then, in the case of *STRUCTURES*, it will be used. In the case of *ARRAYs*, it will be ignored.

If the *SYMBOL* tag is missing from a root *MC-DATA-INSTANCE* then the RTA-A2L behavior will be as follows:

- RTA-A2L will log an error message stating that the *SYMBOL* information is missing for the root *MC-DATA-INSTANCE*.
- Mapping for the ADX entry will be missing for the corresponding *MC-DATA-INSTANCE* and its *subElements* in the *mapping.adx* file generated by RTA-A2L.
- The corresponding *MC-DATA-INSTANCE* equivalent block will be written in the *AR-A2L* snippet generated by RTA-A2L.

In this case, during A2L Merging of the functionality of the A2L template and the AUTOSAR A2L snippet file, there is a possibility that, for this A2L block, the address is not found in the ELF/ADX file. As a result, the A2L Merging functionality will abort.

5.9.10 MC-Support Blocks

BASE-TYPE

The type or *ShortName* of *BASE-TYPE* are not considered to derive *A2L-DataType*. *A2L-DataType* is derived from *BASE-TYPE-SIZE* and *BASE-TYPE-ENCODING*.

Only the following categories of *COMPU-METHODs* are supported by RTA-A2L:

<i>RAT_FUNC</i>	
<i>TEXTTABLE</i>	(like <i>TAB_VERB</i> in <i>MSR</i>)
<i>TAB_NOINTP</i>	
<i>IDENTICAL</i>	
<i>LINEAR</i>	
<i>TAB_INTP</i>	

For the following categories of *COMPU-METHOD*, *COMPU-SCALES* inside both *COMPU-PHYS-TO-INTERNAL* and *COMPU-INTERNAL-TO-PHYS* are supported:

RAT_FUNC, *LINEAR*

For the following categories of *COMPU-METHOD*, *COMPU-SCALES* inside *COMPU-PHYS-TO-INTERNAL* are only supported:

TAB_INTP, *TAB_NOINTP*

For following categories of *COMPU-METHOD*, *COMPU-SCALES* inside *COMPU-INTERNAL-TO-PHYS* are only supported:

TEXTTABLE

For following type of *COMPU-METHOD*, provided *COMPU-SCALES* are ignored. Instead, a default *COEFFS* will be generated:

IDENTICAL. i.e. *COEFFS* 0 1 0 0 0 1

For the *LINEAR* and *IDENTICAL* *COMPU-METHODs* in the *MC-Support* file, RTA-A2L will generate A2L *CompuMethod* blocks of type *RAT_FUNC*.

For the *DISPLAY-FORMAT* at the *COMPU-METHOD* level, only the following pattern is supported: *%<dddd>.<ddd>f*. In this case, the same content without suffix *f* is written for the *FORMAT* attribute.

In a case where another pattern is specified: the *FORMAT* attribute is generated with a default value of *%10.5*.

SW-RECORD-LAYOUT

Currently, *RECORD-LAYOUTS*, which can be used by *CHARACTERISTICS* of type *VALUE*, *COM_AXIS*, Simple *CURVE*, Simple *MAP*, Simple *CUBOID*, Simple *CUBE_4*, Simple *CUBE_5*, Grouped *CURVE* and Grouped *MAP* are supported.

Only in the case of the *GAPS* property is *BASE-TYPE-REF* mandatory in the *SW-RECORD-LAYOUT* definition. In case of other properties, if the *BASE-TYPE-REF* is present at the *RECORD_LAYOUT*, then it will be used. Or else, the *BASE-TYPE-REF* fetched from the *Calprm MC-DATA-INSTANCE* and the corresponding *CalprmAxis* of the *MC-DATA-INSTANCE* will be used.

In case an input *SwRecordLayout* references another *SwRecordLayout*, then the RTA-A2L will log an error message and ignore the *SwRecordLayout* and corresponding *McDataInstance* (except the *VALUE* category) which references this *SwRecordLayout*.

For the *<SW-RECORD-LAYOUT>* which is referenced from *Calprm<MC-DATA-INSTANCE>* of category *VALUE*, there should be at least one *Axis(V)*. Or else, the RTA-A2L will ignore that a block and *DefaultRecordLayout* will be generated and assigned to the corresponding *CHARACTERISTIC* block.

For a *Calprm<MC-DATA-INSTANCE>* of category Simple *CURVE*, at least two axes are required (*V, X/Y*).

For a *Calprm<MC-DATA-INSTANCE>* of category Simple *MAP*, at least three axes should be present (*V, X, Y*).

The *SW-RECORD-LAYOUT-GROUP* categories currently supported are *COLUMN_DIR* and *ROW_DIR*.

For a *Calprm<MC-DATA-INSTANCE>* of category Simple *CUBOID*, at least four axes should be present (*V, X, Y, Z*).

The *SW-RECORD-LAYOUT-GROUP* categories currently supported are *COLUMN_DIR* and *ROW_DIR*.

For a *Calprm<MC-DATA-INSTANCE>* of category Simple *CUBE_4*, at least five axes should be present (*V, X, Y, Z1, Z2*).

The *SW-RECORD-LAYOUT-GROUP* categories currently supported are *COLUMN_DIR* and *ROW_DIR*.

For a *Calprm<MC-DATA-INSTANCE>* of category Simple *CUBE_5*, at least six axes should be present (*V, X, Y, Z1, Z2, Z3*).

The *SW-RECORD-LAYOUT-GROUP* categories currently supported are *COLUMN_DIR* and *ROW_DIR*.

For a *Calprm<MC-DATA-INSTANCE>* of category Grouped *CURVE*, at least one axis should be present (*V*).

For a *Calprm*<*MC-DATA-INSTANCE*> of category Grouped *MAP*, at least one axis should present (*V*).

The *SW-RECORD-LAYOUT-GROUP* categories currently supported are *COLUMN_DIR* and *ROW_DIR*.

For a *Calprm*<*MC-DATA-INSTANCE*> of category *COM_AXIS*, at least one axis should present (*X/Y*).

For the <*SW-RECORD-LAYOUT-V*>, any one of the <*SW-RECORD-LAYOUT-GROUP-AXIS*> or <*SW-RECORD-LAYOUT-GROUP-INDEX*> types should be defined. Otherwise, <*SW-RECORD-LAYOUT-V*> is not considered for that <*SW-RECORD-LAYOUT*> in A2L file generation.

If a *Calprm*<*MC-DATA-INSTANCE*> references a *SW-RECORD-LAYOUT* *RecLayout*, with any one or many *baseTypes* invalid, then the *RecordLayout* is generated with the name *RecLayout_BaseTypes McDataInstance*, and the *baseTypes* are taken from *MC-DATA-INSTANCE*.

For a *Calprm*<*MC-DATA-INSTANCE*> of category Simple *CUBOID*, RTA-A2L will not support the keyword *MAP-LIST*.

Due to limited AUTOSAR interpolation methods, the size of AUTOSAR *Calprm* *Maps* cannot be reduced, even though they use *STATIC_RECORD_LAYOUTS*.

DATA-CONSTR

The RTE Generator, that generates the *MC-Support* file, will only contain *DATA-CONSTR* having *PHYS-CONSTRS*.

If a *DATA-CONSTR* having only *INTERNAL-CONSTRS* is encountered in the *MC-Support* file, neither it nor other blocks that reference it will be generated in the A2L file.

If a *DATA-CONSTR* has multiple *DATA-CONSTR-RULEs*, then RTA-A2L considers only the first *DATA-CONSTR-RULE*.

RTA-A2L doesn't support the generation of default limits for *float32* and *float64*. The upper limits for *float32* and *float64* are *3.40282347E+38* and *1.79769131486231570E+308* respectively. (It doesn't make sense to generate such huge values for calibration in the A2L file.)

e.g.

INCA *optimizes* the Axis scaling accordingly. This means the *MAX* is used by default for the upper value in the Y-axis of an oscilloscope. It might, therefore, be reasonable to describe the range (*MIN/MAX*) in respect of the physical, meaning an upper engine rotation speed of *3.402E+38* isn't sensible. A calibration engineer must adjust the range of an Axis every time he adds it to the OSCI.

SW-SYSTEMCONST

The *SW-SYSTEMCONST-VALUE* associated with a *SW-SYSTEMCONST* is directly written to the A2L file without applying any *SW-COMPU-METHOD*.

If the *SW-SYSTEMCONST* contains the *SW-CALIBRATION-ACCESS* value *NOT-ACCESSIBLE*, then that block will be ignored by the RTA-A2L.

MC-DATA-INSTANCE

MC-PARAMETER-INSTANCES

The following categories of *CalPrm* <MC-DATA-INSTANCE>s are supported:

VALUE	
VAL_BLK	
BOOLEAN	
CURVE	(Simple CURVE, Grouped CURVE)
MAP	(Simple MAP, Grouped MAP)
CUBOID	(Simple CUBOID)
CUBE_4	(Simple CUBE_4)
CUBE_5	(Simple CUBE_5)
COM_AXIS	
STRING	
STRUCTURE	(Having subElement of type ARRAY, VALUE, Simple CURVE, Simple MAP and COM_AXIS)
ARRAY	(Of element types VALUE, ARRAY, STRUCTURE, Simple CURVE, Simple MAP and COM_AXIS BASE-TYPE-REF)

- At VALUE and BOOLEAN *CalPrm*<MC-DATA-INSTANCE> level, it is mandatory to have a BASE-TYPE-REF to create the *DefaultRecordLayout*, if there is no RECORD-LAYOUT-REF.
- For the DISPLAY-FORMAT at *CalPrm* and *CalPrmAxis* level inside MC-DATA-INSTANCE, only the following pattern is supported: %<dddd>. <dddd>f. In a case where another pattern is specified, the same content without the f suffix is written for the FORMAT attribute for the corresponding block in the A2L file.
- In case of a *CalPrm* of type VALUE, Simple CURVE, Simple Map, Grouped CURVE, Grouped MAP, AXIS_PTS and *CalPrmAxis*, the FORMAT attribute is generated with a default value of %10.5.
- For the *CalPrm*<MC-DATA-INSTANCE> (except BOOLEAN) and SW-*CalPrm*-AXIS, DATA-CONSTR-REF is a mandatory attribute only in the case when BASE-TYPE-REF is referencing a FLOAT data type.
- Symbol information will not be available for the subElements of MC-DATA-INSTANCE.
- RTA-A2L will derive the Symbol information for the subElements by considering that, in the MC-Support file, the STRUCTURE of the MC-DATA-INSTANCE and its subElements is identical to the C code generated by RTE Generator.
- If the SW-CALIBRATION-ACCESS of the MC-DATA-INSTANCE is of type NOT-ACCESSIBLE, then that block will not be generated in the A2L or ADX files.
- In the cases of Simple CURVE, Simple MAP and AXIS_PTS, if the provided VariableRef is of type NOT-ACCESSIBLE, then the NO-INPUT-QUANTITY

keyword will be assigned as *InputQuantity* for the created *AXIS_DESCR* block.

- If the *GROUPED CURVE/MAP* type of *<MC-DATA-INSTANCE>* is referencing the *NOT-ACCESSIBLE* type of *MC-DATA-INSTANCE*, then the *AXIS_PTS_REF* attribute will not be set for the generated A2L *CHARACTERISTIC* blocks.
- The keyword *CALIBRATION_ACCESS* is not supported by the INCA tool. Hence, this attribute will not be generated by RTA-A2L.
- If *<MC-DATA-INSTANCE>* does not contain *<DATA-CONSTR-REF>* and contains a *<BASE-TYPE-REF>* other than *FLOAT*, then RTA-A2L will generate the *INTERNAL* default limit values for the *CHARACTERISTIC/AXIS_PTS* blocks based on the *<BASE-TYPE-REF>* provided.
- In the case of the *CURVE/MAP* types of *<MC-DATA-INSTANCE>*s, if the *Parent <SW-CALIBRATION-ACCESS>* is of type *READ-ONLY/READ-WRITE*, but the *<SW-CALPARAM-AXIS>* contains the *<SW-CALIBRATION-ACCESS>* with value *NOT-ACCESSIBLE*, then RTA-A2L will ignore the *<SW-CALIBRATION-ACCESS>* tag of the Axis and continue the *AXIS_DESCR* block generation. The *READ-ONLY* attribute will not be generated for the *Axis_DESC* blocks of the *CHARACTERISTICS*.
- If the *<SW-CALIBRATION-ACCESS>* is not defined at the *<MC-DATA-INSTANCE>* level, or at the *<SW-CALPARAM-AXIS>* level then, by default, ARTOP returns the *READ-ONLY* type of *<SW-CALIBRATION-ACCESS>*. Hence, the *READ-ONLY* attribute will be generated for the *CHARACTERISTIC/Axis_DESC/AXIS_PTS* blocks.
- For a *BOOLEAN Calprm <MC-DATA-INSTANCE>*, if the *COMPU-METHOD-REF* is missing, or the definition is not present, then the *CHARACTERISTIC* block for that *MC-DATA-INSTANCE* is generated in the A2L file, with the *CompuMethod* name as *NO_COMPU_METHOD*.
- For a *VAL_BLK Calprm <MC-DATA-INSTANCE>*, *<SW-VALUE-BLOCK-SIZE>* is a mandatory attribute in the resulting properties of an *MC-DATA-INSTANCE*. If this tag is missing/invalid, then there will not be a corresponding *CHARACTERISTIC* block generated in the A2L file.
- In the case of *ARRAY subElements*, if the leaf-level element does not contain the *<LONG-NAME>/<DESC>/<ANNOTATION>* blocks, then its immediate *ARRAY Parent* element will be checked. If the content exists, then it will be set for the leaf-level element. In the case of multi-dimensional *ARRAYs*, the check will be repeated until the root-level element. If the leaf element's immediate *Parent* is of type *STRUCTURE*, then the looping will stop and no values from *STRUCTURE* elements will be fetched.
- *ETK, XCP IF_DATA* blocks will not be generated if the input *<MC-DATA-INSTANCE>* does not contain the *<SW-REFRESH-TIMING>*, *<CSE-CODE>* and *<CSE-CODE-FACTOR>* entries.
- ARRTA-A2L supports 64-bit values, which RTA-A2L turns into double values
 - losing precision to a certain extent.
- If a block refers to a host variable for *BIT-MASK* generation, the host variable

should be of the type <VALUE>.

MC-VARIABLE-INSTANCES

The following categories are supported:

VALUE	
BOOLEAN	
STRUCTURE	Having a <i>subElement</i> of type ARRAY, VALUE.
ARRAY	Element types of VALUE, ARRAY, STRUCTURE.

- For the *DISPLAY-FORMAT* at variable *MC-DATA-INSTANCE* level, only the following pattern is supported: *%<dddd>. <ddd>f*. In this case, the same content without the *f* suffix is written to the *FORMAT* attribute for the corresponding block in the A2L file.
- In a case where another pattern is specified, the *FORMAT* attribute is not generated.
- It is mandatory for a variable *MC-DATA-INSTANCE* to have a valid *BASE-TYPE-REF*.
- For the variable *MC-DATA-INSTANCE* (except *BOOLEAN*), *DATA-CONSTR-REF* is a mandatory attribute only in the case when *BASE-TYPE-REF* references a *FLOAT* data type.
- For *MEASUREMENT* blocks generated by RTA-A2L, the *ARRAY_SIZE* attribute is only used for Simple *ARRAY MEASUREMENT* blocks. In the case of complex *MEASUREMENT ARRAYS* (i.e. an *ARRAY* inside an *ARRAY* etc.) the *ARRAY_SIZE* attribute is not used. As a result, the *MEASUREMENT* block (containing the *ARRAY* index in the *ShortName*) is generated for each *ARRAY* element of the variable *MC-DATA-INSTANCE*. *Symbol* information will not be available for the *subElements* of *MC-DATA-INSTANCE*.
- RTA-A2L will derive the *Symbol* information for the *subElements* by considering that, in the *MC-Support* file, the *STRUCTURE* of the *MC-DATA-INSTANCE* and its *subElements* is identical to the C code generated by the RTE Generator.
- If the <SW-CALIBRATION-ACCESS> of the <MC-DATA-INSTANCE> is of type *NOT-ACCESSIBLE*, then that block will not be generated in the A2L or ADX files.
- If the <MC-DATA-INSTANCE> does not contain the <DATA-CONSTR-REF> but does contains a <BASE-TYPE-REF> other than *FLOAT*, then RTA-A2L will generate the *INTERNAL* default limit values for the *MEASUREMENT* blocks based on the <BASE-TYPE-REF> provided.
- The *READ-WRITE* attribute will not be supported by RTA-A2L for the *MEASUREMENT* blocks. Hence, by default, all the RTA-A2L generated *MEASUREMENT* blocks will be *READ-ONLY*.
- For a *BOOLEAN MC-DATA-INSTANCE*, if *COMPU-METHOD-REF* is missing, or the definition is not present, then the *MEASUREMENT* block for that

MC-DATA-INSTANCE is generated in the A2L file with the *CompuMethod* name as *NO_COMPU_METHOD*.

- In case of *ARRAY subElements*, if the leaf-level element does not contain the `<LONG-NAME>/<DESC>/<ANNOTATION>` blocks, then its immediate *ARRAY Parent* element will be checked, and if the content exists then it will be set for the leaf-level element. In the case of multi-dimensional *ARRAYs*, the check will be repeated until the root-level element. If the leaf element's immediate *Parent* is *STRUCTURE*, then the looping will be stopped and no values from the *STRUCTURE* elements will be fetched.
- *ETK*, *XCP IF_DATA* blocks will not be generated if the input `<MC-DATA-INSTANCE>` does not contain the `<SW-REFRESH-TIMING>`, `<CSE-CODE>` and `<CSE-CODE-FACTOR>` entries.
- ARRTA-A2L supports 64-bit values, which RTA-A2L generates as double values - losing precision to a certain extent.
- If the block refers to a host variable for *BIT-MASK* generation, the host variable should be of the type `<VALUE>`.

MC-DATA-INSTANCE-VAR-REF

The restriction is for generation of virtual *MEASUREMENTs*.

The `<MC-DATA-INSTANCE>` that is referenced inside `<MC-DATA-INSTANCE-VAR-REF>` should be of type *VALUE*.

D-SERAP STRUCTUREs

RTA-A2L supports the *D-SERAP STRUCTUREs*, if both the *Parent* and child are defined with the *SDGs* as *MC-DATA-CLASS*. Or else, it acts as a normal *STRUCTURE*.

Always for the *D-SERAP STRUCTURE*, the *Parent* root element should contain the *SD GID* as the *MC-DATA-CLASS* type with the value as *CALPRM-GROUP*. All the child elements should also contain the *SD GID* as *MC-DATA-CLASS* with the value as *Calprm*:

```
<MC-DATA-INSTANCE>
  <SHORT-NAME>CanIf_ControllerState</SHORT-NAME>
  <CATEGORY>STRUCTURE</CATEGORY>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="ETAS-RTARTE">
        <SD GID="MC-DATA-CLASS">CALPRM-GROUP</SD>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
  <RESULTING-PROPERTIES>
    -----
  <SUB-ELEMENTS>
    <MC-DATA-INSTANCE>
      <SHORT-NAME>CanIf_ControllerModeType</SHORT-NAME>
      <CATEGORY>VALUE</CATEGORY>
      <ADMIN-DATA>
        <SDGS>
          <SDG GID="ETAS-RTARTE">
            <SD GID="MC-DATA-CLASS">Calprm</SD>
          </SDG>
        </SDGS>
      </ADMIN-DATA>
      <RESULTING-PROPERTIES>
        -----
      </MC-DATA-INSTANCE>
    <SUB-ELEMENTS>
  </MC-DATA-INSTANCE>
```

If any of the *D-SERAP* child elements does not contain the *MC-DATA-CLASS* type as *Calprm*, then the block will be ignored.

5.9.11 MC-FUNCTION

RTA-A2L *FUNCTION* blocks will be generated from *<MC-FUNCTION>* tags inside the *<MC-DATA-INSTANCE>* (for AUTOSAR files < v4.1.1), or *<MC-FUNCTION>* blocks from the input file (for AUTOSAR files = > v4.1.1).

The *A2LFunction* block generated using the *MC-FUNCTION* tags will only have the *Name*, *default Longname*, *DEF_CHARACTERISTIC* and *LOC_MEASUREMENT* blocks; other information cannot be written as it is not available as a part of the *MC-Support* file.

*Calprm<MC-DATA-INSTANCE>*s are associated to a *FUNCTION* block through *DEF_CHARACTERISTICS*. *Variable <MC-DATA-INSTANCE>*s are associated to a *FUNCTION* block through *LOC_MEASUREMENTS*.

Currently, neither *Variable/Calprm <MC-DATA-INSTANCE>*s can be associated to a *FUNCTION* block though *REF_CHARACTERISTICS*, *IN_MEASUREMENTS* or *OUT_MEASUREMENTS* as the import/export information of *<MC-DATA-INSTANCE>*s is not present in the *MC-Support* file.

In case of *Complex<MC-DATA-INSTANCE>*s, if the child element does not contain the *MC-FUNCTION* value, but the *ROOT Parent* element contains the *MC-FUNCTION* value, then this root element *FUNCTION* will be considered for the child element.

e.g. a *STRUCTURE* which contains the *CanIf* module with two *VALUE subElements*:

```
<MC-DATA-INSTANCE>
  <SHORT-NAME>CanIf_ControllerState</SHORT-NAME>
  ---
  <CATEGORY>STRUCTURE</CATEGORY>
  <RESULTING-PROPERTIES>                                <SW-DATA-DEF-PROPS-variantS>
  ---
  <MC-FUNCTION>CanIf</MC-FUNCTION>
  </SW-DATA-DEF-PROPS-VARIANTS>
  </RESULTING-PROPERTIES>
  <SUB-ELEMENTS>
    <MC-DATA-INSTANCE>
      <SHORT-NAME>CanIf_ControllerModeType</SHORT-NAME>
      <CATEGORY>VALUE</CATEGORY>
      -----
    </MC-DATA-INSTANCE>
    <MC-DATA-INSTANCE>
      <SHORT-NAME>CanIf_PduGetModeType</SHORT-NAME>
      <CATEGORY>VALUE</CATEGORY>
      -----
    </MC-DATA-INSTANCE>
  </SUB-ELEMENTS>
  <SYMBOL>CanIf_ControllerStateSymbol</SYMBOL>
</MC-DATA-INSTANCE>
```

```
<MC-DATA-INSTANCE>
  <SHORT-NAME>EpmNEngArray_Tina</SHORT-NAME>
  <CATEGORY>ARRAY</CATEGORY>
  <ARRAY_SIZE>2</ARRAY_SIZE>
  <RESULTING-PROPERTIES>
  <SW-DATA-DEF-PROPS-VARIANTS>
  ---
  <MC-FUNCTION>CanIf</MC-FUNCTION>
  </SW-DATA-DEF-PROPS-VARIANTS>
  </RESULTING-PROPERTIES>
  <SUB-ELEMENTS>
    <MC-DATA-INSTANCE>
      <SHORT-NAME>element_0</SHORT-NAME>
```

```

        <CATEGORY>VALUE</CATEGORY>
    -----
    </MC-DATA-INSTANCE>
    <MC-DATA-INSTANCE>
        <SHORT-NAME>element_1</SHORT-NAME>
        <CATEGORY>VALUE</CATEGORY>
    -----
    </MC-DATA-INSTANCE>
</SUB-ELEMENTS>
<SYMBOL>EpmNEngArray_Tina_Symbol</SYMBOL>
</MC-DATA-INSTANCE>

```

For the *MC_FUNCTION* blocks of AUTOSAR v4.1.1, RTA-A2L currently supports the *Name*, *LongIdentifier*, *DEF_CHARACTERISTICS*, *REF_CHARACTERISTICS*, *IN_MEASUREMENT*, *OUT_MEASUREMENT*, *LOC_MEASUREMENT*, *SUB_FUNCTIONS* and *FUNCTION VERSION* information.

Only the variant information of the *McFunction* will be considered as the *FUNCTION_VERSION* <REVISION-LABEL>.

If no variant information appears for the defined *McFunction* then no *FUNCTION VERSION* information will be generated.

If both *MC-FUNCTION* tags and *MC-FUNCTION* blocks appear in the input files, then preference will be given to *MC-FUNCTION* blocks (as per AUTOSAR v4.1.1 there is a possibility to have *MC-FUNCTION* tags and *MC-FUNCTION* blocks).

The variant information from the *ifdefList* should follow the '[0-9]+.[0-9]+.[0-9]+([_.;])?' format only (e.g. 5.3.0, 5.6.9 etc.) or else RTA-A2L will throw an error.

In the input *MC-Support* files, if <ADMIN-DATA> blocks do not appear, then the complete <ADMIN-DATA> blocks will *NOT* be created in the updated *MC-Support* files.

In the input *MC-Support* file, if an <ADMIN-DATA> block appears with some information then, in this case, the <REVISION-LABEL> tag will *NOT* be created in the updated *MC-Support* files.

5.9.12 Mapping ADX File Generation

If the <SHORT-NAME> and the <SYMBOL> information of the *McDataInstances* are the same, then no ADX entries will be generated for that block in the mapping ADX file.

In this case, it is expected that there are no multiple-file static *variables/Calprms* with the same name in the project. (This can cause problems for the RTA-A2L Generator.)

e.g. *can%var1*, *veh%var1* can cause ambiguity when considering an address, since only one *var1 MC-DATA-INSTANCE* is specified in the *MC-Support ARXML* file.

The RTA-A2L Generator wouldn't be able to decide whether a *variable/Calprm MC-DATA-INSTANCE* is *STATIC* (or) *GLOBAL* in C. By default, it will consider the *Symbol* information specified in the *MC-DATA-INSTANCE* and generate the *LABEL-NAME* tag value in the corresponding mapping ADX entry.

It is recommended, therefore, not to have *STATIC* variables in an A2L file, in case a label is required: it should be *GLOBAL* in C. For more details, please refer to the ARMETH rule: *Rule BSWMD_McSupport_002: C-Scope of MEASUREMENT Vari-*

ables shall be defined as global.

5.9.13 Encoding of A2L Files

RTA-A2L supports the *UTF-8*, *UTF-8 BOM*, *US-ASCII* and *ISO-8859-1(ANSI/WINDOWS-1252/CP-1252)* encoding formats for input A2L files.

6 Non-AUTOSAR A2L Generation

6.1 Introduction

RTA-A2L contains functionality to generate an A2L file directly from the compiled code, without the specification. This can be achieved by extracting elements from the ELF file generated by the Project into an ADX file using *dumpers*. Using a list of configured parameters and signals, RTA-A2L can then be used to generate an A2L file from the ADX file.

6.2 A2L Generation Overview

To generate an A2L file, the following inputs are required:

- Input ADX File.
- Parameter/Signal Text File: Either or both parameter and signal text files can be passed to RTA-A2L.
- Path to Output A2L File: This specifies where the generated A2L file will be saved.

The ADX file is processed by RTA-A2L using an ADX parser. *CHARACTERISTICS* and *MEASUREMENTS* are generated for the matched patterns provided in the parameters and signals file. The output will be the corresponding A2L file.

Additional Options: Command-line options allow customization for generating the A2L file. They include byte order, format, array expansion, and record layout index mode.

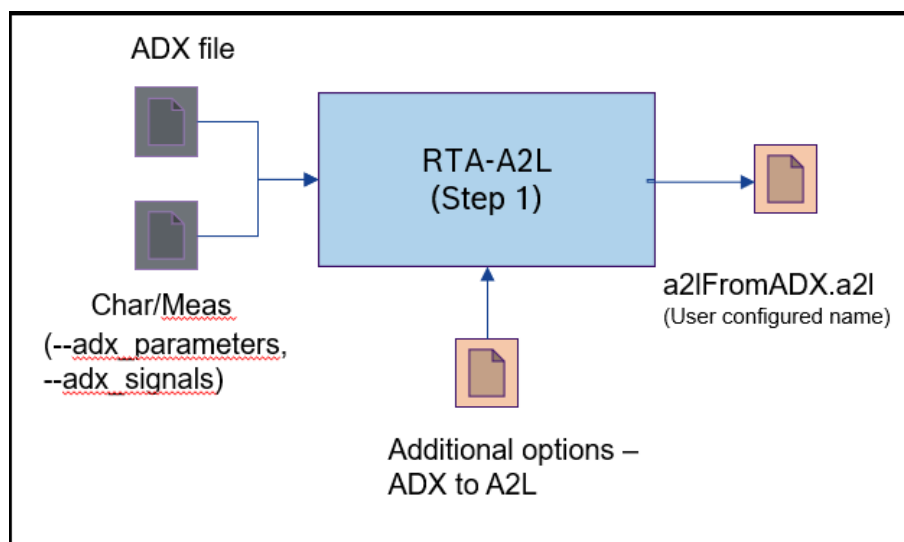


Figure 6.1.A2L Generation Overview

Using RTA-A2L, an A2L file can be generated using an ELF file. Please see *Translating ELF/ADX to A2L* (Section 4.3) for more details.

6.3 A2L File Contents

RTA-A2L supports the following A2L file structure:

6.3.1 ASAP2_VERSION Details

The following is the *ASAP2_VERSION* structure supported by RTA-A2L:

```
ASAP2_VERSION Version Upgrade
```

By default, the *ASAP2_VERSION* is set to 1 61, where:

- Major Version: 1
- Upgrade Number: 61

The User can set a desired *ASAP2_VERSION* for the A2L files using the *Asap2Version* option. Please see *RTA-A2L Options* (Section 3.2) for more details.

6.3.2 A2L Project Details

The *A2LProject STRUCTURE* supported by the RTA-A2L Generator is as follows:

```
/begin PROJECT Name LongIdentifier
  [-> HEADER]
  {-> MODULE}*
/end PROJECT
```

where:

<i>Name</i>	The <i>ADX</i> has been considered as the <i>Name</i> for the <i>A2LProject</i> .
<i>LongIdentifier</i>	The <i>ADX_A2L_Gen</i> has been considered as the <i>longIdentifier</i> . The same value will be appearing as the <i>VERSION</i> information in the <i>MOD_PAR</i> block.
<i>HEADER</i>	The <i>A2Lheader</i> will be created and set to the <i>A2LProject</i> . Please see <i>A2LHeader Details</i> (Section 6.3.3).
<i>MODULE</i>	The <i>A2LModule</i> is created and all the module contents are loaded. The same <i>A2LModule</i> is added to an <i>A2LProject</i> . The <i>A2LProject</i> may contain zero or more <i>A2LModules</i> . RTA-A2L supports the single <i>Module</i> , and all the contents of the parameters and signals file are mapped to the single <i>Module</i> .

6.3.3 A2LHeader Details

The *A2LHeader* has the following *STRUCTURE*:

```
/begin HEADER Comment
  [-> VERSION]
  [-> PROJECT_NO]
/end HEADER
```

where:

<i>Comment</i>	No HEADER Comment will be set by RTA-A2L. Hence, by default an empty string will be set as Comment.
<i>VERSION</i>	The Header Version will be set to <i>ADX_A2L_1.0</i> .
<i>PROJECT_NO</i>	The Project Number will be set to <i>ADX_A2L_Gen_1_0</i> .

This *A2LHeader* will be added to the *A2LProject*.

6.3.4 A2LModule Details

RTA-A2L supports the following *A2LModule STRUCTURE*:

```
/begin MODULE Name LongIdentifier
[-> MOD_PAR]
{-> CHARACTERISTIC}*
{-> MEASUREMENT}*
{-> RECORD_LAYOUT}*
/end
```

where:

<i>MODULE Name</i>	The <i>Name</i> of the <i>Module</i> has been set to <i>ADX</i> .
<i>LongIdentifier</i>	Currently, RTA-A2L does not set a <i>LongIdentifier</i> to the <i>A2LModule</i> . Hence, by default an empty string will appear in the generated A2L file.

The *A2LModule* is loaded with *CHARACTERISTICS*, *MEASUREMENTS*, and *RECORD_LAYOUTS*.

6.3.5 MOD_PAR Block Details

For each *Module*, only one *MOD_PAR* Block will appear in an A2L file. RTA-A2L currently supports the following attributes in the *MOD_PAR* Block:

```
/begin MOD_PAR Comment
VERSION
/end MOD_PAR
```

where:

<i>Comment</i>	Currently, no information has been set for the <i>Comment</i> of the <i>MOD_PAR</i> Block. Hence, by default, an empty string will appear in the generated A2L file.
<i>VERSION</i>	<i>ADX_A2L_Gen</i> has been set as the <i>VERSION</i> information for the <i>MOD_PAR</i> Block and this should be equal to the <i>LongIdentifier</i> of the Project.

6.3.6 RECORD_LAYOUT Details

Default *RECORD_LAYOUTS* are generated for *CHARACTERISTICS* blocks based on the *Data Type*.

The structure of the *RECORD_LAYOUT* in the A2L file is shown below:

```
/begin RECORD_LAYOUT name
  FNC_VALUES <Pos> <Datatype> (ROW_DIR|COLUMN_DIR) DIRECT
/end RECORD_LAYOUT
```

where:

<i>Name</i>	Identification of the <i>RECORD_LAYOUT</i> , which is referenced via this <i>Name</i> in the <i>CHARACTERISTICS</i> block. The name of the A2L <i>RECORD_LAYOUT</i> is derived from the <LABEL-NAME> tag of the input <MEMORY-ELEMENT> and the A2L <i>DataType</i> . The name will be set as <i>DefaultValueRecordLayout_<DataType></i> .
<Pos>	The <Pos> value is always generated as 1.
<DatType>	The <DatType> value is set based on the A2L <i>DataType</i> generated from the respective <MEMORY-ELEMENT>. Please refer to <i>DataTypes for the A2LBlocks</i> (Section 6.3.9) for more information.
<i>INDEX_MODE</i>	<i>ROW_DIR</i> or <i>COLUMN_DIR</i> . The <i>ROW_DIR</i> <i>COLUMN_DIR</i> will be set for the <i>VALUE</i> type of <i>RECORD_LAYOUT</i> based on the index mode given in the --reclay_indexmode option. By default, the index mode will be generated as <i>ROW_DIR</i> .
<i>DIRECT</i>	The <i>DIRECT</i> value will be set as <i>DIRECT</i> for the <i>VALUE</i> type of <i>RECORD_LAYOUTS</i> .
<i>FNC_VALUES</i>	This keyword describes how the function values of the adjustable object are deposited in memory.

6.3.6.1 RECORD_LAYOUT Example

A *DefaultRecordLayout* will be created for each *DataType* used in *CHARACTERISTIC* blocks in the A2L file:

```
/begin CHARACTERISTIC
  Avs_AdjValCfgTab_cs[0].CalVal_u8
  ""
  VALUE
  0x80024670
  DefaultValueRecordLayout_SLONG
  2.362267295E9
  NO_COMPU_METHOD
  -2.14783648E8
  2.147483647E9
  BIT_MASK 0x3FFFC
/end CHARACTERISTIC

/begin RECORD_LAYOUT DefaultValueRecordLayout_SLONG
  FNC_VALUES      1 SLONG ROW_DIR DIRECT
/end RECORD_LAYOUT
```

6.3.7 CHARACTERISTIC Details

Currently, RTA-A2L supports *VALUE* and *VAL_BLK* as input to create the *CHARACTERISTIC* blocks.

The following *CHARACTERISTIC* details are supported:

```

/begin CHARACTERISTIC Name
  LongIdentifier
  Type
  Address
  Deposit
  MaxDiff
  Conversion
  LowerLimit
  UpperLimit
  [-> FORMAT]
  {[BYTE_ORDER]}
  {[[-> BIT_MASK]}
  {[[-> MATRIX_DIM]}
/end CHARACTERISTIC

```

where:

Name:

The Name is taken from the <LABEL-NAME> tag of the ADX <MEMORY-ELEMENT>.

LongIdentifier:

The *LongIdentifier* will be generated as an empty string.

Type:

The *Type* will be the value fetched from the <CATEGORY> tag of the ADX Memory Element. For the *ARRAY* category, the *Type* will be generated as *VAL_BLK* and, for other categories, *Type* will be generated as *VALUE* in the A2L file.

Address:

The *Address* value will be fetched from the <ABSOLUTE-ADDRESS> tag in the ADX Memory Element.

Deposit:

A reference to the corresponding data record for the description of the *RECORD_LAYOUT*. Please refer to *RECORD_LAYOUT Details* (Section 6.3.6) for more information.

MaxDiff:

The *MaxDiff* is calculated as the difference between the *UpperLimit* and the *LowerLimit* of the *CHARACTERISTIC* block. The formula is:

$$\text{MaxDiff} = \text{UpperLimit} - \text{LowerLimit}$$

If the difference between the *UpperLimit* and *LowerLimit* is large i.e. beyond the range of a double data type, then *MaxDiff* will be set to $1.7976931348623157 \times 10^{308}$.

Conversion:

The Conversion method will be always set as *NO_COMPU_METHOD*.

LowerLimit:

UpperLimit:

These are mandatory parameters for creating the *CHARACTERISTIC* block. To create these parameters, there must be a valid A2L *DataType*, upon which *LowerLimit* and *UpperLimit* will be set.

Please see *Generation of A2LLimits based on Input Format* under Limits Details

(Section 6.3.10) for more information.

FORMAT:

The *Format* will be created only if the `--format` option is provided with a value.

- If the `--format` is supplied with *Formats* like `%[ddd.dd]`, `%[.dd]`, `%[ddd]`, or only digits, the *Format* is set exactly as given in the input.
- For the *VALUE* and *VAL_BLK* types of *CHARACTERISTICS*, the value will be set as given in the input.
- If the `--format` is not given as an option, then no *Format* will be generated for *CHARACTERISTIC* blocks.

BYTE_ORDER:

The *BYTE_ORDER* will be created only if the `--byte_order` option is given a value.

- Only the *MSB_FIRST* and *MSB_LAST* values are supported for the *BYTE_ORDER* in A2L file generation.

MATRIX_DIM:

This attribute is generated for the *VAL_BLK* type of *CHARACTERISTIC*. It is supported for single/multi-dimensional arrays (i.e. *Arrays* of *VALUE* type). This includes single/multi-dimensional arrays that are leaf *SUB-ELEMENTS* of a *STRUCTURE*.

It is generated if the *MATRIX_DIM* switch `--expand_arrays` is disabled in the options file or if the *ASAP_VERSION* is ≥ 1.6 .

Support for One-Dimensional Arrays:

- If the *ASAP_VERSION* is < 1.7 , Y and Z dimensions are generated as 1. Otherwise, only the X dimension is generated.
- The value of the tag `<ARRAY-NBR-ELEMENTS>` is set as the X-dimension of *MATRIX_DIM*.

Support for Single/Multi-Dimensional Arrays:

- Single/multi-dimensional arrays whose *SUB-ELEMENTS* are of type *VALUE* are generated with *MATRIX_DIM*.
- *STRUCTURE* leaf elements that are single/multi-dimensional arrays and of type *VALUE* are generated with *MATRIX_DIM*.
- Single-dimension arrays are generated with the X dimension.

The dimensions are determined from the `<ARRAY-NBR-ELEMENTS>` tag. For multi-dimensional arrays, the dimensions are set based on the respective `<ARRAY-NBR-ELEMENTS>` tags of the *SUB-ELEMENTS*, depending on the availability of the X, Y, and Z dimensions.

e.g. Single-dimension with `<ARRAY-NBR-ELEMENTS>`:

```
If ASAP_VERSION < 1.7 then
MATRIX_DIM 18 1 1
```

```
If ASAP_VERSION >= 1.7 then
MATRIX_DIM 18
```

BIT_MASK:

RTA-A2L supports *BIT_MASK* creation for *CHARACTERISTICS* blocks. These blocks are generated with the values specified in the tags <BIT-POSITION> and <NUMBER-OF-BITS> present with the ADX <MEMORY-ELEMENT>.

The *BIT_MASK* keyword can be used to mask out single bits of the value to be processed. The least significant bit in *BIT_MASK* determines how far the masked value is shifted to the right.

If <BIT-POSITION> and <NUMBER-OF-BITS> are not present with the ADX <MEMORY-ELEMENT> then, for the corresponding block, *BIT_MASK* is not considered for A2L generation.

ADX ENTRY for BIT_MASK:

An *ADX ENTRY* is created for every block which contains the tags <BIT-POSITION> and <NUMBER-OF-BITS>.

e.g. ADX snippet:

```
<MEMORY-ELEMENT>
  <LABEL-NAME>MoFFrm_Rx_bbOut1</LABEL-NAME>
  <CATEGORY>DATA</CATEGORY>
  <ENCODING>UNSIGNED</ENCODING>
  <ABSOLUTE-ADDRESS>0x600062F5</ABSOLUTE-ADDRESS>
  <SIZE>1</SIZE>
  <BIT-POSITION>4</BIT-POSITION>
  <NUMBER-OF-BITS>1</NUMBER-OF-BITS>
  <COMP-UNIT-NAME>_merged_dat.c</COMP-UNIT-NAME>
  <COMP-UNIT-DIR>_gen\swb\filegroup\src_files\data</COMP-UNIT-DIR>
</MEMORY-ELEMENT>
```

Generated A2L snippet:

```
/begin CHARACTERISTIC
  MoFFrm_Rx_bbOut1
  ""
  VALUE
    0x600062F5
  DefaultValueRecordLayout_UBYTE
    255.0
  NO_COMPU_METHOD
    0.0
    255.0
  BIT_MASK 0x10
/end CHARACTERISTIC
```

6.3.8 MEASUREMENTS Details

Currently, RTA-A2L supports *UBYTE*, *SBYTE*, *UWORD*, *SWORD*, *ULONG*, *SLONG*, *FLOAT32_IEEE* and *FLOAT64_IEEE* to create the MEASUREMENT blocks. The following MEASUREMENT details are supported:

```
/begin MEASUREMENT Name
  LongIdentifier
  Datatype
  Conversion
  Resolution
  Accuracy
  LowerLimit
  UpperLimit
  [-> FORMAT]
  {[BYTE_ORDER]}
  {[-> BIT_MASK]}
  {-> MATRIX_DIM}
```

/end MEASUREMENT

Name:

The *Name* is taken from the <LABEL-NAME> tag of the ADX <MEMORY-ELEMENT>.

LongIdentifier:

The *LongIdentifier* will be generated as an empty string.

Type:

The *Type* will be the value determined from the <ENCODING> and <SIZE> tags of the ADX Memory Element and it would be set as *UBYTE*, *SBYTE*, *UWORD*, *SWORD*, *ULONG*, *SLONG*, *FLOAT32_IEEE* or *FLOAT64_IEEE*.

Address:

The *Address* value will be fetched from the <ABSOLUTE-ADDRESS> tag in the ADX Memory Element.

Conversion:

The *Conversion* method will always be set as *NO_COMPU_METHOD*.

Resolution:

RTA-A2L always sets the *Resolution* of the MEASUREMENT block to 1.

Accuracy:

RTA-A2L always sets the *Accuracy* of the MEASUREMENT block to 0.

LowerLimit:**UpperLimit:**

These are mandatory parameters for creating the *MEASUREMENT* block. To create these parameters, there must be a valid A2L *DataType*, upon which *LowerLimit* and *UpperLimit* will be set.

Please see *Generation of A2LLimits based on Input Format* under Limits Details (Section 6.3.10) for more information.

FORMAT:

The *Format* will be created only if the --format option is provided with a value.

- If the --format is supplied with *Formats* like %[ddd.dd], %[.dd], %[ddd], or only digits, the *Format* is set exactly as given in the input.
- If the --format is not given as an option, then no *Format* will be generated for *MEASUREMENT* blocks.

BYTE_ORDER:

The *BYTE_ORDER* will be created only if the --byte_order option is provided with a value.

- Only the *MSB_FIRST* and *MSB_LAST* values are supported for the *BYTE_ORDER* in A2L file generation. Consequently, *MSB_FIRST* and *MSB_LAST* will be produced in the A2L file.

MATRIX_DIM:

This attribute is generated for the *VAL_BLK* type of *MEASUREMENT*. It is also supported for single/multi-dimensional arrays (i.e. *Arrays* of *VALUE* type). This includes single/multi-dimensional arrays that are leaf *SUB-ELEMENTS* of a *STRUCTURE*.

The attribute is generated if the *MATRIX_DIM* switch `--expand_arrays` is disabled in the options file, or if the *ASAP_VERSION* is ≥ 1.6 .

Support for One-Dimensional Arrays:

- If the *ASAP_VERSION* is < 1.7 , the Y and Z dimensions are generated as 1. Otherwise, only the X dimension is generated.
- The value of the tag `<ARRAY-NBR-ELEMENTS>` is set as the X-dimension of *MATRIX_DIM*.

Support for Single/Multi-Dimensional Arrays:

- Single/multi-dimensional arrays whose *SUB-ELEMENTS* are of type *VALUE* are generated with *MATRIX_DIM*.
- *STRUCTURE* leaf elements that are single/multi-dimensional arrays and of type *VALUE* are generated with *MATRIX_DIM*.
- Single-dimensional arrays are generated with the X-dimension.

The dimensions are determined from the `<ARRAY-NBR-ELEMENTS>` tag. For multi-dimensional arrays, the dimensions are set based on the respective `<ARRAY-NBR-ELEMENTS>` tags of the *SUB-ELEMENTS*, depending on the availability of the X, Y, and Z dimensions.

e.g. single dimension with `<ARRAY-NBR-ELEMENTS>`:

```
If ASAP_VERSION < 1.7 then
MATRIX_DIM 18 1 1
If ASAP_VERSION >= 1.7 then
MATRIX_DIM 18
```

BIT_MASK:

RTA-A2L supports *BIT_MASK* creation for *MEASUREMENT* blocks. These blocks are generated with the values specified in the tags `<BIT-POSITION>` and `<NUMBER-OF-BITS>` present with the ADX `<MEMORY-ELEMENT>`.

The *BIT_MASK* keyword can be used to mask out single bits of the value to be processed. The least significant bit in *BIT_MASK* determines how far the masked value is shifted to the right.

If `<BIT-POSITION>` and `<NUMBER-OF-BITS>` are not present with the ADX `<MEMORY-ELEMENT>` then, for the corresponding block, *BIT_MASK* is not considered for A2L generation.

ADX ENTRY for BIT_MASK:

An *ADX ENTRY* is created for every block which contains the tags `<BIT-POSITION>` and `<NUMBER-OF-BITS>`.

e.g.:

```
<MEMORY-ELEMENT>
  <LABEL-NAME>MoFFrm_Rx_bbOut1</LABEL-NAME>
  <CATEGORY>DATA</CATEGORY>
  <ENCODING>UNSIGNED</ENCODING>
  <ABSOLUTE-ADDRESS>0x600062F5</ABSOLUTE-ADDRESS>
  <SIZE>1</SIZE>
  <BIT-POSITION>4</BIT-POSITION>
  <NUMBER-OF-BITS>1</NUMBER-OF-BITS>
  <COMP-UNIT-NAME>merged_dat.c</COMP-UNIT-NAME>
  <COMP-UNIT-DIR>_gen\swb\filegroup\src_files\data</COMP-UNIT-DIR>
</MEMORY-ELEMENT>
```

Generated A2L snippet:

```
/begin MEASUREMENT
MoFFrm_Rx_bbOut1
""
UBYTE
NO_COMPU_METHOD
1
0
0.0
255.0
BIT_MASK 0x10
ECU_ADDRESS 0x600062F5
/end MEASUREMENT
```

6.3.9 DataTypes for the A2LBlocks

The *DataTypes* are created using the <ENCODING> tag from the ADX <MEMORY-ELEMENT>. The following example represents the tag structure in an ADX file:

```
<ENCODING>UNSIGNED</ENCODING>
```

If the <CATEGORY> is *DATA*, *UNKNOWN* or *TRICOREBIT*, then the A2L *DataType* will be recognized.

DataTypes are created based on the <SIZE> and <ENCODING> tags.

RTA-A2L supports the following *DataTypes*: *UBYTE*, *SBYTE*, *UWORD*, *SWORD*, *ULONG*, *SLONG*, *FLOAT32_IEEE*, *FLOAT64_IEEE*.

The following table represents mappings from ADX Encoding to A2L *DataType*:

SIZE (ADX)	ENCODING (ADX) contains...	DataType(A2L)
1	UNSIGNED	UBYTE
1	SIGNED	SBYTE
2	UNSIGNED	UWORD
2	SIGNED	SWORD
4	UNSIGNED	ULONG
4	SIGNED	SLONG
4	FLOAT	FLOAT32_IEEE
8	UNSIGNED	AUINT64
8	SIGNED	AINT64
8	FLOAT	FLOAT64_IEEE

6.3.10 Limits Details

The A2L *DataType* will be used as the input for the creation of the *LowerLimit* and *UpperLimit* for the *CHARACTERISTIC/MEASUREMENT* blocks.

Default Limit Values for the A2L *DataTypes*:

DataType(A2L)	Default <i>LowerLimit</i>	Default <i>UpperLimit</i>
UBYTE	0	255
UWORD	0	65535

ULONG	0	4294967295L
SBYTE	0	127
SWORD	-3276	32767
SLONG	-214783647L - 1	2147483647L
FLOAT32_IEEE	-3.4028235E38	3.4028235E38
FLOAT64_IEEE	-1.7976931E308	1.7976931E308

Generation of A2LLimits based on the Input Format:

- The *A2LLimits* (*UpperLimit* and *LowerLimit*) are generated in the A2L file based on the *Format* (e.g. *Format* = 10.5, 5 is given priority).
- The decimal part of the display *Format* is given priority for the generation. The number of digits after the decimal in the *A2LLimit* should be equal to the value of decimal part of the *Format*. If the number of digits in the *A2LLimit* after the decimal is less than the required *Format*, zeroes are added to the decimal part. If the number of digits is greater than the required *Format*, then the number is rounded up to match the *Format*.
- If the *A2LLimit* is an exponential number, the *Format* is applied to the non-exponential part.

e.g.:

Input limit = 3.2 Display format = 8.5 Generated limit = 3.20000 Input limit = 12345.5 Display format = 10.0 Generated limit = 12346.

Exponential numbers:

Input limit = 1.4E20 format = 8.3 Generated limit = 1.400E20 Input limit = 1.526E12 format = 10.2 Generated limit = 1.53E12

6.4 Restrictions

ADX Categories:

- The categories *DATA*, *STRUCTURE*, *ARRAY* and *TRICOREBIT* are supported for A2L file generation.
- The categories *UNION*, *FUNCTION* and *POINTER* are not supported for A2L file generation.
- The category *UNKNOWN* is supported only if the *DataType* and size are valid in the ADX file.

Static Elements:

- If only one static label is present, RTA-A2L shall generate the *labelName* in the A2L file. e.g.:

ADX - *Source1_c%labelName*

A2L - *labelName*

- If two static labels are present with the same *labelName*, RTA-A2L shall

generate labels by prefixing the source file name. e.g.:

ADX - *Source1_c%labelName* and *Source2_c%labelName*

A2L - *Source1_c_labelName*, *Source2_c_labelName*



NOTE

A2L Generation tool performance:

1. AUTOSAR A2L Generation:

- For elements with an array size of 100,000, the tool takes approximately eight minutes to generate an A2L file.

2. Non-AUTOSAR A2L Generation:

- For elements with array sizes up to 63,000, the tool takes around 50 seconds to generate an A2L file.

7 Address Extractor (Address Dumper)

RTA-A2L parses the given absolute ELF file in the *DWARF2* format, generated by the HighTec compiler, and extracts the contained address information. The resulting address information is written to the output ADX file in XML format (ADX).

The output ADX file contains all available information (if present) and thus the following XML sub-tags are generated per memory element. Please refer to Section 10.5 for more details.

Memory elements include C-functions, global data elements and file static data elements. The tool will not generate all array elements in the output ADX file, in order to reduce the size of the output file e.g. for an element array[10], only array[0] will be printed in the output file. The address information for remaining elements (array[1], array[2]...) will be calculated on the fly based on the information present for array[0].

In case of file static data elements, they are prefixed by the C-file name.

e.g.

```
main_c%St1.Ar[0].St2e1.Ar2[0].Ar3
St1.Ar[0].St2e1.Ar2[0].Ar3
```

is a static variable in the *main.c* file.

In case insufficient/incorrect options are given, log messages are thrown.

e.g.

Sl.No	Scenarios	Log Messages
1.	When the <i>compiler</i> option is not given.	Compiler "compilerValue" is not recognized.
2.	When an <i>incorrect</i> option for <i>compiler</i> is given.	Compiler "compilerValue" is not recognized.
3.	When output ADX file path is not given.	"adxoutput" argument is not passed, please update the arguments with "adxoutput" to execute Dumper.
4.	When <i>elf</i> file path is not given.	"elffile" argument is not passed, please update the arguments with "elffile" to execute Dumper.

8 A2LHeader Module

8.1 Introduction

The *A2LHeader* generator module generates the A2L file (with *A2ML*, *memory_segments*, *Global if* data blocks) by replacing the address (present in the ADX /ELF/Symbols files) in place of the freemarker template file's API.



Figure 8.1.Overview of A2LHeader Generator

8.2 Input Files

A2L template (freemarker template): A freemarker template file which consists of A2L content and APIs, if required, for address replacement which has to be called from the respective Java classes.

A sample *A2LTemplate.ftl* is shown below:

```

ASAP2_VERSION 1 61
/begin PROJECT P_000 "medc18"
/begin HEADER ""
    VERSION "kommt_noch"
    PROJECT_NO P_000
/end HEADER
/begin MODULE DIM ""
/begin A2ML
/end MEASUREMENT
/begin MEASUREMENT
    SyC_stSub01d
    ""
    UBYTE
    ExeCon_State_02
    1
    100
    0.00
    6.000000000000
    FORMAT "%13.11"
    ECU_ADDRESS 0x${a2lutils.getAddress("SyC_stSub01d")}
/end MEASUREMENT
/end MODULE
/end PROJECT
  
```

These input files will be passed through an options file to the *A2LHeader* Generator module.

8.3 List of APIs Supported by A2LHeader

To call place-holder APIs in the A2L template, prefix "\${a2lutils." and postfix "}

e.g.

```

${a2lutils.getAddress("SyC_stSub01d")}
  
```

8.3.1 Place-Holder APIs Supported for A2LHeader

getAddress("labelName") or getAddress("expression")
 getMapAddress("labelName") or getMapAddress("expression")
 getMemSegName(1, "Dst", "labelName") or getMemSegName(1, "Dst", "expression")
 getMemSegStartAddress(1, "labelName") or getMemSegStartAddress(1, "expression")
 getMemSegLength(1, "labelName") or getMemSegLength(1, "expression")

8.3.2 APIs for Fetching Address

getAddress("labelName") or getAddress("expression"): Returns the address of the label or the expression. The argument to this API is a string object and the return type is a hexadecimal address value in the form of a string object. This string object is a label name whose corresponding hexadecimal address value is fetched from the ADX/ELF/Symbols file by the *A2LHeader* Module and written in the generated A2L file.

8.3.2.1 APIs for Dataset Area

getMemSegName(1, "Dst", "labelName") or getMemSegName(1, "Dst", "expression"): Returns the memory section name of the dataset area.

e.g.

```
/begin MEMORY_SEGMENT
  getMemSegName(1,"Dst", "0x8000") "" DATA EPROM EXTERN
  getMemSegStartAddress(1,"0x9000")
  getMemSegLength(1,"0x1000") -1 -1 -1 -1 -1
```

Here, the name is constructed by the prefix "Dst" appended with the address of the label or expression without "0x". The dataset area name is *Dst8000* in the above memory segment.

getMemSegStartAddress(1, "labelName") or getMemSegStartAddress(1, "expression"): Returns the map address of the dataset area.

e.g.

```
/begin MEMORY_SEGMENT
  getMemSegName(1,"Dst", "0x8000") "" DATA EPROM EXTERN
  getMemSegStartAddress(1,"0x9000")
  getMemSegLength(1,"0x1000") -1 -1 -1 -1 -1
```

The returned address is nothing but the map address of the dataset area. In the above example, the map address is *9000*.

getMemSegLength(1, "labelName") or getMemSegLength(1, "expression"): Returns the length of the dataset area.

e.g.

```
/begin MEMORY_SEGMENT
  getMemSegName(1,"Dst", "0x8000") "" DATA EPROM EXTERN
  getMemSegStartAddress(1,"0x9000")
  getMemSegLength(1,"0x1000") -1 -1 -1 -1 -1
```

The returned address is nothing but the length of the dataset area. In the above

example, the length of the dataset area is *1000*.

8.3.3 API for Fetching Map Address

getMapAddress("labelName") or getMapAddress("expression"): The address is calculated by the formula (address for "labelName" or "expression" in EPROM) - (start address of dataset area) + (map address of dataset area).

The start address and map address are taken from the dataset area upon which the address of "labelName" or "expression" falls in range.

- Determine the address for "labelName" or "expression" in EPROM (i.e. the address from the ADX file).
- Determine the dataset area which contains the address of "labelName" or "expression".
- If the dataset area has a map address then the address of X is (address for "labelName" or "expression" in EPROM) - (start address of dataset area) + (map address of dataset area).
- If the dataset area has no map address then the address of "labelName" or "expression" is just its address in EPROM (i.e. the address from the ADX/ELF /Symbols file).

Detailed explanation of getMapAddress("labelName") or getMapAddress("-expression") with examples.

- **Dataset area with map address:**

A2L representation dataset area:

```
/begin MEMORY_SEGMENT
  Dst8000 "" DATA EPROM EXTERN 0x9000 0x1000 -1 -1 -1 -1
```

With place holders:

```
/begin MEMORY_SEGMENT
  getMemSegName(1,"Dst", "0x8000") "" DATA EPROM EXTERN
  0x${a2lutils.getMemSegStartAddress(1,"0x9000")} 0x${a2lutils.getMemSegLength(1,"0x1000")} -1 -1 -1 -1
```

The address for ABC in EPROM (from the ADX file) = 0x8002 (which falls in dataset 1, Range: 0x8000, 0x9000(0x8000 + 0x1000 = 0x9000)) getMapAddress("ABC"): 0x8002 - 0x8000 + 0x9000 = 9002 (according to the formula (address for ABC in EPROM) - (start address of dataset area) + (map address of dataset area))

- **Dataset area without map address:**

A2L representation dataset area:

```
/begin MEMORY_SEGMENT
  Dst7000 "" DATA EPROM EXTERN 0x7000 0x1000 -1 -1 -1 -1
```

With place holders:

```

/begin MEMORY_SEGMENT
  getMemSegName(2, "Dst", "0x7000") "" DATA EPROM EXTERN 0x${a2lutils.getMemSegStartAddress(2,
  "0x7000")} 0x${a2lutils.getMemSegLength(2, "0x1000")} -1 -1 -1 -1

```

The address for ABC in EPROM (from the ADX file) = 0x7002 (Which falls in dataset 2, Range: 0x7000, 0x8000 (0x7000 + 0x1000 = 0x8000) getMapAddress("ABC"): 0x7002 - 0x7000 + 0x7000 = 7002 (according to the formula (address for ABC in EPROM) - (start address of dataset area) + (map address of dataset area))

Example A2ltemplate ftl file which is provided as template to A2L Header module:

```

ASAP2_VERSION 1 60
/begin PROJECT MDG1C "MDG1"
/begin HEADER ""
VERSION "MG1CS042_B020I1_EPFADTX"
PROJECT_NO MDG1C
/end HEADER
${a2lutils.startA2LHeader()}
/begin MODULE DIM ""
/begin A2ML
/begin MEMORY_SEGMENT
${a2lutils.getMemSegName(1,"Dst", "BSS_BASE")}
""
DATA EPROM EXTERN
0x${a2lutils.getMemSegStartAddress(1,"BSS_BASE")}
0x${a2lutils.getMemSegLength(1,"0x1000")}
-1 -1 -1 -1 -1
/end MEMORY_SEGMENT

/begin MEMORY_SEGMENT
${a2lutils.getMemSegName(2,"Dst", "0x8000")}
""
DATA EPROM EXTERN
${a2lutils.getMemSegStartAddress(2,"0x9000")}
${a2lutils.getMemSegLength(2,"0x1000")}
-1 -1 -1 -1 -1
/end MEMORY_SEGMENT
/begin MEMORY_SEGMENT
${a2lutils.getMemSegName(3,"Ast", "0xFFFF6000")}
""
DATA EPROM EXTERN
0x${a2lutils.getMemSegStartAddress(3,"0xFFFF6000")}
0x${a2lutils.getMemSegLength(3,"0XA000")}
-1 -1 -1 -1 -1
/end MEMORY_SEGMENT
/begin MEMORY_SEGMENT
${a2lutils.getMemSegName(4,"Cst", "0x500")}
""
DATA EPROM EXTERN
${a2lutils.getMemSegStartAddress(4,"0x3000")}
${a2lutils.getMemSegLength(4,"0x2000")}
-1 -1 -1 -1 -1
/end MEMORY_SEGMENT
/begin MEMORY_SEGMENT
${a2lutils.getMemSegName(5,"Ast", "0xD0006000")}
""
DATA EPROM EXTERN
0x${a2lutils.getMemSegStartAddress(5,"0xE0007000")}
0x${a2lutils.getMemSegLength(5,"0X2000")}
-1 -1 -1 -1 -1
/end MEMORY_SEGMENT
/begin MEMORY_SEGMENT
${a2lutils.getMemSegName(6,"Ast", "0xCFFF8000")}
""
DATA EPROM EXTERN
0x${a2lutils.getMemSegStartAddress(6,"0xDFFF8000")}
0x${a2lutils.getMemSegLength(6,"0X2000")}
-1 -1 -1 -1 -1
/end MEMORY_SEGMENT
/begin MEMORY_SEGMENT
${a2lutils.getMemSegName(7,"Ast", "0x20 + 0x30")}
""
DATA EPROM EXTERN
0x${a2lutils.getMemSegStartAddress(7,"0x10 + 0x50")}
0x${a2lutils.getMemSegLength(7,"0x20 + 0x10")}
-1 -1 -1 -1 -1
/end MEMORY_SEGMENT
/end A2ML
---
---
getAddress("ASS_BASE + 0x200") ==> 0x${a2lutils.getAddress("ASS_BASE + 0x200")}
getAddress("BSS_BASE + DATA_BASE") ==> 0x${a2lutils.getAddress("BSS_BASE + DATA_BASE")}
getAddress("DATA_BASE - BSS_BASE") ==> 0x${a2lutils.getAddress("DATA_BASE - BSS_BASE")}
getMapAddress("DATA_BASE - BSS_BASE") ==> 0x${a2lutils.getMapAddress("DATA_BASE - BSS_BASE")}
getMapAddress("0x7002 + 0x000A") ==> 0x${a2lutils.getMapAddress("0x7002 + 0x000A")}
getMapAddress("0x20 + 0x40") ==> 0x${a2lutils.getMapAddress("0x20 + 0x40")}
getMapAddress("0xA0 - 0x40") ==> 0x${a2lutils.getMapAddress("0xA0 - 0x40")}
getMapAddress("0xA0 - 0x40 + 0x10") ==> 0x${a2lutils.getMapAddress("0xA0 - 0x40 + 0x10")}
---
---
${a2lutils.stopA2LHeader()}
/end MODULE
/end PROJECT

```

9 A2L Merger Module

A2L Merger is a sub-module of RTA-A2L and is used for merging master A2L and AUTOSAR A2L snippet files into a single A2L file.

A2L Template and the A2L snippet are merged with the address coming from ADX/ELF/Symbols.

If any of the ADX/ELF/Symbols input is not passed, then the address will be updated with an 0x0 address in the A2L file.

In case the input is ADX or ELF:

- If the address for A2L label is missing in the ADX/ELF, then the label is not considered for merging in final A2L.

In case the input is Symbols:

- If the address for the A2L label is missing in the Symbols, then the label is considered for merging with the existing address from the input label.

9.1 Fetching Address for an A2L Element

While merging the A2Ltemplate and the AUTOSAR A2L snippet, the A2L Merger module will fetch the address of the AUTOSAR labels from the ADX/ELF/Symbols file provided in the input.

The ADX/ELF/Symbols file consists of the address information of the A2L labels.

If the address of the A2L label name is not present, then A2L Merger will not consider the label in the final A2L file.

9.2 ADXProcessor

RTA-A2L merges the address for A2L labels. ADXProcessor is a sub-module of RTA-A2L, which takes the ADX from the ADX extractor tool (containing the Project's address information) and mapping ADX files (*-custadllist*, containing offset and/or mapping information as inputs) and generates a merged ADX file (*-adxFile*).

e.g.

ADX from ADX extractor tool:

```
<MEMORY-ELEMENT>
  <LABEL-NAME>SampleCElement</LABEL-NAME>
  <CATEGORY>DATA</CATEGORY>
  <ABSOLUTE-ADDRESS>0x40005678</ABSOLUTE-ADDRESS>
  <ROOT-OFFSET>0x0</ROOT-OFFSET>
  <SIZE>4</SIZE>
  <COMP-UNIT-NAME>ddrc_auto_conf.c</COMP-UNIT-NAME>
  <COMP-UNIT-DIR>_out</COMP-UNIT-DIR>
</MEMORY-ELEMENT>
```

Mapping ADX file Entry:

```
<MEMORY-ELEMENT>
  <LABEL-NAME> SampleCElement<LABEL-NAME>
  <MAP-LABEL-NAME>SampleA2LName</MAP-LABEL-NAME>
</MEMORY-ELEMENT>
```

Final ADX file entry:

```

<MEMORY-ELEMENT>
  <LABEL-NAME> SampleCElement<LABEL-NAME>
  <CATEGORY>DATA</CATEGORY>
  <ABSOLUTE-ADDRESS>0x40005678</ABSOLUTE-ADDRESS>
  <ROOT-OFFSET>0x0</ROOT-OFFSET>
  <SIZE>4</SIZE>
  <MAP-LABEL-NAME>SampleA2LName </MAP-LABEL-NAME>
  <COMP-UNIT-NAME>ddrc_auto_conf.c</COMP-UNIT-NAME>
  <COMP-UNIT-DIR>_out</COMP-UNIT-DIR>
</MEMORY-ELEMENT>

```

Where:

<MAP-LABEL-NAME> : A2L label name <LABEL-NAME> : Corresponding C-element name

9.3 Merging the A2L

The A2L Merger module performs the following operations:

1. Replaces the addresses in the AUTOSAR A2L file(s) with the correct addresses from the ADX/ELF/Symbols file.
2. Merges the AUTOSAR A2L files into a single A2L file.

Entries are merged into the A2L template/main A2L file:

1. MEASUREMENT
2. CHARACTERISTIC
3. AXIS_PTS
4. COMPU_METHOD
5. COMPU_TAB
6. COMPU_VTAB
7. COMPU_VTAB_RANGE
8. RECORD_LAYOUT
9. GROUP
10. FUNCTION
11. SYSTEM_CONSTANT
12. UNIT

9.3.1 Ignore Address Update for VIRTUAL_CHARACTERISTIC and VIRTUAL MEASUREMENTS

According to the concept, neither the *VIRTUAL*, *MEASUREMENT* nor the *CHARACTERISTIC* should have any address in the memory. So, if there is no address entry in *ADX/ELF/Symbols* for that particular *MEASUREMENT* or *CHARACTERISTIC*, then the tool does not abort and that specific block will be considered for merging.

According to the above concept, the A2L Merger module behavior is as follows:

1. A2L Merger should ignore the address update for *VIRTUAL MEASUREMENTS* and *CHARACTERISTICS*.

2. It will merge the *MEASUREMENT* and the *CHARACTERISTIC* with the original address from the AUTOSAR A2L file in the final merged A2L file. The ADX/ELF/Symbols file entry for that particular block will not be considered.
3. Supposing there is no *ECU_ADDRESS* available for the *MEASUREMENT* in the AUTOSAR A2L file then, in the final merged A2L file, there will be no address update for that *MEASUREMENT*. In the case of *CHARACTERISTICS*, by default the address will be available, hence it will be updated with the same address in the final A2L file.

9.3.2 Customer A2L Record Layouts Merging

- When merging A2L files, set the switch of the *-enableCustomerA2LAlignment* to *y* will include alignment sizes from the *MOD_COMMON* block of the customer A2L file into the customer A2L record layout after merging.
- The alignment sizes in the customer record layouts are updated, but only when they are referred from *AXIS_PTS* or *CHARACTERISTIC* blocks within the customer A2L file.
- If there are duplicate record layouts from MAIN and Customer A2Ls, the names of the customer record layouts will be suffixed with *_custAlignment_s1|_custAlignment_s2* for those with updated alignments from multiple customer A2L files.

9.3.3 Input A2L Files Encoding

Output A2L files are generated in UTF-8 BOM encoding. Encoding of the input A2L files is logged as an Info message. In some scenarios, RTA-A2L might not be able to detect the correct encoding of the input file; it will be processed with UTF-8 encoding and some wrong characters might be observed in the output file generated by RTA-A2L.

A warning message is logged when RTA-A2L cannot detect encoding of the input files. The User can provide them with the correct encoding format using the options *--encoding_UTF_8*, *--encoding_US_ASCII* and *--encoding_ISO_8859_1*. (Please refer to Section 3.2 for more details.)

9.4 Namespace Validations

- In an *ASAP2* file, there are different object types which can be referenced via their object names. To be able to resolve these references, it is necessary to have unique identifiers.
- The unambiguousness of identifiers is not required in the whole *ASAP2* file, but within special namespaces.
- Namespace validation is applicable for the following A2L elements:
 - Within one *MODULE*, each *FUNCTION* identifier must be unique.
 - Within one *MODULE*, each *GROUP* identifier must be unique.
 - Within one *MODULE*, each measure and adjustable object should be unique. The keywords for measure and adjustable objects which are supported by the A2LParser are *MEASUREMENT*, *AXIS_PTS* and

CHARACTERISTIC.

- Within one *MODULE*, each *COMPU_METHOD* identifier must be unique.
 - Within one *MODULE*, each conversion table identifier must be unique.
Relevant keywords for conversion tables are *COMPU_TAB*,
COMPU_VTAB and *COMPU_VTAB_RANGE*.
 - Within one *MODULE*, each *MEMORY_SEGMENT* identifier must be unique.
 - Within one *MODULE*, each *RECORD_LAYOUT* identifier must be unique.
- If the namespaces are not according to the ASAM specification e.g. if *COMPU_METHODs* within a module do not have unique names, then the parsing of the A2L file would fail, an *A2LBuilderException* would be thrown, a *SAPException* would be thrown and RTA-A2L would abort.

9.5 Merger for CPFlex Use Case

CPFlex Merger is used to merge the cluster and host A2L files. Multiple cluster A2L files will be generated from multiple cluster builds, with the correct addresses updated. These multiple cluster A2L files will be merged with the host A2L file using *CPFlex Merger*.

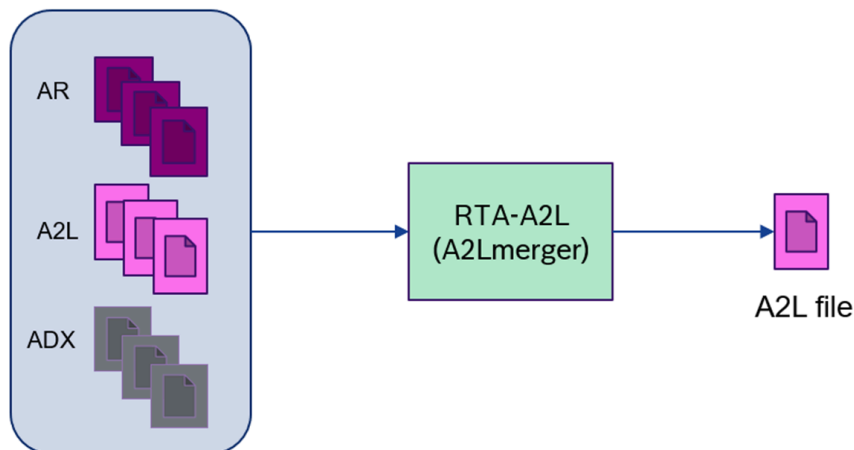


Figure 9.1.CPMerger block diagram

Sample cluster mapping file content for the input option `--cluster_mapping_file`:

```

Cluster1.a2l=cluster1.adx=cluster1.arxml
Cluster2.a2l=cluster2.adx=cluster2.arxml
...
Host.a2l=host.adx=host.arxml
  
```

- Cluster A2L files can be passed to the A2L Merger module with the option `--cluster_mapping_file`, which would contain the mapping information of all the cluster A2L, ADX and binary manifest filepaths.
- In *CPFlex Merger*, the addresses for the cluster A2L file elements will not be updated from the ADX file. Instead, the original address would be retained for the cluster and host A2L elements. The host A2L, ADX and binary manifest are identified based on the *CATEGORY* of *CP-SOFTWARE-CLUSTER-BI-*

NARY-MANIFEST-DESCRIPTOR with value *HOST_SW_CLUSTER*. The host A2L is the main A2L file for the A2L Merger Module.

- The ADX file contains the address for the cluster element in the above use case; even then, the cluster A2L element address would be retained as it is in the cluster A2L file.
- In the case of CPFlex Merger, even the AUTOSAR A2L file(s) can be passed as input and the behavior of merging is not changed here. The addresses of the AUTOSAR A2L elements will still be updated from the ADX file.
- *A2LMERGE-I-PREFIX-POSTFIX-FOR-CLUSTER-ELEMENTS* will contain the message as "file name of the cluster A2L file will be prefixed/postfixed based on the input parameter provided for *-clusterElementNameConflicts* = prefix / postfix and by default postfix is considered".
- Additionally, a warning message is logged for each cluster A2L file with a name in conflict with either the host or other cluster A2L file(s). An *A2LMERGE-W-NON-UNIQUE-CLUSTER-ELEMENT* will contain the message "Cluster element <element_name> in <cluster_file_path> is having same name as other element in <other_cluster_file_path> / <host_a2l_path>".

9.5.1 Cluster A2L Merger: Ignore Receiver Parameters Use Case

When the *--cluster_mapping_file* option is enabled, it includes a list file to specify the mapping information of all the cluster A2L, ADX and binary manifest files. The host A2L, ADX and binary manifest are identified based on the *CATEGORY* of *CP-SOFTWARE-CLUSTER-BINARY-MANIFEST-DESCRIPTOR* with value *HOST_SW_CLUSTER*. The host A2L is the main A2L file for the A2L Merger Module.

- To identify the receiver parameter, process the *SHORT-NAME* of the *BINARY-MANIFEST-ITEM* of both *BINARY-MANIFEST-PROVIDE-RESOURCE* (*MEASUREMENTs* in the A2L file) and *BINARY-MANIFEST-REQUIRE-RESOURCE* (*CHARACTERISTICs* in the A2L file), with the category as *PROVIDER_HANDLE* and its global resource ID and *RESOURCE-GUARD-VALUE*.
- While merging the A2L elements from the main(host)/cluster A2L files, ignore the require resources(*CHARACTERISTICs*) which have a corresponding provider resource with the same global resource ID and resource guard value.
- If the *BINARY-MANIFEST-REQUIRE-RESOURCE* is a *STRUCTURE* or *ARRAY* element, then all its *subElements* are processed and ignored during merge.
- <BINARY-MANIFEST-ITEM>'s <SHORT-NAME> is available as <SYMBOL> in the *McSupport.arxml* file i.e. it would be available as <LABEL-NAME> in the ADX file (which is the C-name).

e.g.

Binary manifest sample (*SwCluC_Cfg_Abs_BManif.arxml* file):


```

<BINARY-MANIFEST-PROVIDE-RESOURCE>
<SHORT-NAME>BMPR_pre_Bernd</SHORT-NAME>
<GLOBAL-RESOURCE-ID>7003</GLOBAL-RESOURCE-ID>
<ITEMS>
  <BINARY-MANIFEST-ITEM>
    <SHORT-NAME>Rte_Rips_XccSR_Bernd</SHORT-NAME>
    <CATEGORY>PROVIDER_HANDLE</CATEGORY>
    <ADDRESS>0x60008100</ADDRESS>
    <SYMBOL>SwCluC_BManif_OfferedInterfaceHandleColumn[19].dptr</SYMBOL>
  </BINARY-MANIFEST-ITEM>
</ITEMS>
<RESOURCE-GUARD-VALUE>4076315036</RESOURCE-GUARD-VALUE>
....
</BINARY-MANIFEST-PROVIDE-RESOURCE>

```

McSupport.arxml file sample (*Rte_Rips_XccSRMcSupportData.arxml*):

```

<MC-DATA-INSTANCE>
<SHORT-NAME>Bernd</SHORT-NAME>  -> A2L name
.....
<SYMBOL>Rte_Rips_XccSR_Bernd</SYMBOL>  -> C-name or binary manifest item ShortName
</MC-DATA-INSTANCE>

```

When this option is enabled, `--ma2l` and `--adxfile` are not only mandatory options. And if two binary manifest files are found with the category set as *HOST_SW_CLUSTER* then the tool has to be aborted.

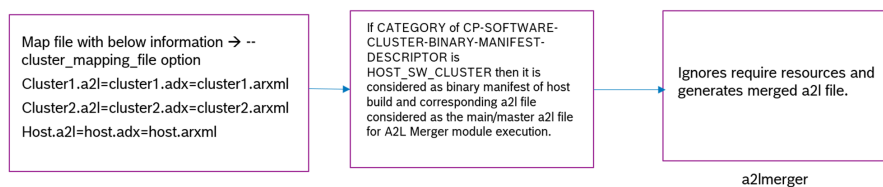


Figure 9.2.CPFlex Merger

9.5.2 Merging of Memory Segments

- The memory segments under the *MOD_PAR* block of cluster A2L files are merged to the memory segments from the host A2L file.
- The memory segments are merged only in the case of cluster A2L files (CPFlex Merger). Memory segments would not be merged in the case of the AUTOSAR A2L file.

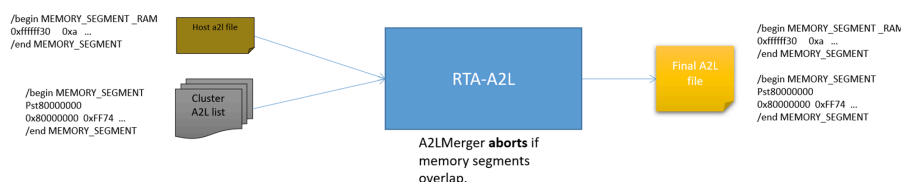


Figure 9.3.Memory Segment Merger

- If the memory segment name is not unique, either in two-cluster A2L files or in a cluster and host A2L file, then a message *A2LMERGE-E-MEMORY-SEGMENT-NOT-UNIQUE* is logged with a message as memory segment < memo-

ry_segment_name > defined in the cluster A2L file. If the A2L {path} has duplicate entries in the main A2L file, or any other cluster A2L file, then the tool will be aborted.

- If the address range is overlapping for the memory segments in the cluster and host A2L file, then the tool will be aborted by logging *A2LMERGE-E-OVERLAPPING-MEMORY-SEGMENT* with the message "Memory segment <overlapping_memory_segment_name> in the A2L {path} is having overlapping address range with the A2L memory segment <memory_segment_name> defined in either master or any other cluster A2L files". The tool will be aborted in the case of overlapping address range.

9.5.3 CPFlex - Linker Invocation File Generation Specific to Cluster

9.5.3.1 Introduction

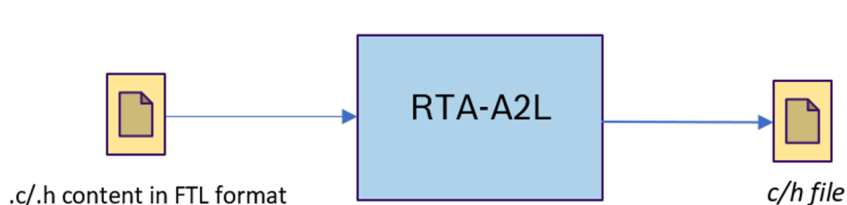
The Linker Invocation file generation module generates the required output file (e.g. .h, .c) from the template provided with option `--genericfiletemplate` of the tool input. The tool supports replacing the address of the elements (functions, elements) from the ADX file. The Linker Invocation file generation module can act as a standalone tool.

9.5.3.2 Overview of Linker Invocation File Generation Specific to Cluster

The tool will generate the .h file by replacing the function address (present in the ADX file) in place of the API of the Java file. This is developed for supporting the CPFlex project.

For a CPFlex project, the Software Cluster (SWCL) needs a specific Linker Invocation File required to create the Hex file for the SWCL by using the Software Cluster Build. The specific Linker Invocation Files (.h) for a certain Software Cluster will be used to build the Cluster separately.

The .h file content will be passed as a template to the A2L Header module. The template will contain place holders for the functions.



In the above diagram, the .h/.c file content will be passed as a template to the Linker Invocation file generation. The template will contain place holders for the functions. The tool will be able to parse and read ADX information which contains function addresses. A new output file will be generated with the function addresses.

9.5.3.3 Input Files

The following files will be the input to build the generic file generator. They are:

1. The ADX file from the cluster build (please refer to Section 10.5 for more details) with the addresses of the functions.
2. The template file (freemarker template) which, it is expected, will consist of the contents of the .h/.c files, with the corresponding expressions for fetching the function address:

Template.ftl template file from a specific cluster

```
#include "OsMinusConfig.h"
#include "IKernel.h"
#define OS_START_SEC_CODE_LIB
#include "Os_MemMap.h"

void external_function_proxy() {
    void ((*func_ptr)()) func_ptr = (cast) 0x${a2lutils.getFunctionAddress("externalFunctionName")};
    func_ptr();
}
#define OS_STOP_SEC_CODE_LIB
#include "Os_MemMap.h"
```

These input files will be passed through an options file to the RTA-A2L.

9.5.3.4 API supported by Linker Invocation File Generation

Placeholder APIs, which are supported for the generic file generator (apart from this *A2LHeader*) can be used.

```
getFunctionAddress("FUNC_NAME")
```

getFunctionAddress: The argument to this API is a string object and the return type is a hexadecimal address value in the form of a string object. This string object is a function name, whose corresponding hexadecimal address value is fetched from the final ADX file by *A2Lheader* and written in the output file.

10 File Formats

10.1 A2L Output File

RTA-A2L writes *ASAM MCD-2 1.61 A2L*.

10.2 MCSD File

RTA-A2L accepts AUTOSAR Measurement and Calibration Support Data from AUTOSAR 4.x. RTA-A2L will accept any well-formed ARXML file with AUTOSAR Schema validations.

10.3 Symbols File

The symbols file is a flat 8-bit text file containing one hexadecimal address and one corresponding symbol on each line.

Addresses must be hexadecimal and preceded by 0x. They are matched by the regular expression *0x[0-9A-Fa-f]+*. Valid symbols match the regular expression *[A-Za-z_][A-Za-z0-9_]*.

e.g.

```
0x00000001 CanIf_Init_Flag
0x00000002 CanIf_Priv_ControllerState_ast
```

10.4 Template File

A freemarker template file (*ftl*) consists of ASAM A2L content and APIs required for address replacement. These APIs are options and, if present, the address replacement will be done.

By default, RTA-CAR will provide a sample template during project configuration. This template can be edited according to the needs of the project.

e.g.

```
ASAP2_VERSION 1 61
/begin PROJECT SamplePrj "SamplePrj"
/begin HEADER ""
    VERSION "SamplePrj"
    PROJECT_NO SamplePrj
/end HEADER
/begin MODULE DIM_CUST ""
/end MODULE
/end PROJECT
```

10.5 ADX File

Sl.No	Tags	Explanation
1.	<MEMORY-ELEMENT>	Specifies an element in memory. This includes C-functions, global data elements and file static data elements.
2.	<LABEL-NAME>	C-name of a memory element.

3.	< CATEGORY >	Category of a memory element.
4.	< ABSOLUTE-ADDRESS >	Absolute address of a memory element (hexadecimal).
5.	< ROOT-OFFSET >	Offset of a memory element (e.g. sub element of a structure) to a corresponding root memory element (hexadecimal).
6.	< SIZE >	Size of a memory element in bytes (decimal).
7.	< MAP-LABEL-NAME >	(If present) the corresponding A2L name (identifier).
8.	< BIT-POSITION >	(In case of a bit) the position of a bit relative to absolute address.
9.	< NUMBER-OF-BITS >	(In case of a bit) the number of bits, only applicable for category DATA.
10.	< ARRAY-NBR-ELEMENTS >	(In case of an array) the number of array elements, only applicable for category ARRAY.
11.	< ARRAY-ELEMENTS-SIZE >	(In case of an array) the size of one array element, only applicable for category ARRAY.
12.	< COMP-UNIT-NAME >	(If present) the name of the C-file where the memory element is defined.
13.	< COMP-UNIT-DIR >	(If present) the path of the C-file where the memory element is defined.
14.	< STACK-CONSUMPTION >	Allocated stack of a function in bytes (decimal), only applicable for category FUNCTION.
15.	< CALLED-SYMBOLS >	Container for < CALLED-SYMBOL > tags, only applicable for memory elements having category FUNCTION.
16.	< SH-INDEX >	(In case of root elements) the index of the section to which this memory element is assigned.
17.	< DEMANGLED-NAME >	The original name of a function (demangled), only applicable for memory elements having category FUNCTION.

Address dumpers generate the section element tag with the following sub-tags:

Sl.No	Tags	Explanation
1.	< SECTION-NAME >	Name of section.
2.	< SECTION-TYPE >	Type of section. Values correspond to the <i>sh_type</i> from the ELF specification.

3.	<SECTION-START-ADDRESS>	Address of the section beginning (hexadecimal).
4.	<SECTION-SIZE>	Size of the input/output section (hexadecimal).
5.	<SECTION-INDEX>	Index of the section to which this memory element is assigned. Only applicable for root memory elements.

Example for ADX file

```

<?xml version="1.0"?>
<ADDRESS-CALCULATOR version="1.31.0" spec="1.11">
  <MEMORY-ELEMENT>
    <LABEL-NAME>main_c%St1.Ar[0].St2el.Ar2[0].Ar3</LABEL-NAME>
    <CATEGORY>ARRAY</CATEGORY>
    <ABSOLUTE-ADDRESS>0x8011B2EA</ABSOLUTE-ADDRESS>
    <ROOT-OFFSET>0x02</ROOT-OFFSET>
    <SIZE>125</SIZE>
    <ARRAY-NBR-ELEMENTS>125</ARRAY-NBR-ELEMENTS>
    <ARRAY-ELEMENT-SIZE>1</ARRAY-ELEMENT-SIZE>
    <COMP-UNIT-NAME>main.c</COMP-UNIT-NAME>
    <COMP-UNIT-DIR>C:\myCfiles</COMP-UNIT-DIR>
  </MEMORY-ELEMENT>

  <MEMORY-ELEMENT>
    <LABEL-NAME>Struct.bit1</LABEL-NAME>
    <CATEGORY>DATA</CATEGORY>
    <ABSOLUTE-ADDRESS>0x8011C000</ABSOLUTE-ADDRESS>
    <ROOT-OFFSET>0x00</ROOT-OFFSET>
    <SIZE>1</SIZE>
    <BIT-POSITION>5</BIT-POSITION>
    <NUMBER-OF-BITS>4</NUMBER-OF-BITS>
  </MEMORY-ELEMENT>

  <SECTION-ELEMENT>
    <SECTION-NAME>.ram1_free</SECTION-NAME>
    <SECTION-START-ADDRESS>0x12000000</SECTION-START-ADDRESS>
    <SECTION-SIZE>0x4D20</SECTION-SIZE>
    <SECTION-TYPE>NOBITS</SECTION-TYPE>
  </SECTION-ELEMENT>
</ADDRESS-CALCULATOR>

```

11 ETAS Contact Addresses

11.1 Technical Support

Technical support is available to all users with a valid support contract. If you do not have a valid support contract, please contact your regional sales office.

- <https://www.etas.com/en/contact.php>

The best way to get technical support is using the online support portal RTA Hotline:

- <https://rtahotline.etas.com/>

It is helpful if you can provide technical support with the following information (where relevant):

- Your support contract number
- The version of the ETAS tools you are using
- The version of the compiler tool chain you are using
- The command line (or reproduction of steps) that result in an error message
- The error messages or return codes you received (if any)
- Your RTA-CAR Project or if not possible the arxml file(s) containing the error(s).
- The *Diagnostic.dmp* if it was generated

11.2 ETAS HQ

ETAS GmbH Borsigstraße 24 70469 Stuttgart Germany	Phone:	+49 711 3423-0
	Fax:	+49 711 3423-2106
	WWW:	www.etas.com